

ARIN 5201 & MAIE 5421

Machine Learning for Computer Vision

Wenhan LUO

9/13/2026

Content

Topic	Week
Machine Learning and Computer Vision Basics	
Introduction to Computer Vision	1
Machine Learning Elements	2
Deep Learning	3-4
 Advanced Topics in Computer Vision	
Image Classification with CNNs	5-6
Detection & Segmentation	7
Attention Mechanism & Transformer	8
Generative Models	9-10
Multi-modal Learning	11
Final Project Representation	12-13

Machine Learning Elements

What is Machine Learning?

- How human gains understanding, knowledge and skills? Why we are “intelligent”?
 - Congenital (Born with)
 - By hundreds-of-thousands of years, humans gain the perception, knowledge and skills through interacting with and adapting to the real world. Such adaptation is inherited throughout generations.
 - Acquired (**Learning** after born)
 - Humans obtain new information, in forms of structural knowledge or non-structural data, and improve themselves through a certain objectives.
 - Books
 - Videos
 - Sounds
 - ...
 - **This process makes people much different with each other!**

What is Machine Learning?

- If you want to make a “looks-like” intelligent machine, what would you do?

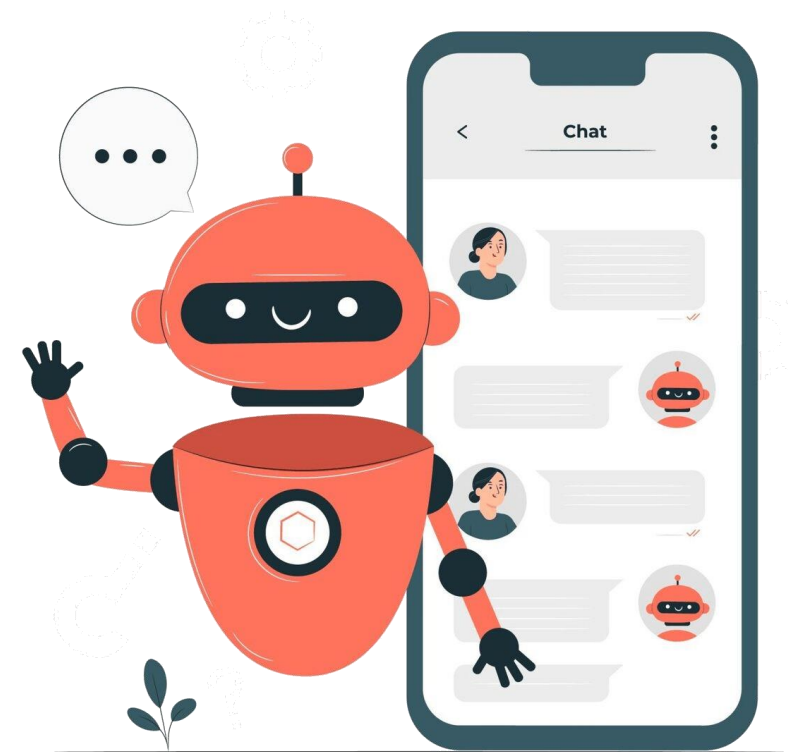
Making a Chatbot

Option A: Rule-based Systems

- Transfer your knowledge and understanding into rules

```
if input == "hello":  
    output = "how are you?"  
elif input == "goodbye":  
    output = "see you later"
```

- **Drawbacks:** The rules can be inaccurate and incomplete! Defining the rules needs a lot of human expertise! (think about weather prediction using mathematical models)



What is Machine Learning?

- If you want to make a “looks-like” intelligent machine, what would you do?

Making a Chatbot

Option B: Data-driven Machine Learning Systems

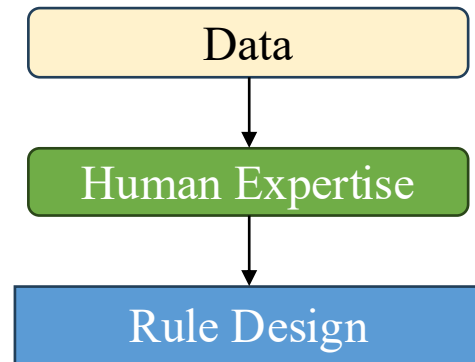
- Making the machine find patterns from data, just like human



- **Much reduced reliance on human knowledge, more accurate, flexible and generalizable.**

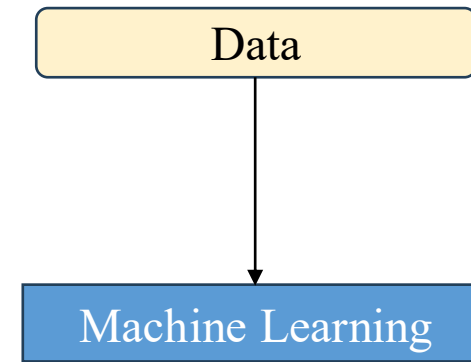
What is Machine Learning?

Rule-based Systems



Knowledge-driven

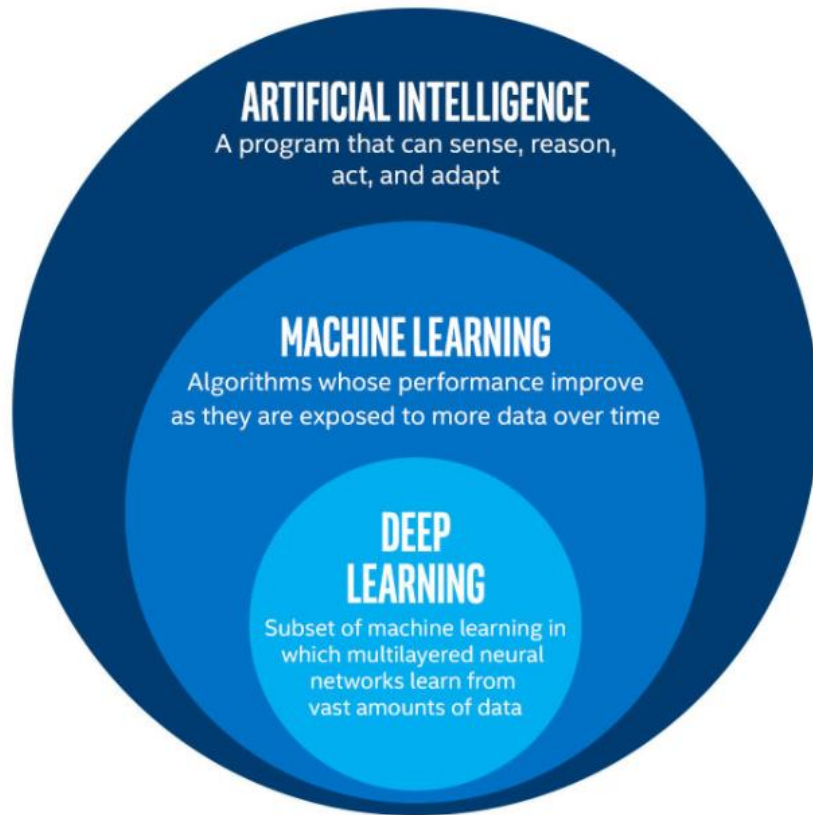
Machine Learning Systems



Data-driven

Machine learning is a **subset** of **artificial intelligence** that involves training algorithms to recognize patterns and make decisions **based on data without being explicitly programmed for specific tasks**. These algorithms improve their performance as **they are exposed to more data over time**.

What is Machine Learning?



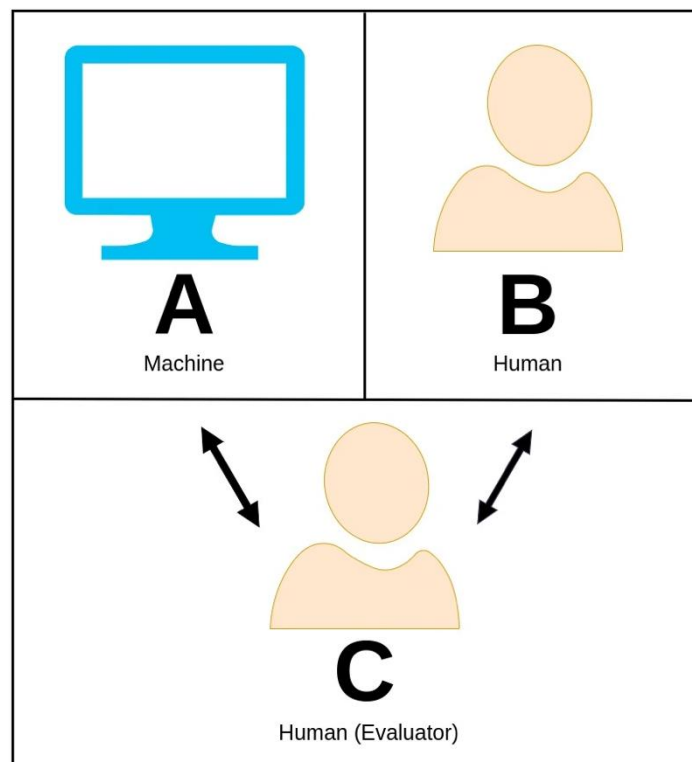
- Some Commonly Discussed Concepts
 - **Artificial Intelligence (AI):** Mimic human cognitive functions like learning, problem-solving, perception, and understanding language. It doesn't always have to learn from data; some AI systems are programmed to follow strict rules (expert systems) or logic (symbolic AI).
 - **Machine Learning (ML):** ML is a subset of AI that focuses specifically on the development of systems that can learn from and make decisions based on data. It involves algorithms that improve their performance at a given task with more data.
 - **Deep Learning (DL):** A further subset of ML that specifically involves neural networks with many layers, hence the term “deep”.

Brief History of Machine Learning

1950s

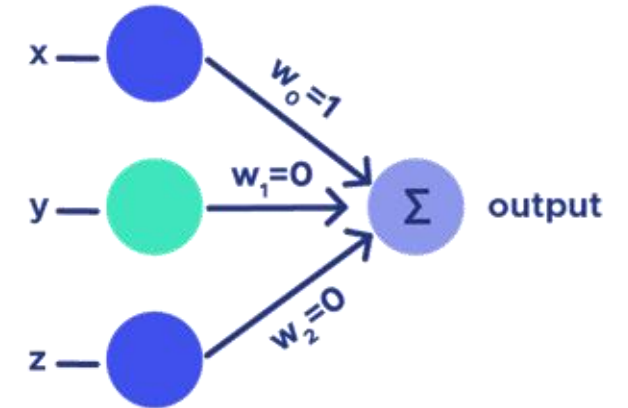
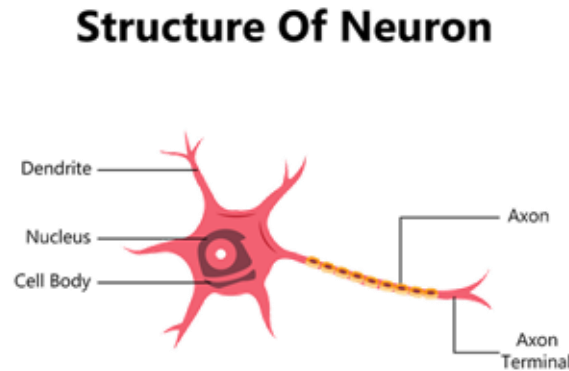
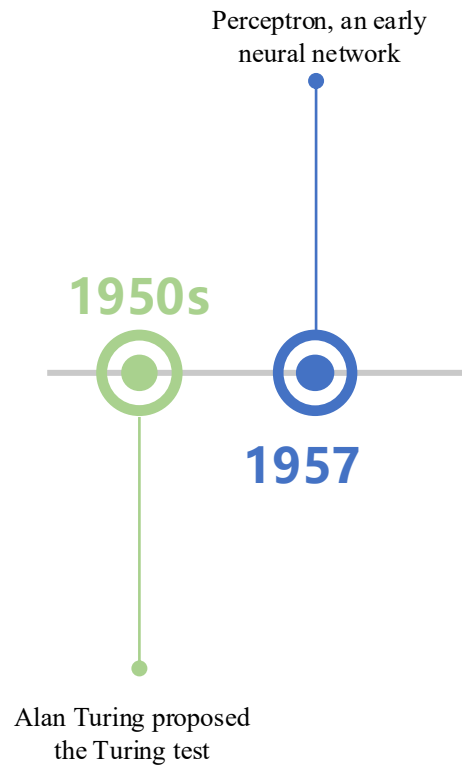


Alan Turing proposed
the Turing test



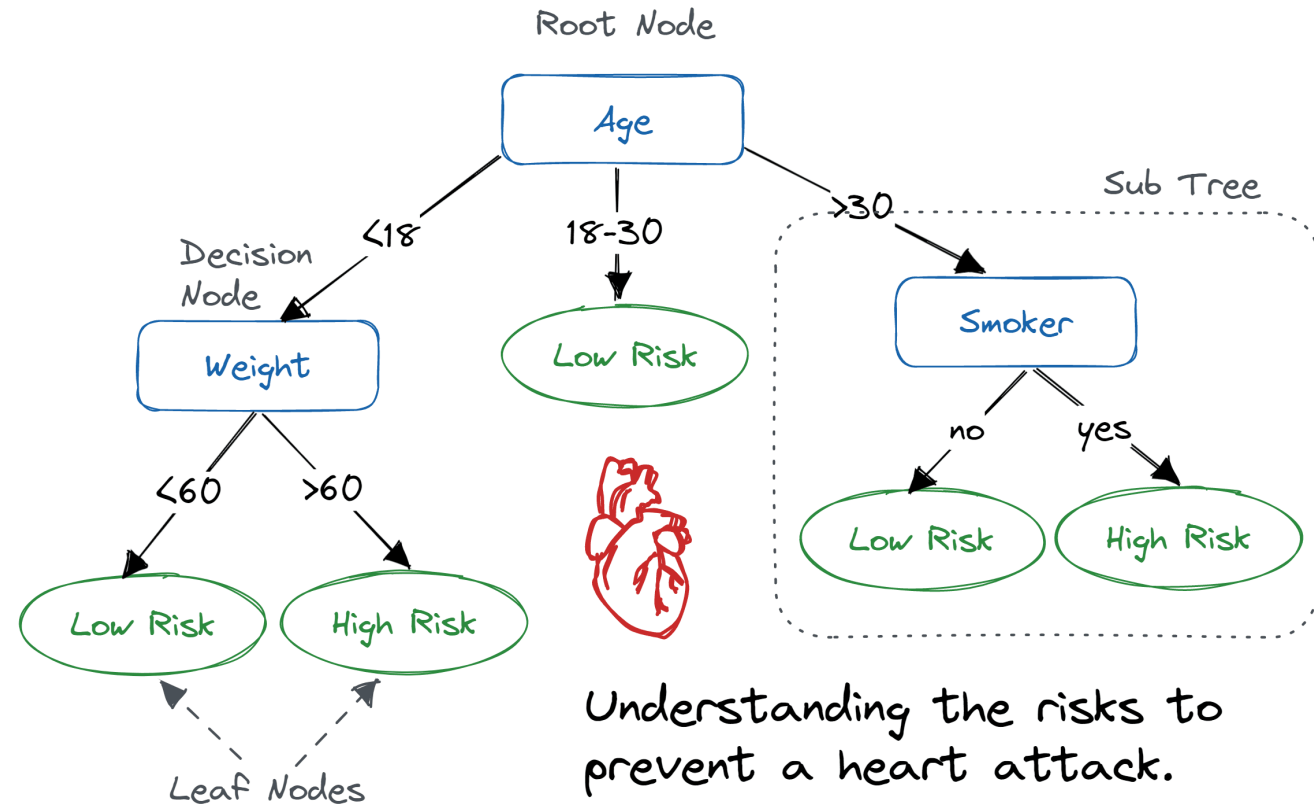
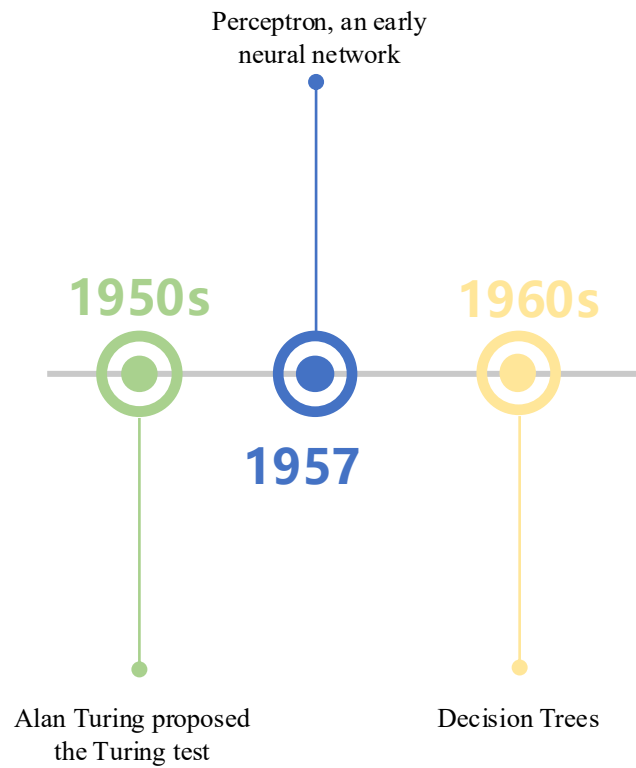
If the evaluator is unable to reliably tell the machine from the human in interactions, the machine is said to have passed the Turing Test.

Brief History of Machine Learning



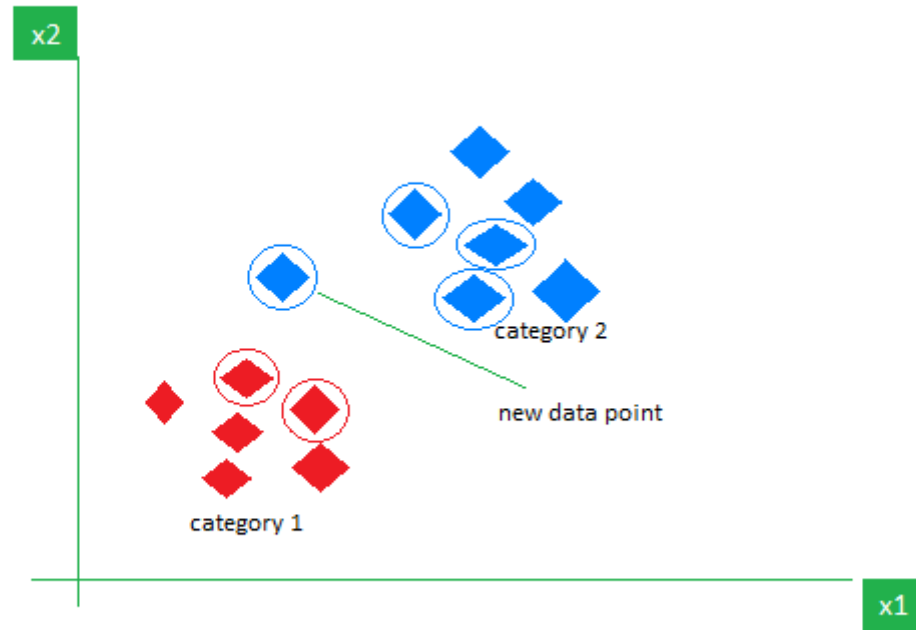
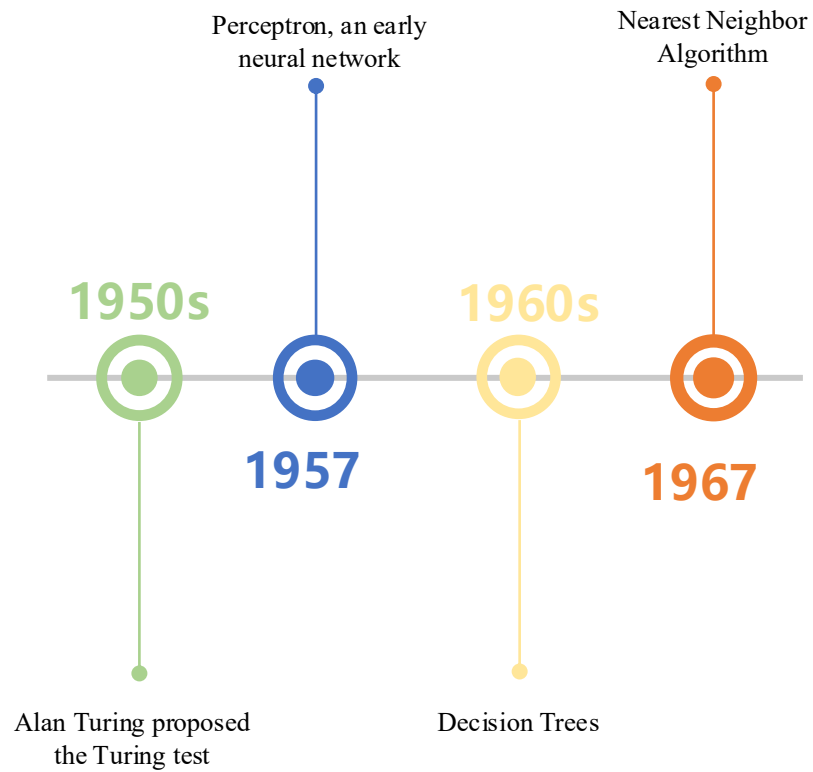
The analogy to human neuron, the foundation of neural networks

Brief History of Machine Learning



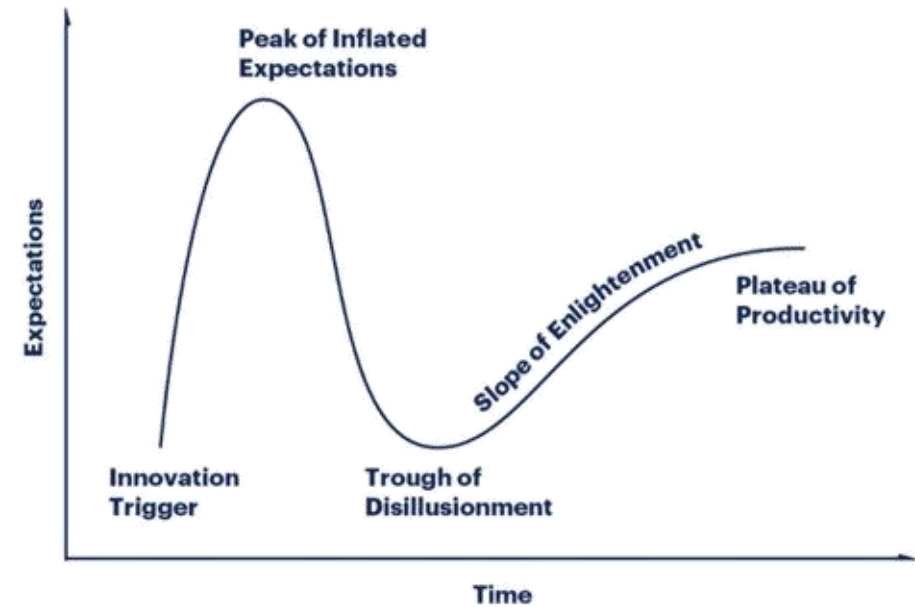
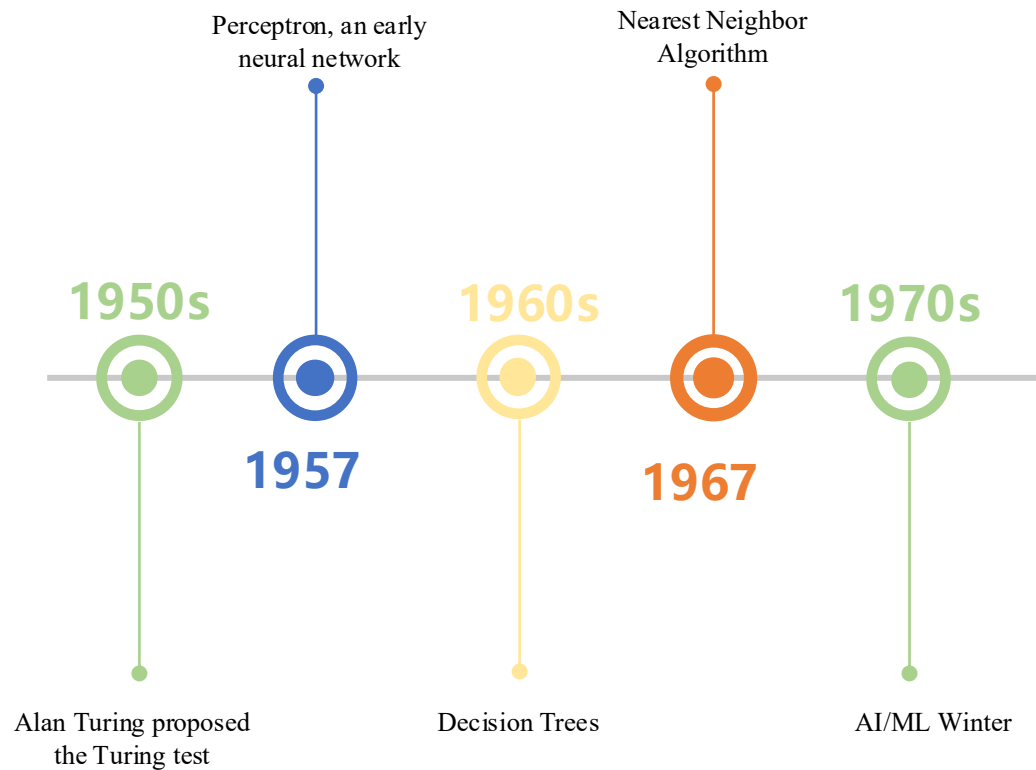
Decision Trees: Making machines to design the rules from data

Brief History of Machine Learning



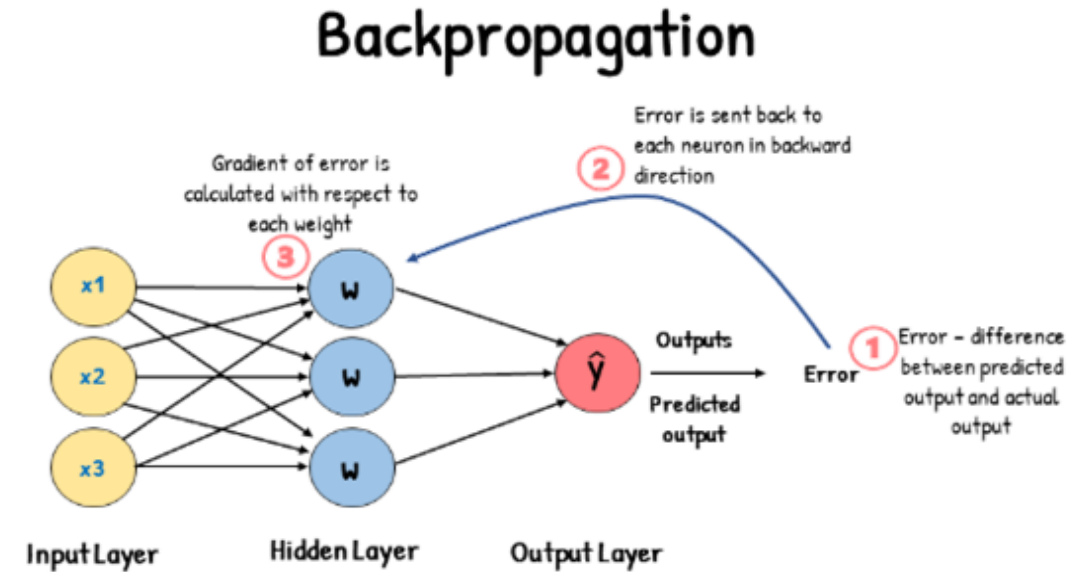
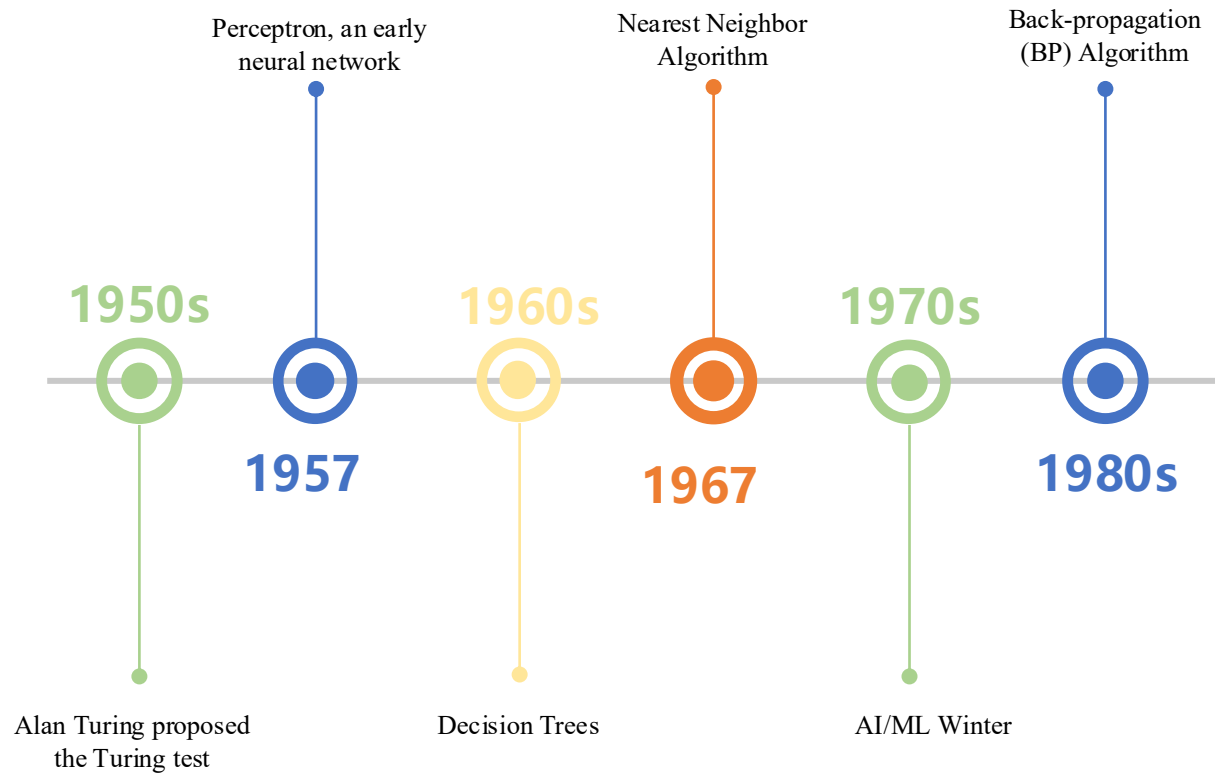
Classifies a new case based on the most similar instances (neighbors) from the training dataset.

Brief History of Machine Learning



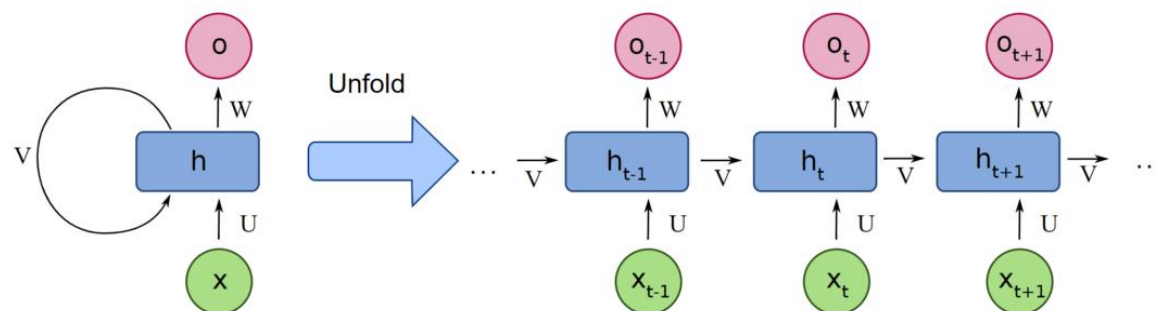
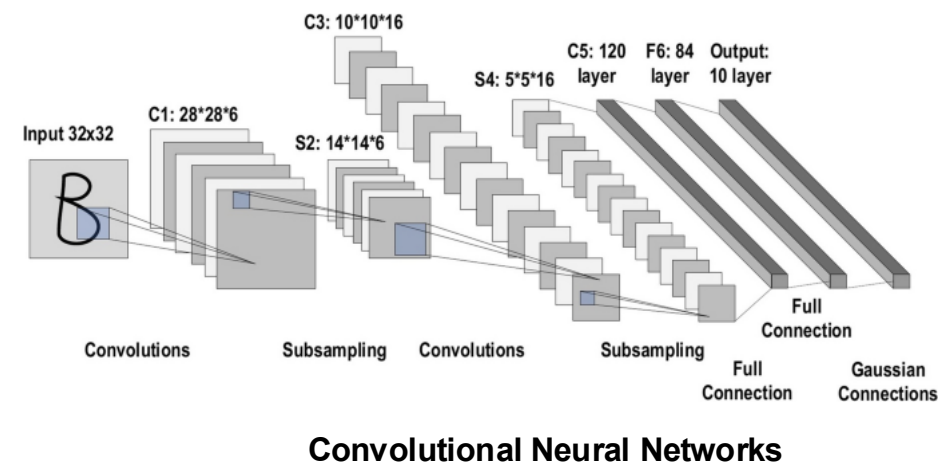
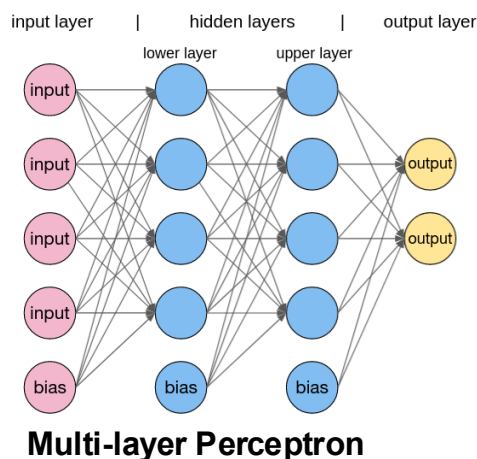
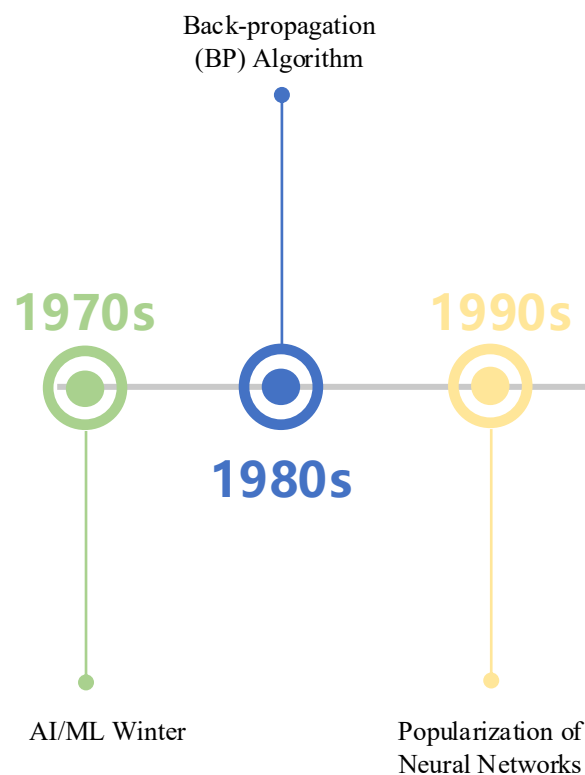
Funding and interest in the field dropped due to high expectations and subsequent disappointment in AI progress

Brief History of Machine Learning

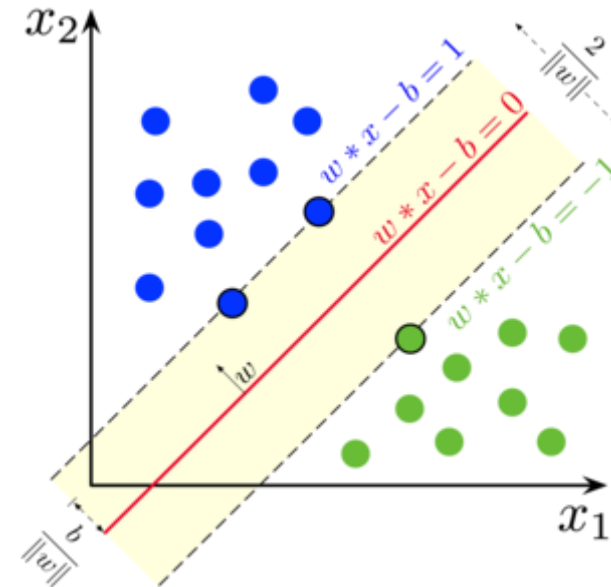
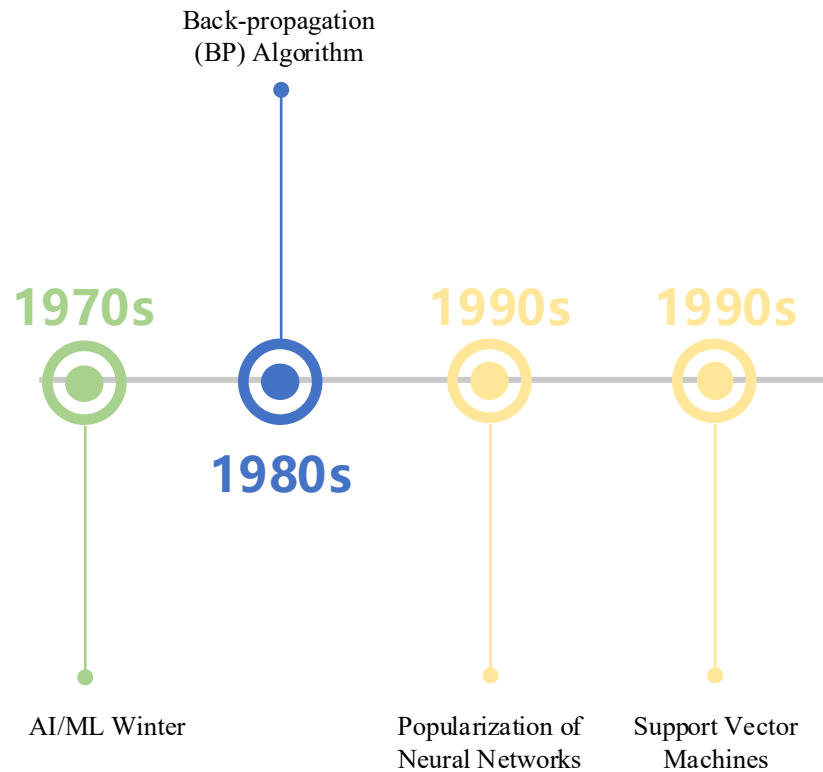


Back-propagation is proposed to allow networks to learn from their errors

Brief History of Machine Learning

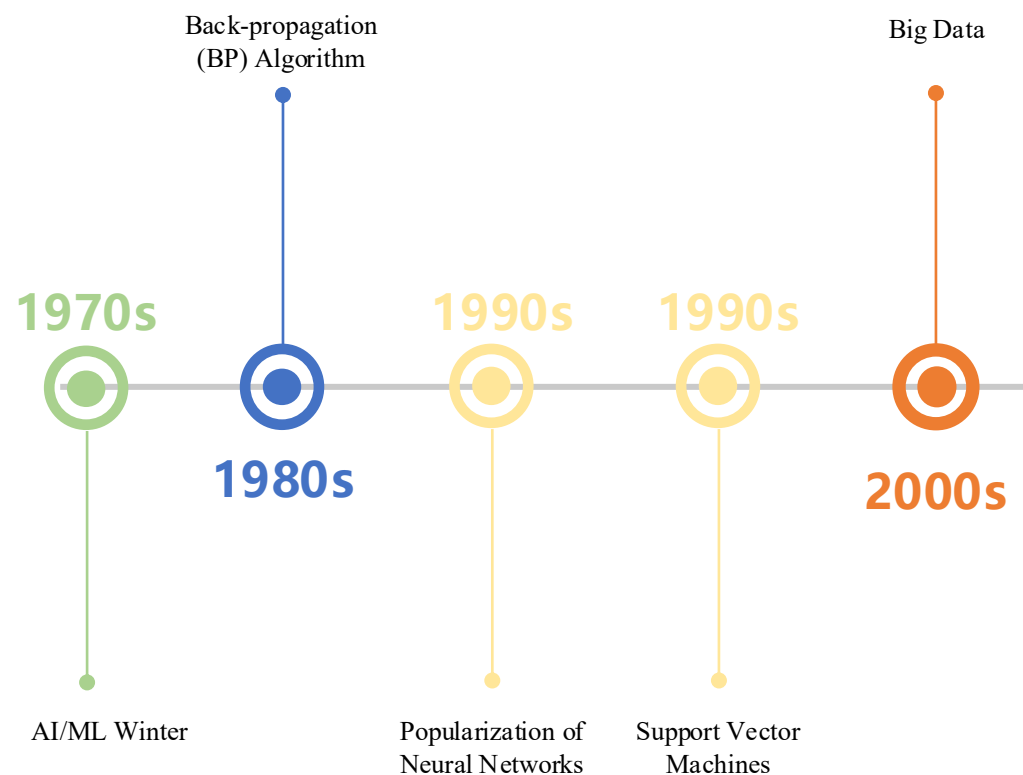


Brief History of Machine Learning



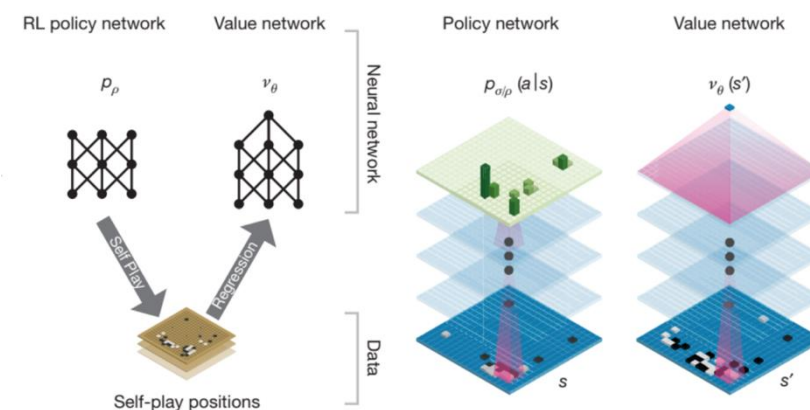
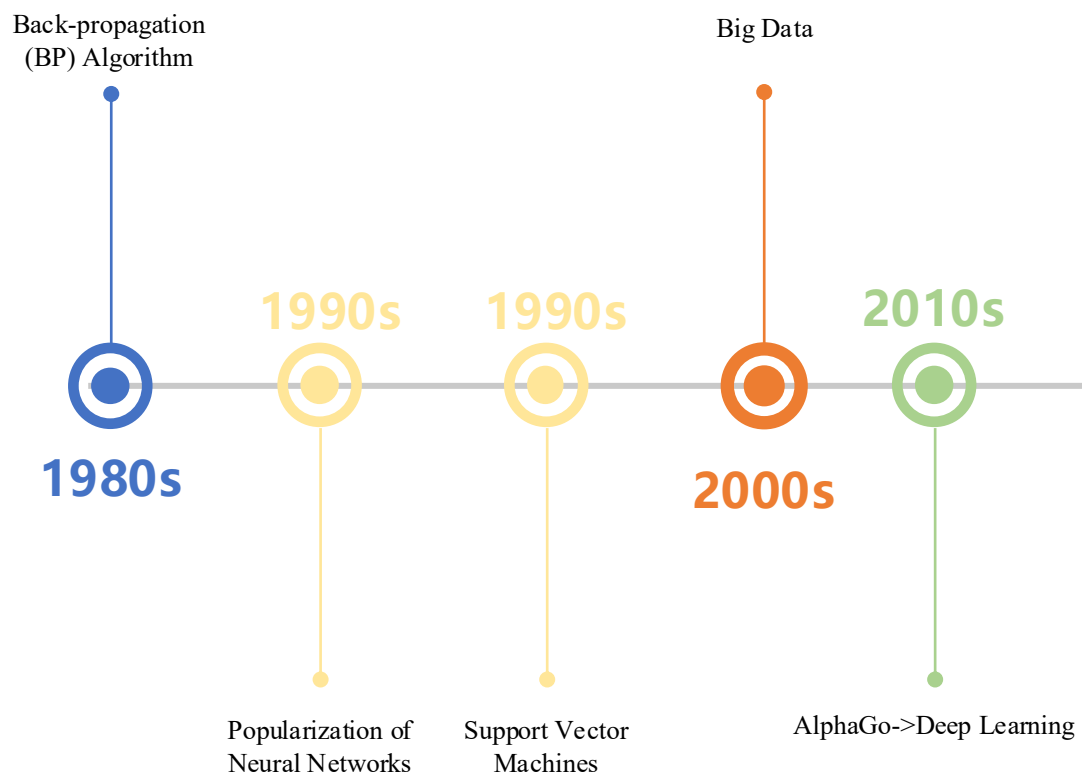
Finding the hyperplane that best separates different classes in the feature space, with the goal of maximizing the margin between the data points

Brief History of Machine Learning



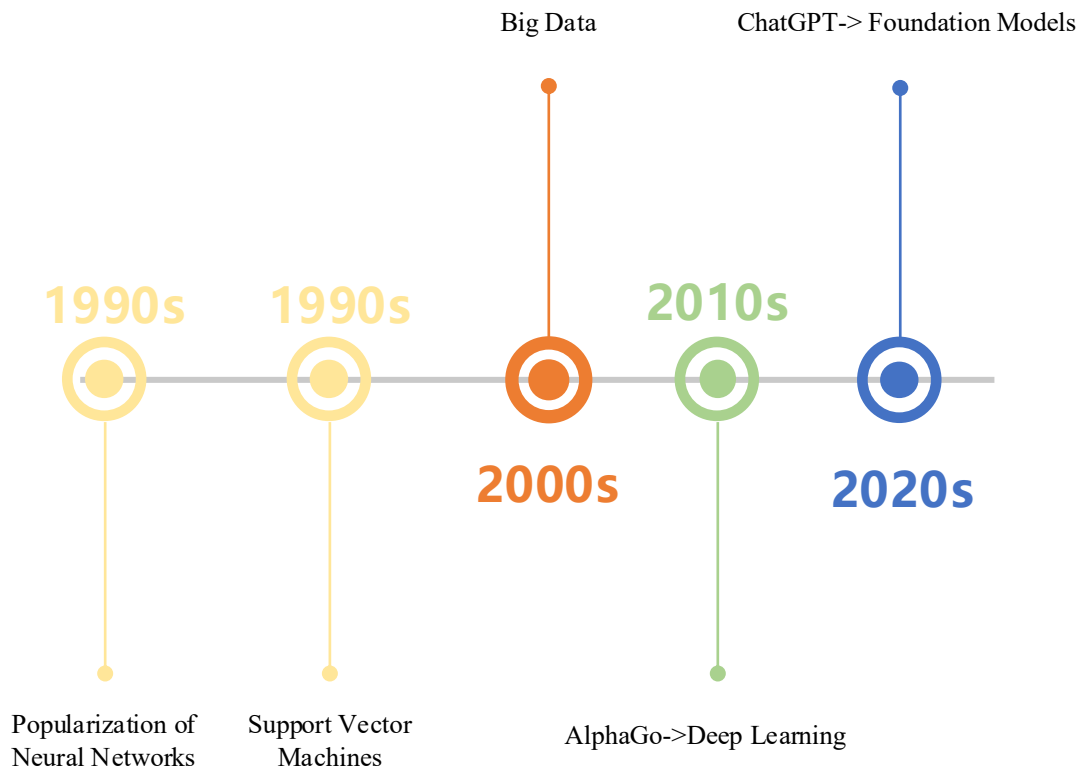
The growth of the internet and provided massive datasets for machine learning algorithms

Brief History of Machine Learning



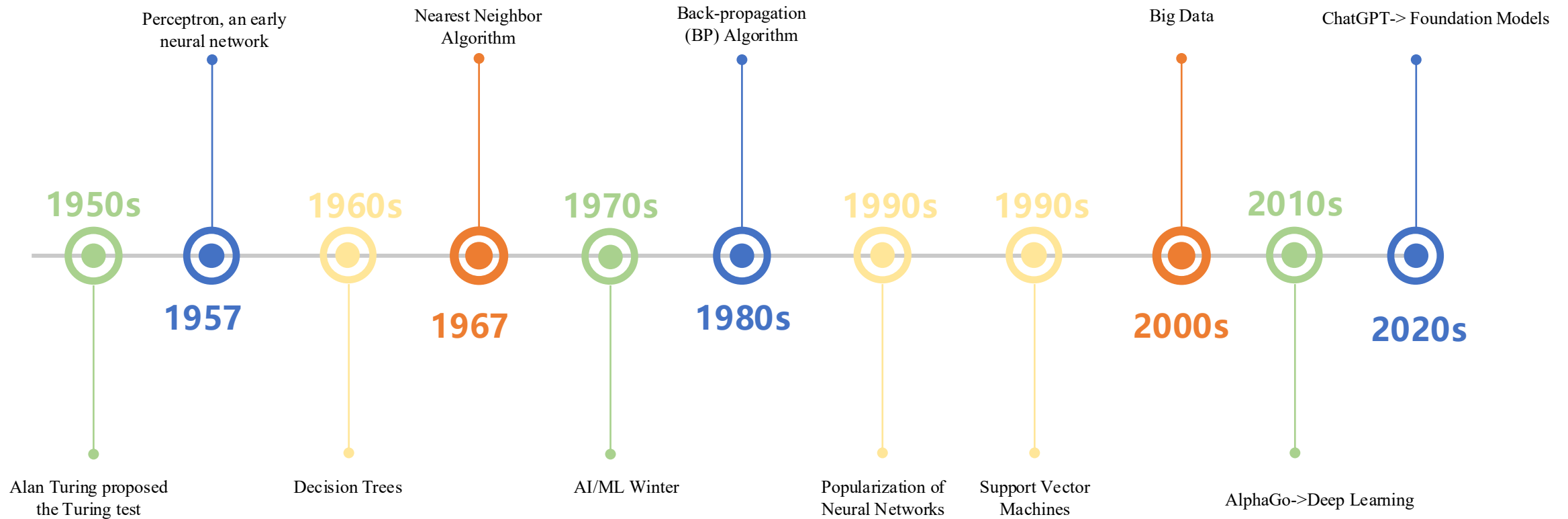
Deep learning, a subset of machine learning involving deep neural networks, led to groundbreaking advancements. AlphaGo's win over a human Go champion was a significant milestone.

Brief History of Machine Learning



Foundation models like GPT emerge as large-scale machine learning models that are pretrained on vast amounts of data and can be fine-tuned for a wide range of tasks,

Brief History of Machine Learning



What is the Future?

A Pathway to AGI

- From Perception to Creation (Generative AI)
- From Virtual to Physical (Embodied AI)
- Emotion, Inspiration (Symbiotics)
- Human-AI Collaboration
- Reasoning, Tool using, Memory (Agent)

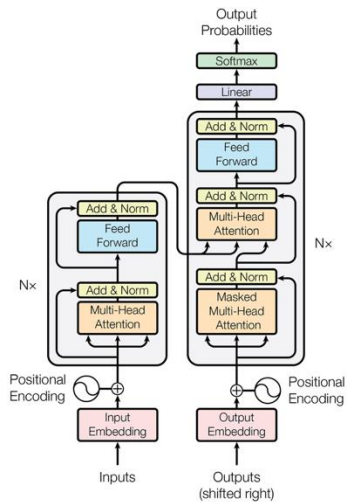
Applications of Machine Learning

- Machine learning has changed almost every industry
 - **Healthcare:** Predictive analytics for patient diagnosis, medical imaging analysis, drug discovery
 - **Finance:** Fraud detection, credit scoring, algorithmic trading, and risk management.
 - **Retail:** Recommendation systems, customer segmentation, inventory management, and sales forecasting
 - **Transportation:** Self-driving cars, route planning, and demand forecasting in public transportation
 - **Education:** Personalized learning, automated grading systems, and learning analytics.
 - **Entertainment:** Content recommendation for streaming services, game AI, and music composition.
 - **Agriculture:** Crop and soil monitoring, predictive analysis for crop yields, and precision farming.

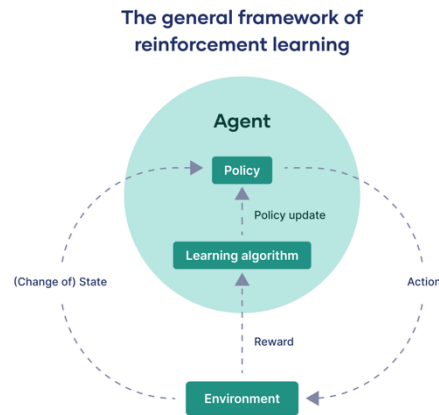
Key Research Areas of Machine Learning

ML Fundamental Research

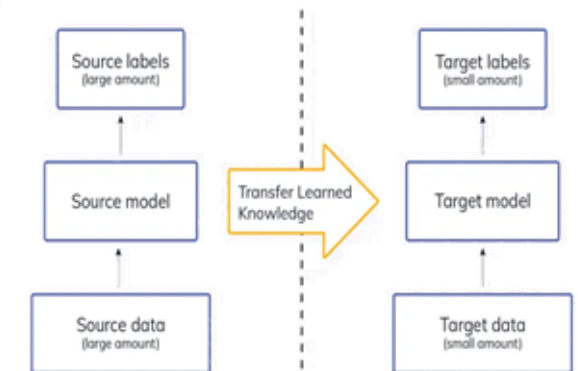
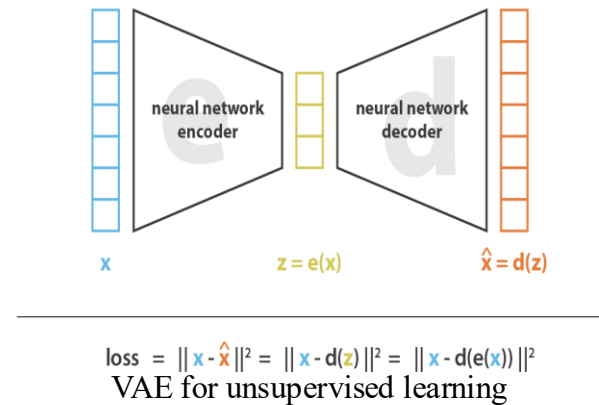
- Neural network design, ML algorithms (reinforcement learning, unsupervised learning, adversarial learning, transfer learning), causal inference and reasoning, planning



Transformer



Reinforcement learning

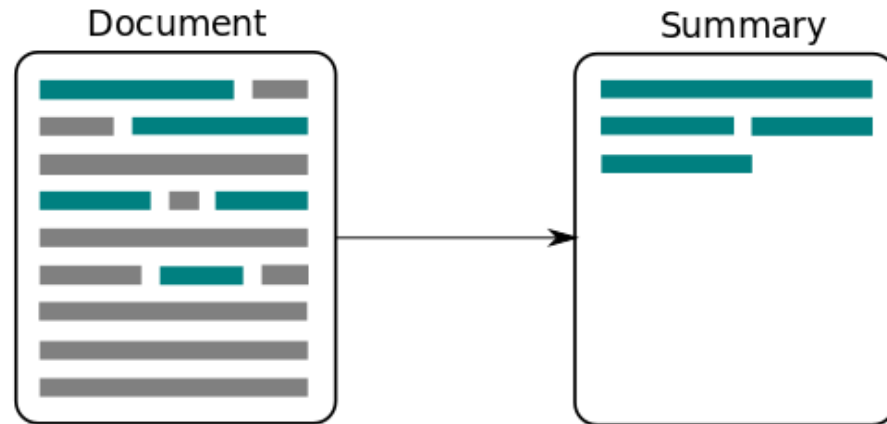


Transfer learning

Key Research Areas of Machine Learning

Natural Language Processing (NLP)

Language understanding, translation, and generation



The screenshot shows the Google Translate web interface. At the top, there's a navigation bar with 'Google Translate' and a 'Sign in' button. Below that, there are tabs for 'Text', 'Images', 'Documents', and 'Websites'. The 'Text' tab is selected. The interface is split into two main sections: 'Detect language' on the left and 'Translation' on the right. The 'Detect language' section has a dropdown menu showing 'English', 'Spanish', and 'French'. The 'Translation' section has a dropdown menu showing 'English', 'Spanish', and 'Arabic'. The text 'What is ChatGPT?' is entered in the 'Detect language' section, and its Spanish translation is shown in the 'Translation' section. Below the translation, there are icons for 'History', 'Saved', and 'Contribute'. At the bottom, there's a section for 'What is ChatGPT?' with a green icon and a detailed explanation of ChatGPT.

Google Translate

Text Images Documents Websites

Detect language English Spanish French

Translation English Spanish Arabic

History Saved Contribute

A What is ChatGPT?

ChatGPT is an AI-powered chatbot developed by OpenAI. It is based on the GPT (Generative Pre-trained Transformer) architecture, specifically GPT-3.5. ChatGPT is designed to engage in text-based conversations with users, providing responses that are coherent, contextually relevant, and generated in a human-like manner.

The underlying technology behind ChatGPT is deep learning, specifically employing a transformer model. This architecture allows the model to process and understand the structure, context, and meaning of text input, enabling it to generate appropriate and informative responses.

Computer Vision

A street scene with various vehicles and a person on a bicycle, annotated with bounding boxes and labels. The labels include: truck, car, car, car, car, car, car, car, car, person, bicycle, bicycle, and traffic light. The bounding boxes are colored: yellow for cars, red for bicycles, pink for the person, and blue for the traffic light.

[illegible]

25

Key Research Areas of Machine Learning

Speech and Audio Processing

Enable the machines to process, understand and generate spoken language and other audio signals



Text-to-Speech



Speech Recognition



Speaker Verification



AI Music

Key Research Areas of Machine Learning

Recommendation Systems

Predict the preferences or ratings that a user would give to a product or item

Customers who viewed this item also viewed

Page 1 of 6



Spigen Liquid Air Armor Designed for OnePlus 8 Case (2020) [NOT Compatible with Verizon UW Version] - Matte Black
★★★★☆ 673
INR926.48



Spigen Ultra Hybrid Designed for OnePlus 8 Case (2020) [NOT Compatible with Verizon UW Version] - Crystal Clear
★★★★☆ 1,406
INR926.48



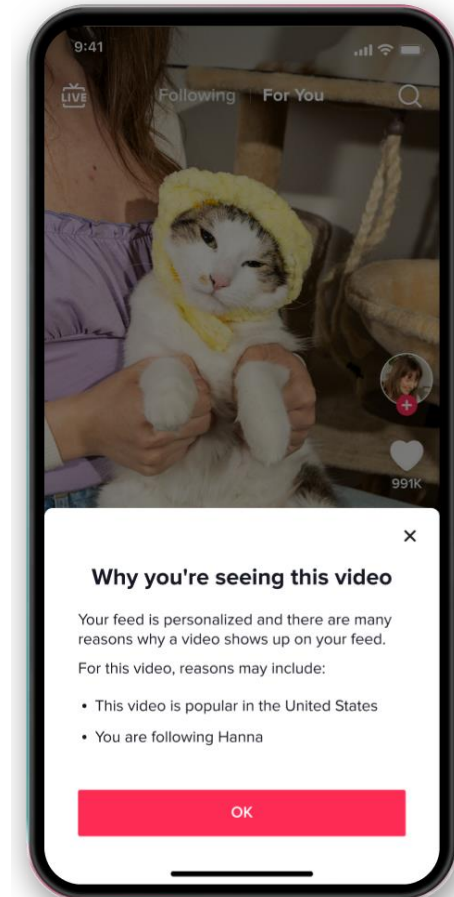
SAMSUNG Galaxy A71 5G Factory Unlocked Android Cell Phone 128GB US Version Smartphone 6.7-inch AMOLED Screen Long...
★★★★☆ 2,021
INR45,048.27



OtterBox Symmetry Clear Series Case for OnePlus 8 5G - Clear, Model Number: 77-64863
★★★★☆ 177
INR1,049.34



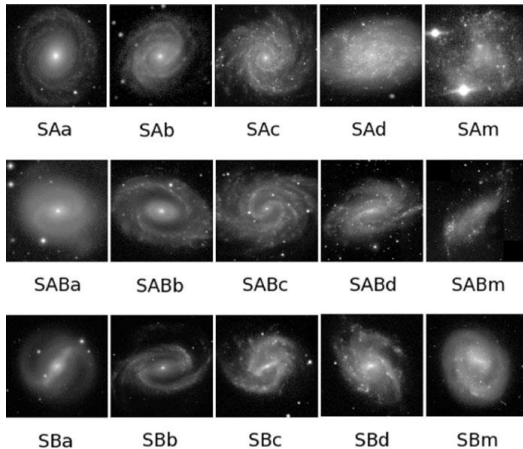
Spigen Rugged Armor Designed for OnePlus 8 5G UW (Verizon Version) Case (2020) - Matte Black
★★★★☆ 358
INR926.48



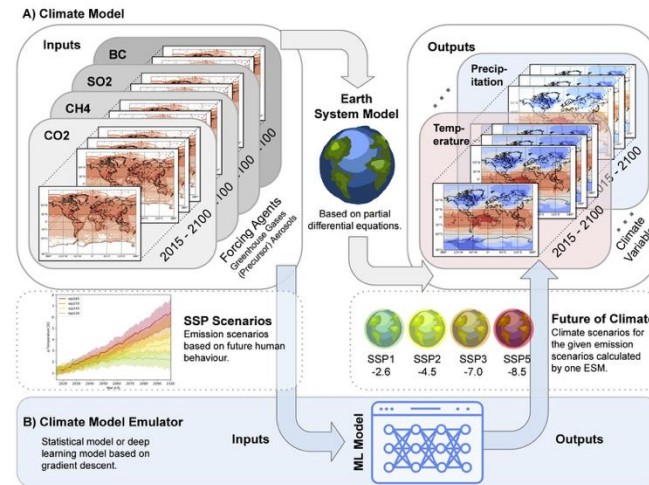
Key Research Areas of Machine Learning

AI4Science

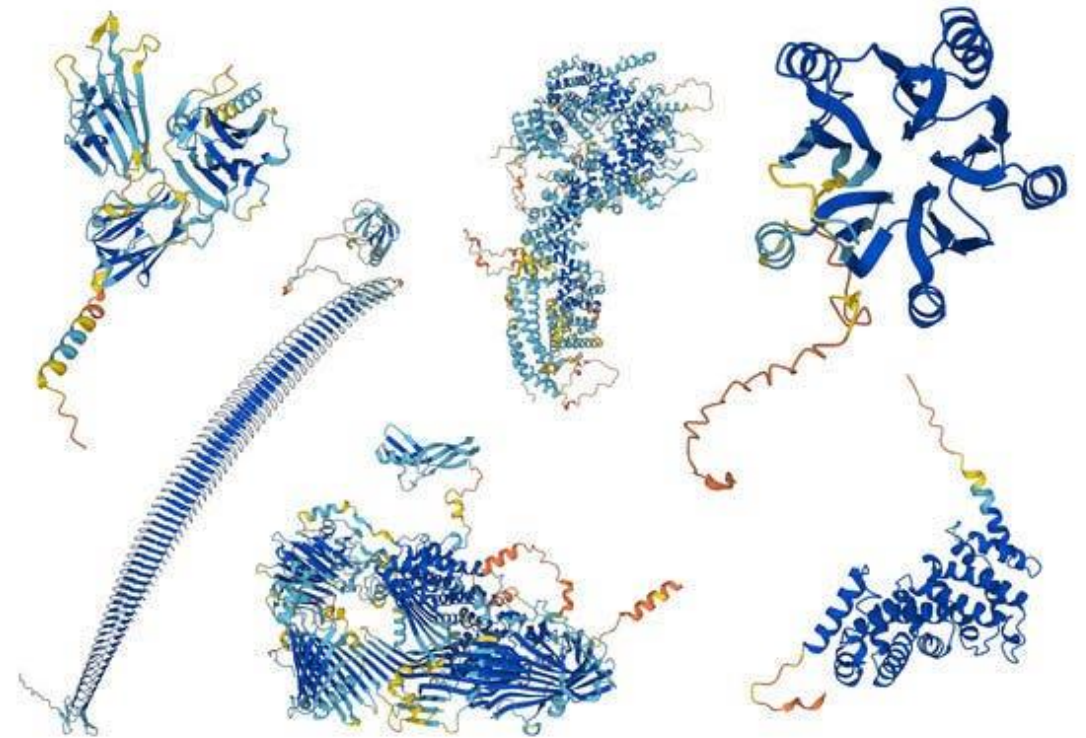
Analyze vast amounts of data, recognize patterns, make predictions for fundamental science research.



Galaxy Classification

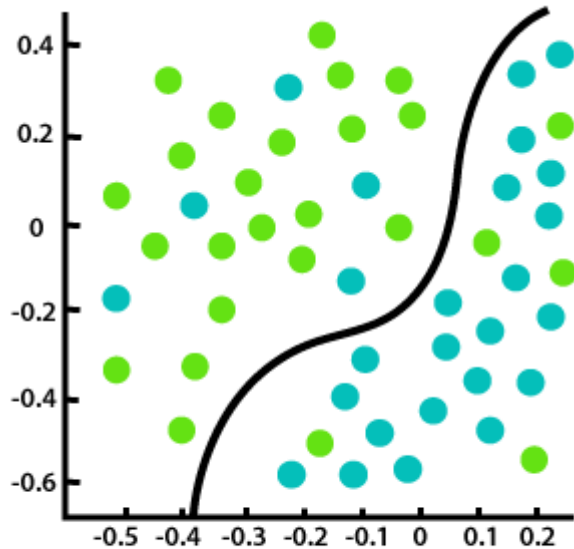


Climate Change

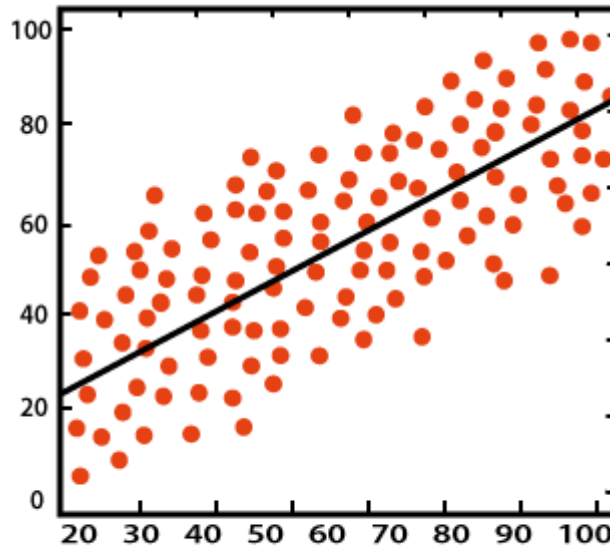


AlphaFold for protein structure prediction

Basic Concepts of Machine Learning



Classification



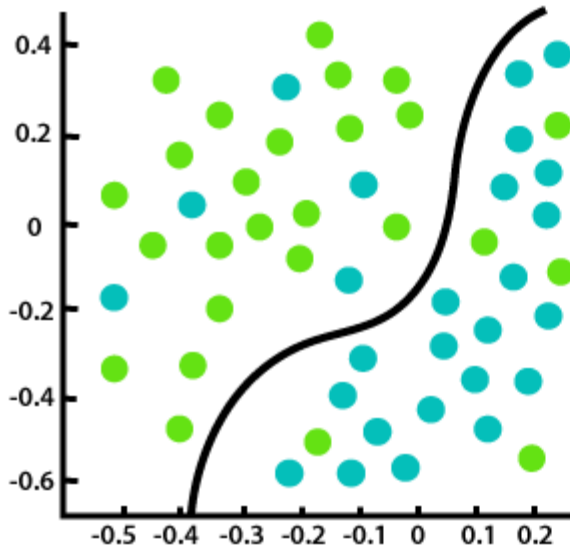
Regression

ML Tasks

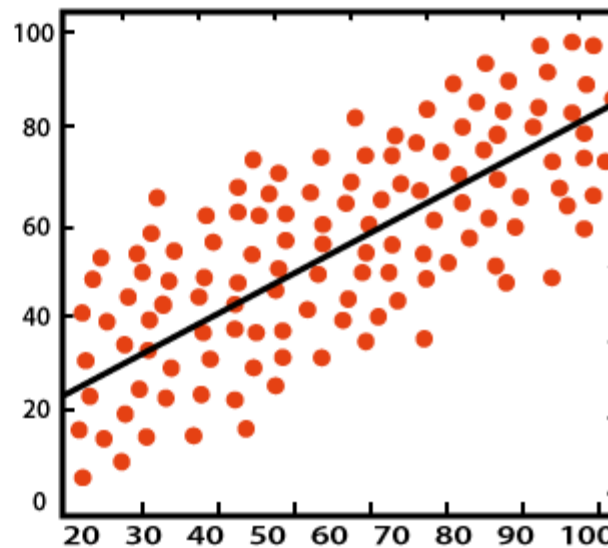
- Regression algorithms are used to predict a **continuous numerical** value. For example, regression can be used to predict temperatures, prices, or any quantity that can vary and doesn't have a finite set of values.
- Classification algorithms are used to **predict discrete outputs, typically in the form of classes or categories**. For example, classification can be used to determine if an email is spam or not spam, or to recognize hand-written digits.

Basic Concepts of Machine Learning

ML Tasks



Classification

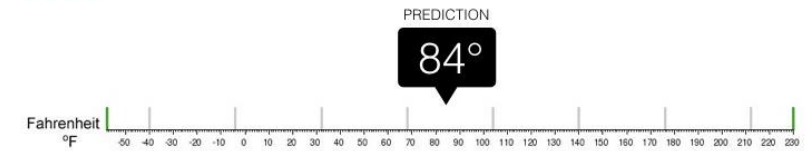


Regression



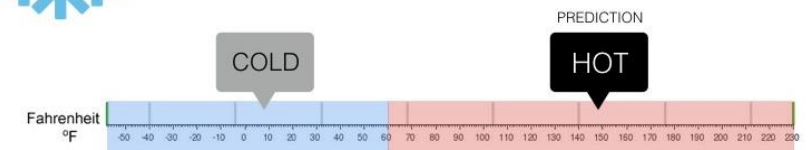
Regression

What is the temperature going to be tomorrow?



Classification

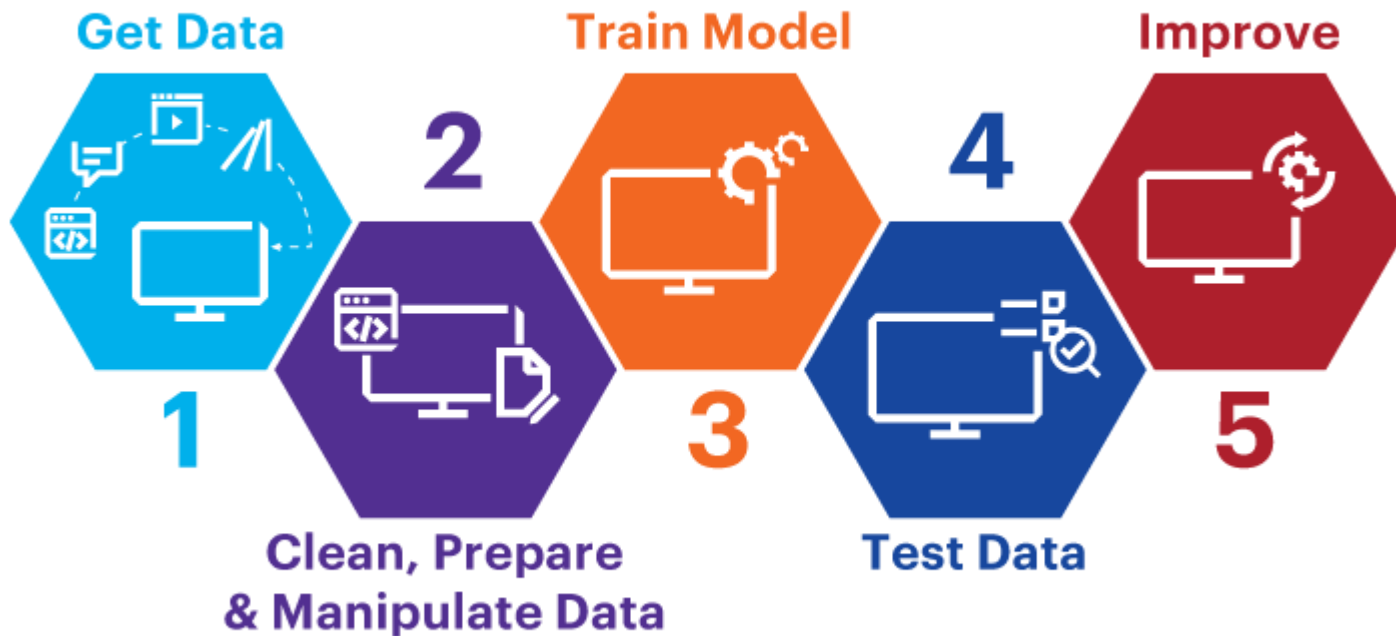
Will it be Cold or Hot tomorrow?



Is face recognition regression or classification?

Basic Concepts of Machine Learning

The typical workflow of ML system



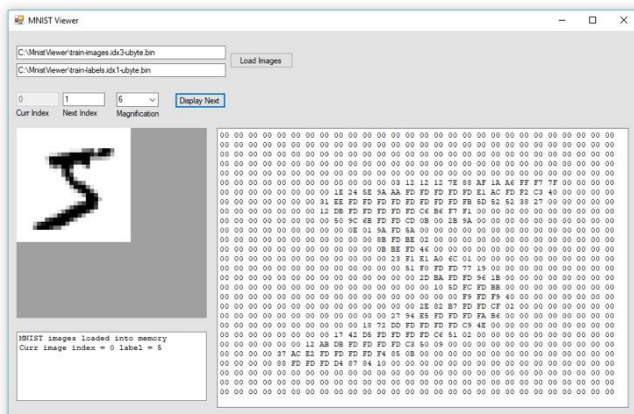
Key elements to design

- Data (the “textbooks”)
- Model (the “students”, e.g., SVM, Neural Networks)
- Evaluation (the “grading system”, e.g., mean squared error)

Basic Concepts of Machine Learning

What is the typical format of data used for ML?

- A **data sample** is (X_i, Y_i) where X_i keeps the raw information of the sample, and Y_i indicates the class, or the regression target of the sample.
- X_i is typically a vector describing different sample attributes.
- A collection of data sample becomes a **dataset**.

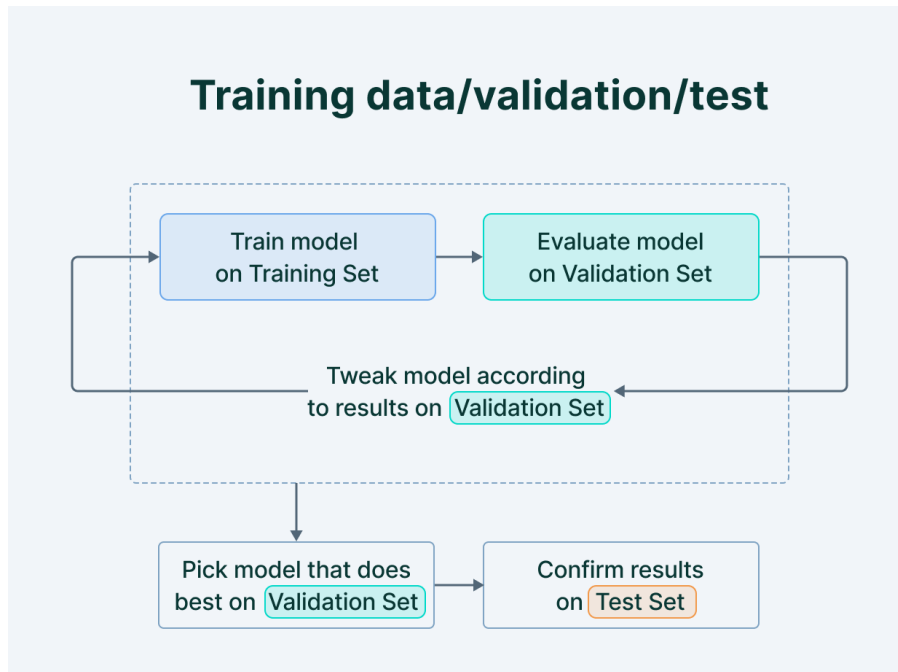


Class: 5



Basic Concepts of Machine Learning

What is the typical format of data used for ML?



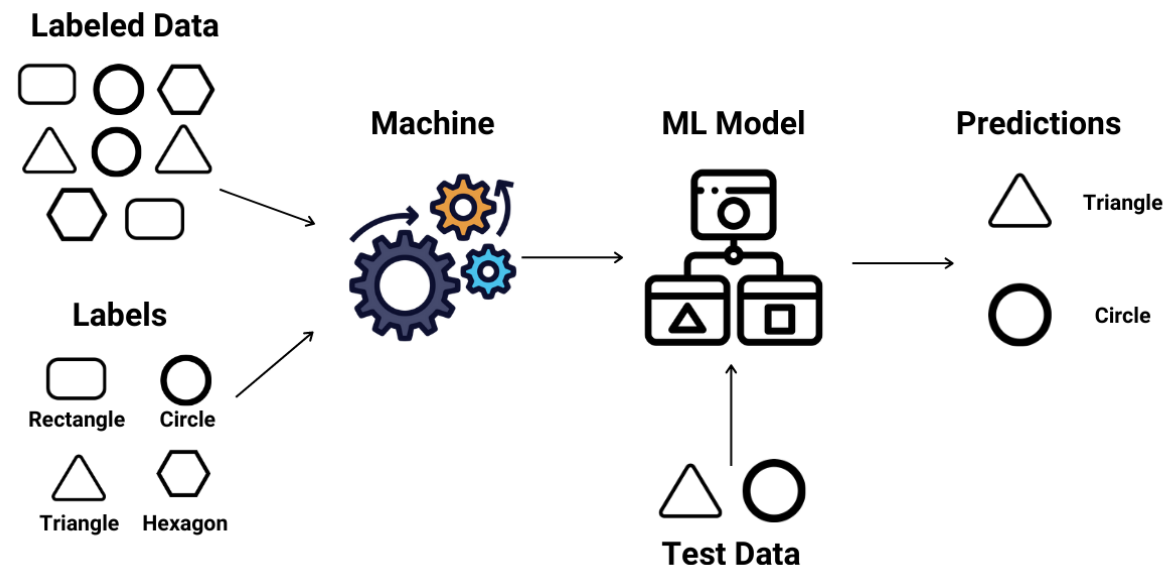
Splitting of dataset

- **Training set:** used to train the ML model
- **Validation set:** provide an evaluation of a model fit on the training dataset while tuning the model's design.
- **Test set:** assess the performance of the machine learning model after the training process is complete
- Typical splits: 80/10/10, or 90/5/5.

Basic Concepts of Machine Learning

Types of ML Algorithms

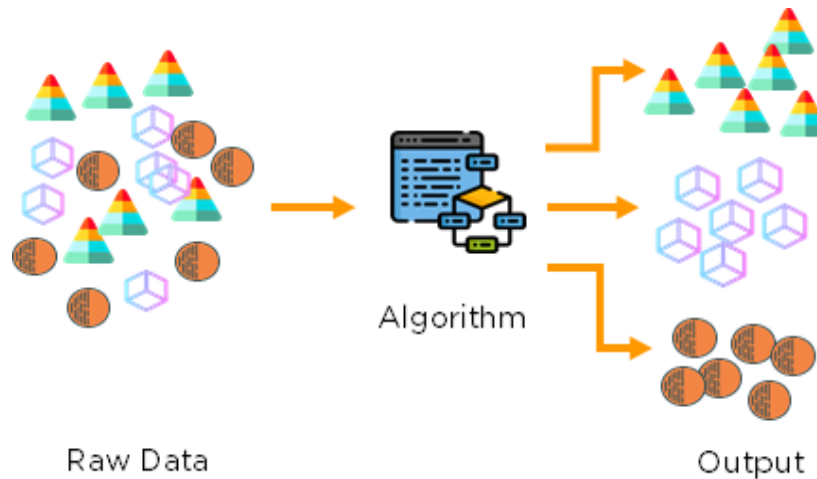
- Depending on the availability of data labels, ML algorithms can be broadly categorized into
 - Supervised Learning:** Algorithms that learn from **labeled** training data, predicting labels for new, unseen data. The most common type of ML algorithms.



Basic Concepts of Machine Learning

Types of ML Algorithms

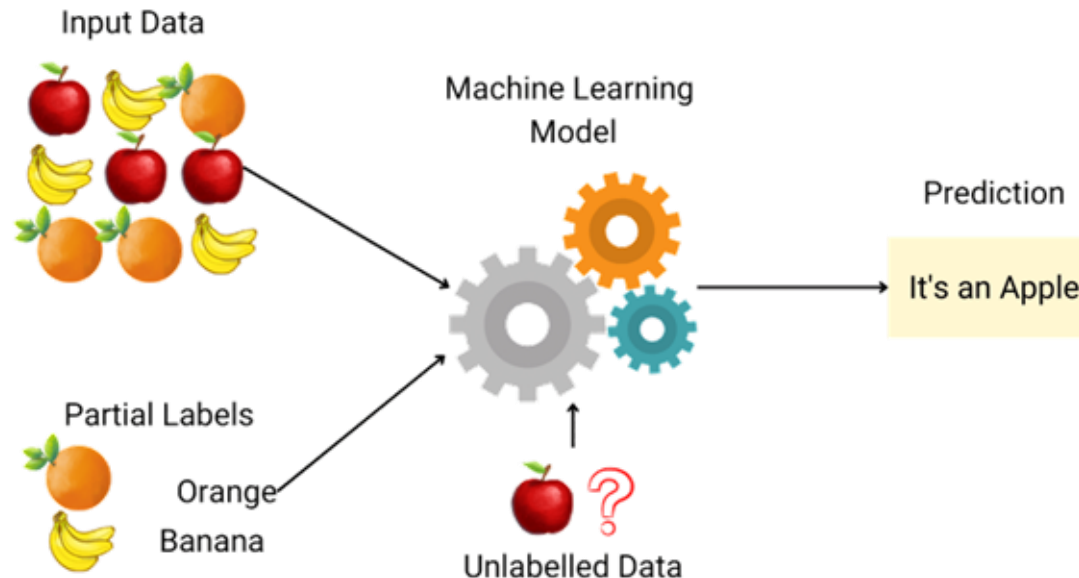
- Depending on the availability of data labels, ML algorithms can be broadly categorized into
 - **Unsupervised Learning:** Algorithms that identify patterns in data **without any labels**. The data difference can be automatically found.



Basic Concepts of Machine Learning

Types of ML Algorithms

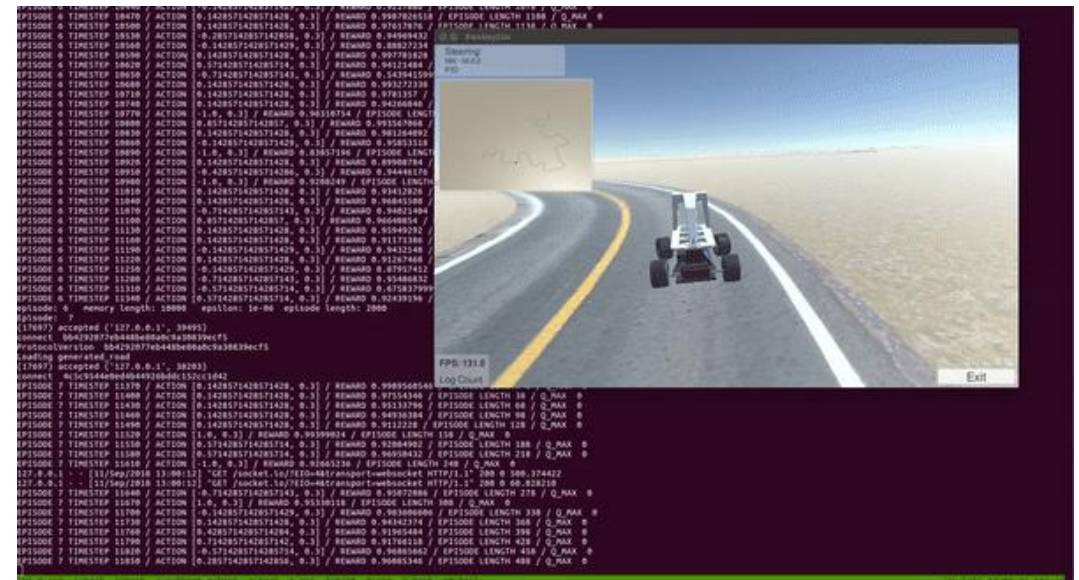
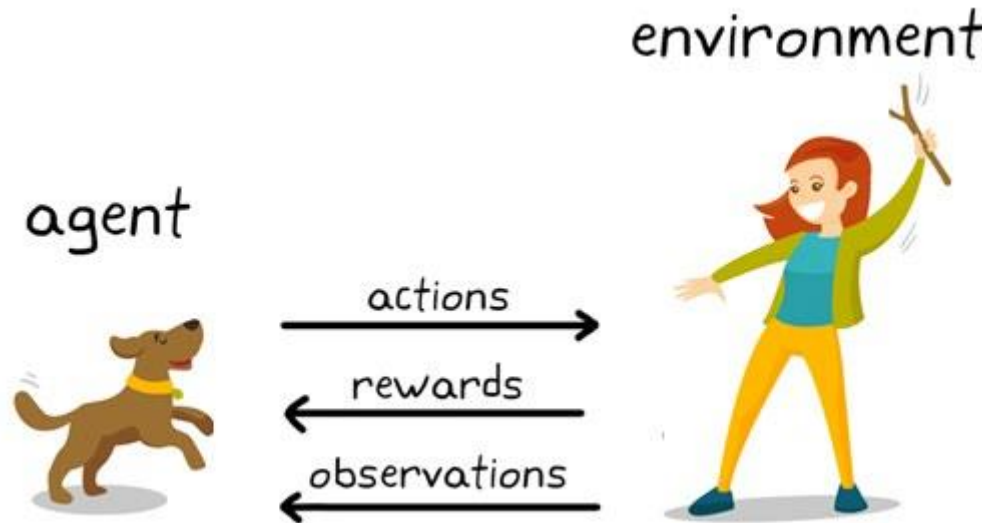
- Depending on the availability of data labels, ML algorithms can be broadly categorized into
 - **Semi-supervised Learning:** Uses a **combination** of a small amount of labeled data and a large amount of unlabeled data in training.



Basic Concepts of Machine Learning

Types of ML Algorithms

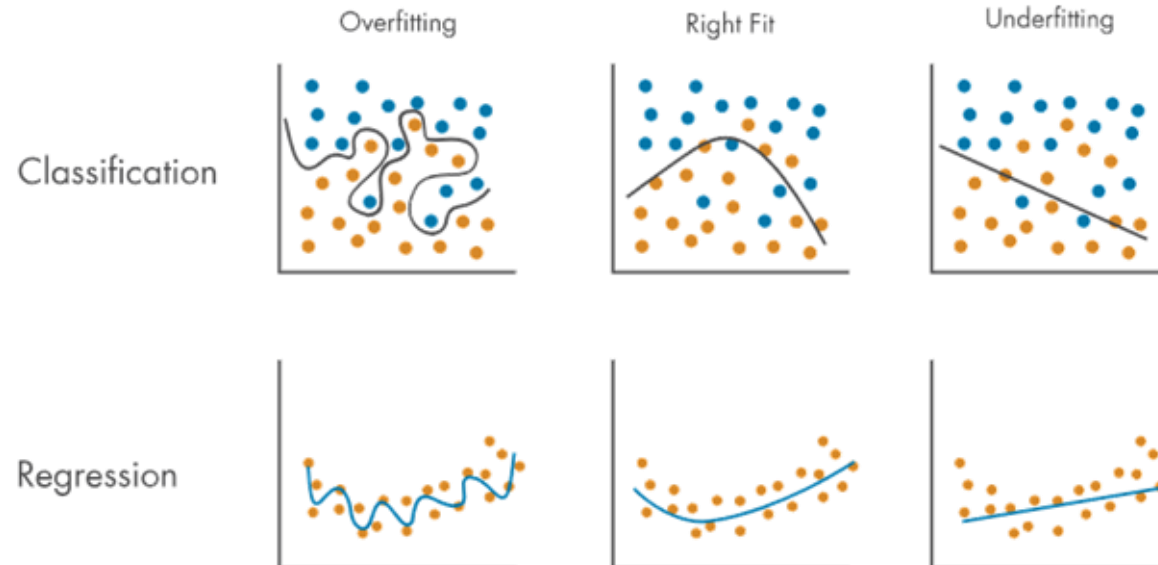
- Depending on the type of interaction with the environment
 - Reinforcement Learning:** Algorithms learn to make specific decisions by optimizing the cumulative reward they receive **for their actions**



Basic Concepts of Machine Learning

Overfitting and Underfitting

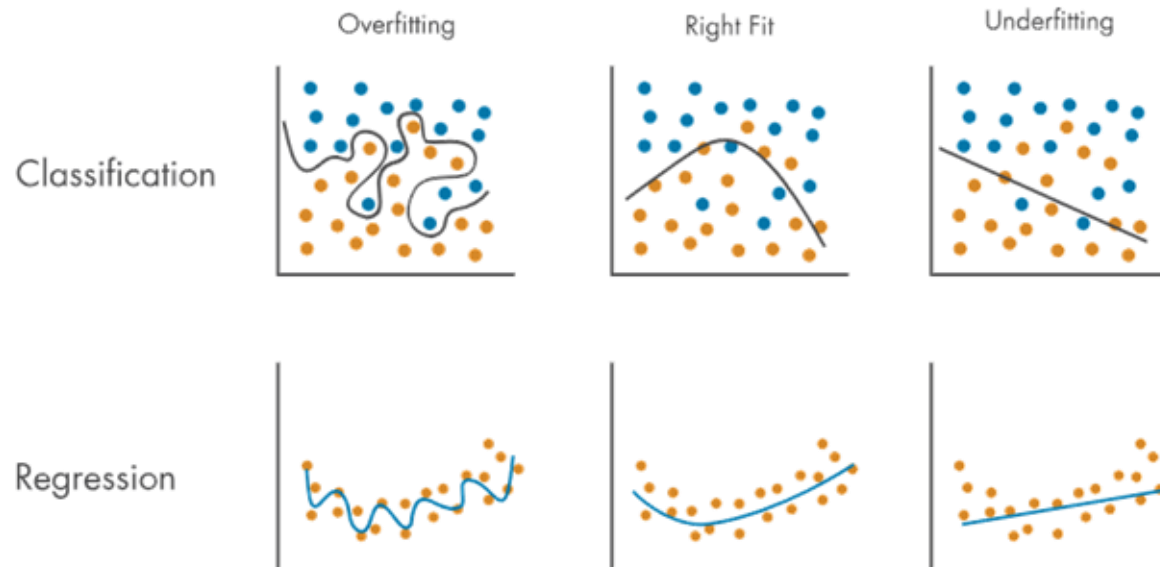
- Overfitting: the model performs very well on the training data but poorly on new
 - The model is too complex
 - The training set is too small



Basic Concepts of Machine Learning

Overfitting and Underfitting

- Underfitting: fails to learn the relationship between the input and target, leading to poor performance both on the training data and on new, unseen data.
 - The model is too simple as compared with the data size.

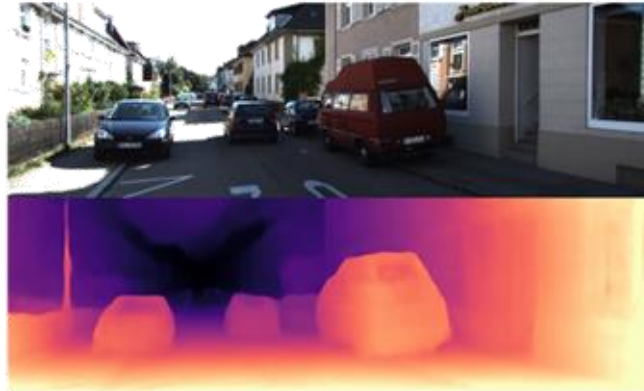


What is Regression?

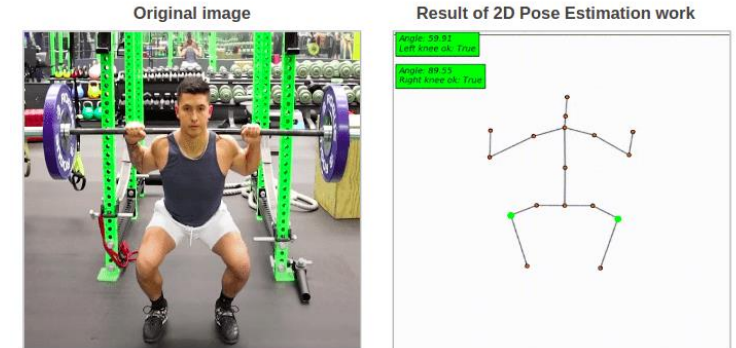
The input and output can be multi-dimensional.

	X	Y
	YearsExperience	Salary
0	1.1	39343.0
1	1.3	46205.0
2	1.5	37731.0
3	2.0	43525.0
4	2.2	39891.0

Salary Dataset



Depth Estimation: Multi-dim in, Single-dim out



Pose Estimation: Multi-dim in, Multi-dim out

Is there any example with a single-dimensional input and multi-dimensional output?

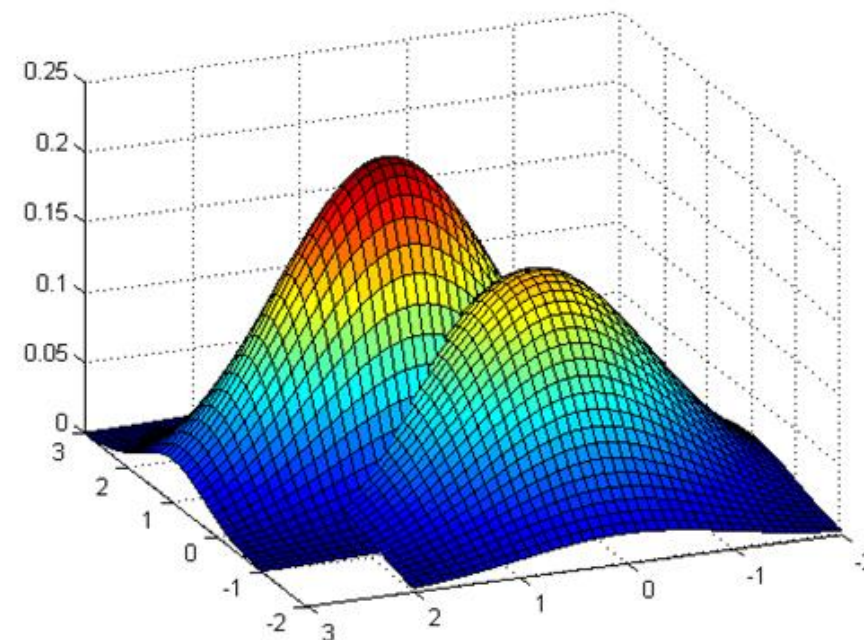
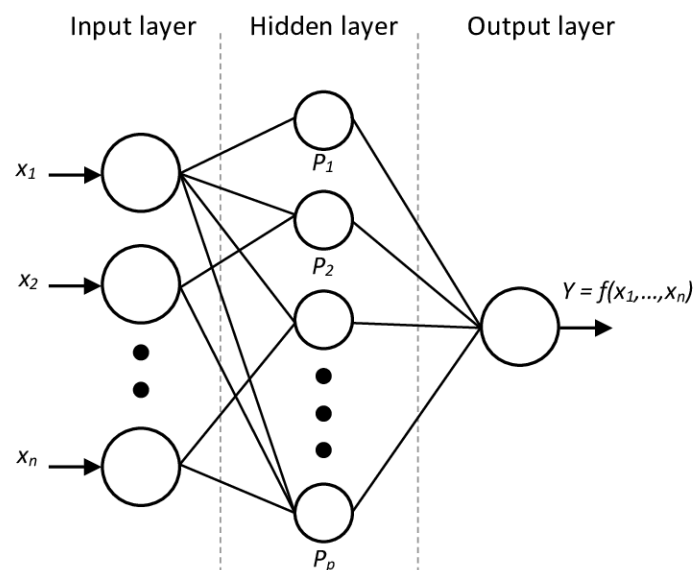
Types of Regression

Tradition Types of Regression Models

- Linear Regression
 - Simple Linear Regression
 - Multiple Linear Regression
- Polynomial Regression
- Regularization Methods
 - Ridge Regression (L2 Regularization)
 - Lasso Regression (L1 Regularization)

Types of Regression

Deep Learning Regression Model



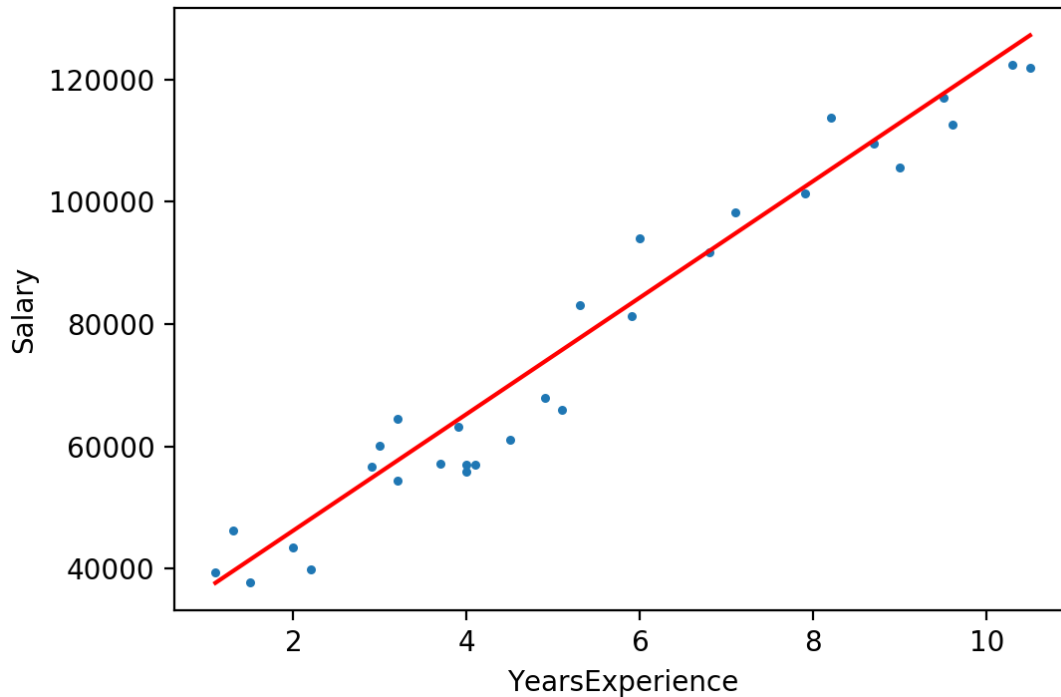
We mainly focus on traditional types of regression in this lesson, and will learn DNN later.

Types of Regression

Tradition Types of Regression Models

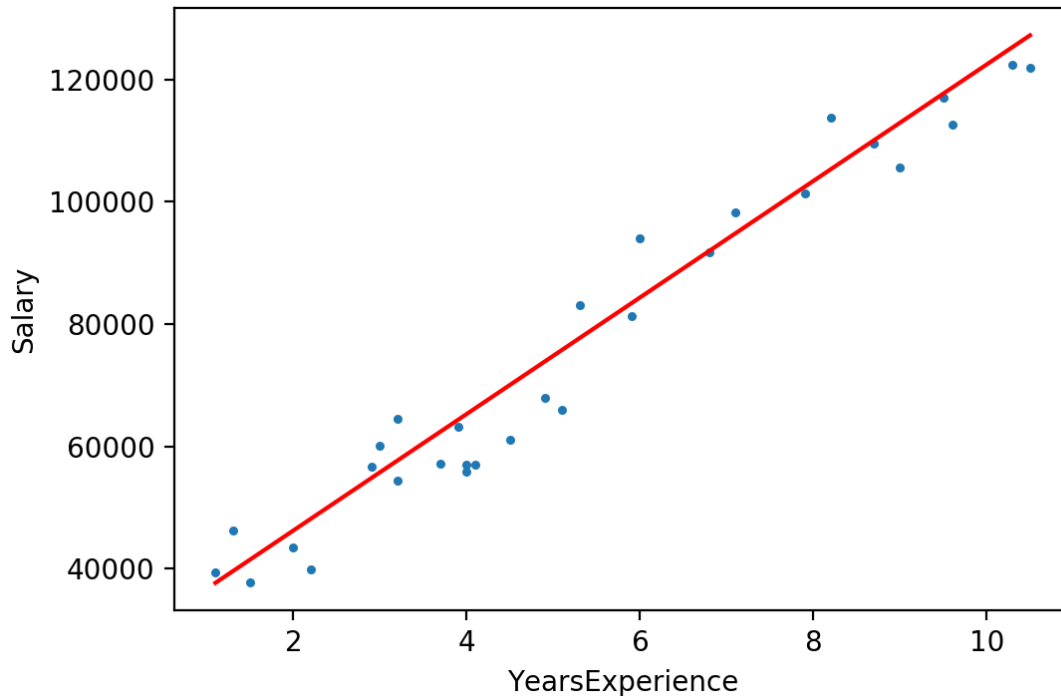
- Linear Regression
 - **Simple Linear Regression**
 - Multiple Linear Regression
- Polynomial Regression
- Regularization Methods
 - Ridge Regression (L2 Regularization)
 - Lasso Regression (L1 Regularization)

Simple Linear Regression



- Given the dataset of years of working experience and the salary, how to predict the salary for an arbitrary number of years (e.g., 3.5 Yrs)?
- Simple linear regression attempts to find the best-fitting straight line through these points.

Simple Linear Regression



- This line is often called the “line of best fit” or “regression line”. The equation for this line is

$$Y = \alpha X + \beta + \epsilon$$

- Y is the observed variable (salary)
- X is the input variable (number of working years)
- α is the slope of the line
- β is the y-intercept of the line
- ϵ is the error term (difference between prediction and observation)
- Simple linear regression tries to estimate α and β from data.

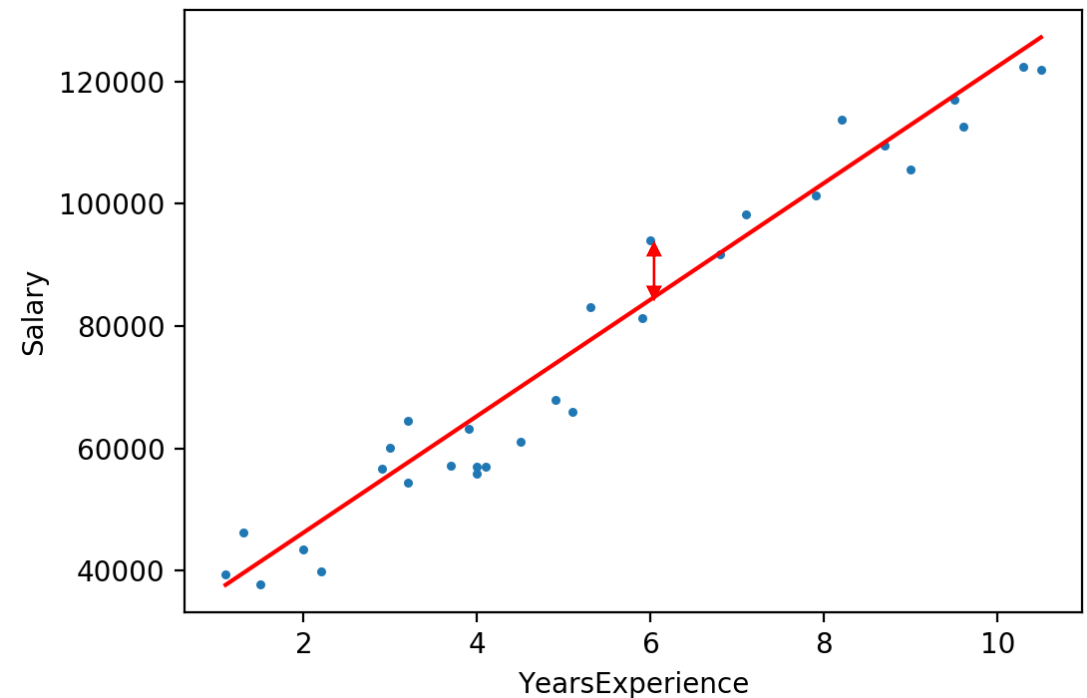
Simple Linear Regression

Ordinary Least Squares

We aim to minimize the sum of the squared differences between the observed values and the values predicted by the linear model.

Sum of squared residuals (SSR)

$$SSR = \sum_{i=1}^n (y_i - (\beta + \alpha x_i))^2$$



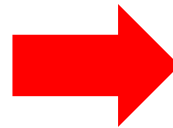
Simple Linear Regression

- Taking the partial derivatives of SSR with respect to α and β and setting them to zero, we have

$$SSR = \sum_{i=1}^n (y_i - (\beta + \alpha x_i))^2$$

$$\frac{\partial SSR}{\partial \alpha} = -2 \sum_{i=1}^n (y_i - \beta - \alpha x_i) x_i = 0$$

$$\frac{\partial SSR}{\partial \beta} = -2 \sum_{i=1}^n (y_i - \beta - \alpha x_i) = 0$$



$$\alpha = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$\beta = \bar{y} - \left(\frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^n (x_i - \bar{x})^2} \right) \bar{x}$$

Simple Linear Regression

Years of Experience (x)	Salary in K HKD (y)
1.2	40
2.4	45
2.8	50
3.1	52
4.5	58
4.9	60
6.3	68
7.2	72
8.6	80
9.1	85

$$\bar{x} = \frac{1.2 + 2.4 + 2.8 + 3.1 + 4.5 + 4.9 + 6.3 + 7.2 + 8.6 + 9.1}{10} = \frac{50.1}{10} = 5.01$$

$$\bar{y} = \frac{40 + 45 + 50 + 52 + 58 + 60 + 68 + 72 + 80 + 85}{10} = \frac{610}{10} = 61$$

$$\alpha = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2} = \frac{364.4}{66.21} \approx 5.5$$

$$\beta = \bar{y} - \alpha\bar{x} = 33.426$$

The appropriate salary for a person with 5-year working experience is $5.5 * 5 + 33.426 = 61$ K HKD.

What is the salary for a freshman?

Types of Regression

Tradition Types of Regression Models

- Linear Regression
 - Simple Linear Regression
 - **Multiple Linear Regression**
- Polynomial Regression
- Regularization Methods
 - Ridge Regression (L2 Regularization)
 - Lasso Regression (L1 Regularization)

Multiple Linear Regression

Consider an example where we want to predict the price of houses based on various features

- Total square footage of the house
- Number of bedrooms
- Number of bathrooms
- Age of the house (in years)
- Distance to the nearest city center

Multiple Linear Regression

- Multiple linear regression is an extension of simple linear regression that allows for the prediction of a dependent variable based on two or more independent variables.

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

- Y is the dependent variable that we are trying to predict
- $X_1, X_2, \dots, X_p$ are the independent variables (also known as features)
- β_0 is the y-intercept of the regression plane
- $\beta_1, \beta_2, \dots, \beta_p$ are the partial regression coefficients
- ε is the error term, which accounts for the variability in Y that cannot be explained by the linear model

Multiple Linear Regression

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

- For n samples, in the matrix form, we have

$$\mathbf{Y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

- $\mathbf{Y}$ is an $n \times 1$ vector of the dependent variable, n is the number of samples.
- $\mathbf{X}$ is an $n \times (p + 1)$ matrix of the independent variables with a column of ones for the intercept.
- $\boldsymbol{\beta}$ is a $(p + 1) \times 1$ vector of the regression coefficients.
- $\boldsymbol{\varepsilon}$ is an $n \times 1$ matrix (vector) of the model errors

$$\begin{array}{ccc} \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_n \end{bmatrix} & \begin{bmatrix} 1 & X_{11} & X_{12} & \dots & X_{1p} \\ 1 & X_{21} & X_{22} & \dots & X_{2p} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & X_{n1} & X_{n2} & \dots & X_{np} \end{bmatrix} & \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} \\ \mathbf{Y} & \mathbf{X} & \boldsymbol{\beta} \end{array}$$

Multiple Linear Regression

Ordinary Least Squares

We aim to find the best-fitting model by minimizing the sum of squared residuals (SSR).

Sum of squared residuals (SSR)

$$\|Y - X\beta\|_2^2 = (Y - X\beta)^\top (Y - X\beta)$$

Minimizing the SSR leads to

$$\beta = (X^\top X)^{-1} X^\top Y$$

Ref: derivatives of matrix: *The Matrix Cookbook* <http://www2.imm.dtu.dk/pubdb/doc/imm3274.pdf>

Multiple Linear Regression

Consider a small dataset for the real estate prices

House	Size (1000 sq ft)	Age (Years)	Selling Price (1000 USD)
1	1.5	5	250
2	2.0	3	300
3	2.5	10	320
4	3.0	15	360
5	3.5	20	420

$$\mathbf{X} = \begin{bmatrix} 1 & 1.5 & 5 \\ 1 & 2.0 & 3 \\ 1 & 2.5 & 10 \\ 1 & 3.0 & 15 \\ 1 & 3.5 & 20 \end{bmatrix}$$

$$\mathbf{Y} = \begin{bmatrix} 250 \\ 300 \\ 320 \\ 360 \\ 420 \end{bmatrix}$$

$$\text{Then } \boldsymbol{\beta} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{Y} = \begin{bmatrix} 120 \\ 88.07 \\ -0.96 \end{bmatrix}$$

If there is a house with 2.8K sq ft and 3 years, the ideal selling price would be $[1, 2.8, 3] * \boldsymbol{\beta} = 364$ K USD.

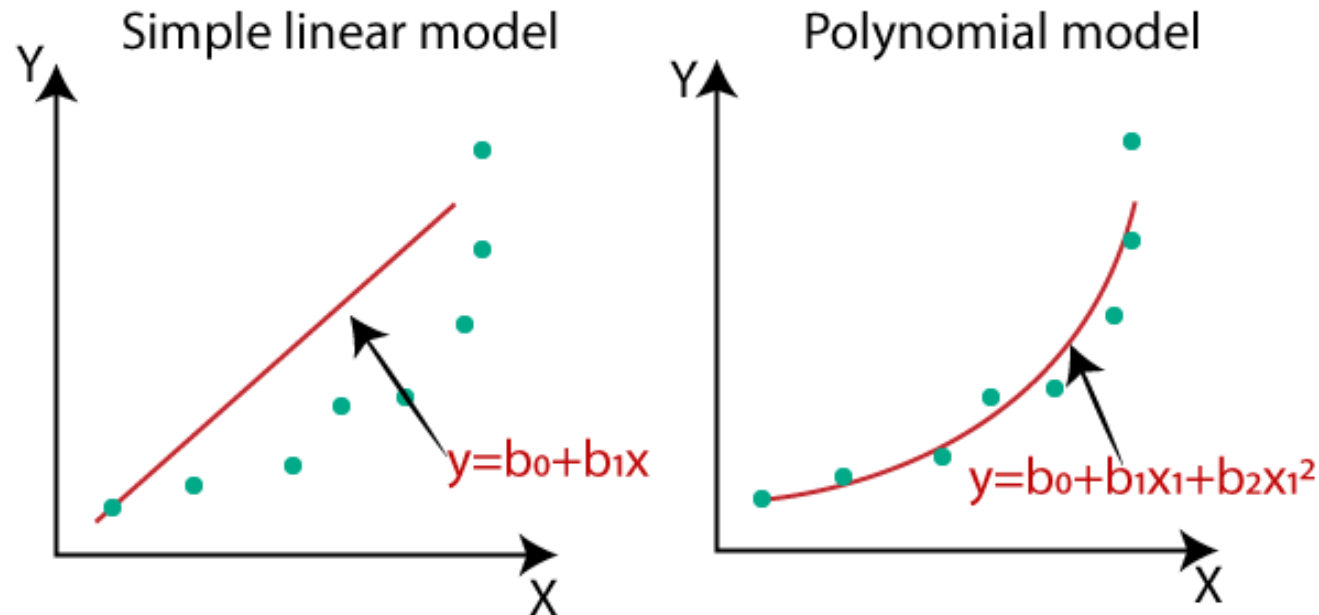
Types of Regression

Tradition Types of Regression Models

- Linear Regression
 - Simple Linear Regression
 - Multiple Linear Regression
- **Polynomial Regression**
- Regularization Methods
 - Ridge Regression (L2 Regularization)
 - Lasso Regression (L1 Regularization)

Polynomial Regression

Linear regression may not be accurate, many relations are non-linear



Polynomial Regression

Polynomial regression models the relationship between the independent variable, x , and the dependent variable (observation), y , as an n -th degree polynomial.

$$y = \beta_0 + \beta_1x + \beta_2x^2 + \beta_3x^3 + \dots + \beta_nx^n + \epsilon$$

where β_i for $i = 0, \dots, n$ are polynomial coefficients, ϵ is the error term between the observation and the prediction.

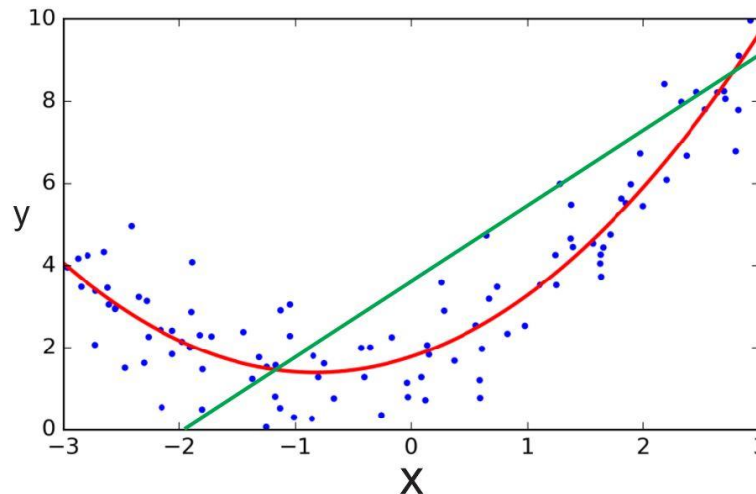
Polynomial Regression

Linear regression: $Y = \alpha X + \beta + \epsilon$

- Models the relationship as a straight line, limited to linear associations
- Simple

Polynomial regression $y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n + \epsilon$

- Extends linear regression by adding polynomial terms, the best-fit line can bend
- Can model non-linear associations



Polynomial Regression

How to fit the polynomial regression?

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots + \beta_n x^n + \epsilon$$

For m samples, in the matrix form we have

$$\mathbf{Y} = \mathbf{X}\beta + \epsilon$$

$$\begin{array}{ccc} \begin{bmatrix} Y_1 \\ Y_2 \\ \vdots \\ Y_m \end{bmatrix} & \begin{bmatrix} 1 & x_1 & x_1^2 & x_1^3 & \dots & x_1^n \\ 1 & x_2 & x_2^2 & x_2^3 & \dots & x_2^n \\ 1 & x_3 & x_3^2 & x_3^3 & \dots & x_3^n \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_m & x_m^2 & x_m^3 & \dots & x_m^n \end{bmatrix} & \begin{bmatrix} \beta_0 \\ \beta_1 \\ \vdots \\ \beta_p \end{bmatrix} \\ \mathbf{Y} & \mathbf{X} & \boldsymbol{\beta} \end{array}$$

Polynomial Regression

Again, similar to the multiple linear regression, we define sum of squared residuals (SSR)

$$\|Y - X\beta\|_2^2 = (Y - X\beta)^\top (Y - X\beta)$$

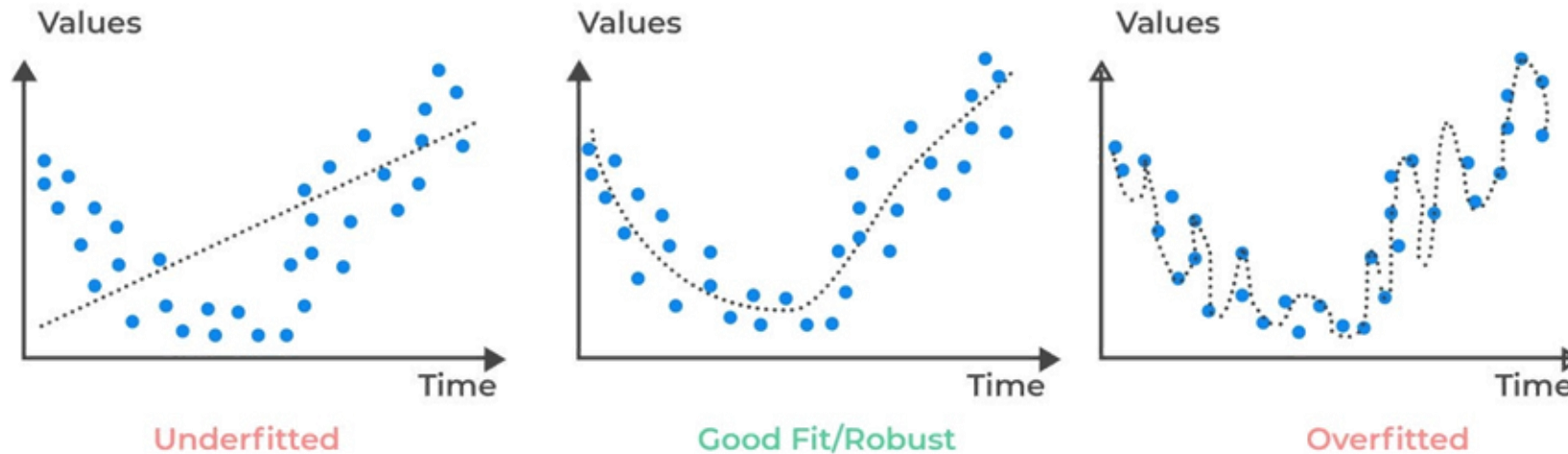
Minimizing the SSR leads to

$$\beta = (X^\top X)^{-1} X^\top Y$$

The solution shares the same form with multiple linear regression, while the definition of X is different.

Polynomial Regression

Considerations: Higher-degree polynomials can lead to overfitting

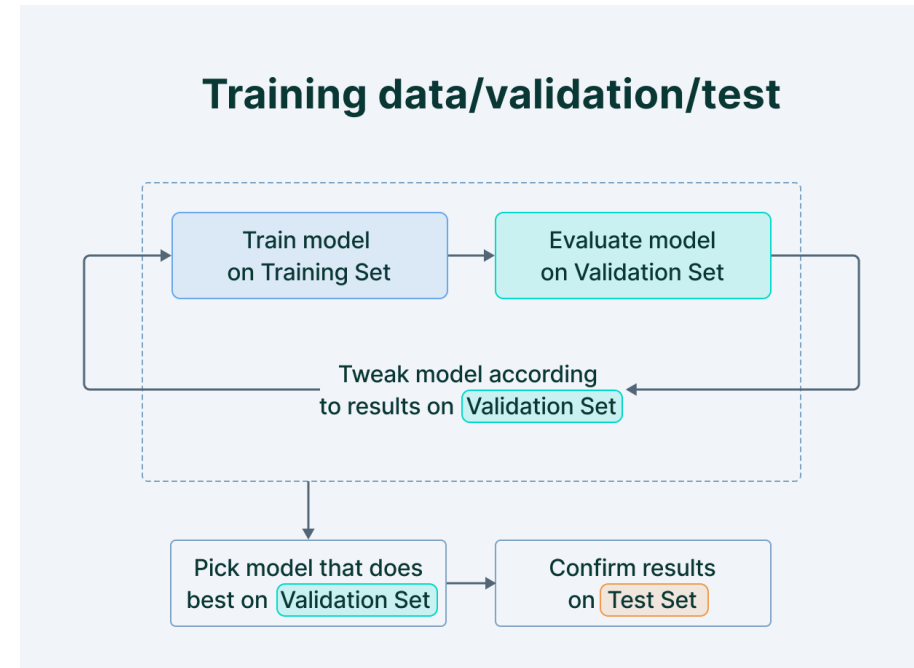


- How to choose the appropriate degree of polynomials?

Polynomial Regression

Choose the appropriate degree of polynomials

- Cross-Validation
- Regularization



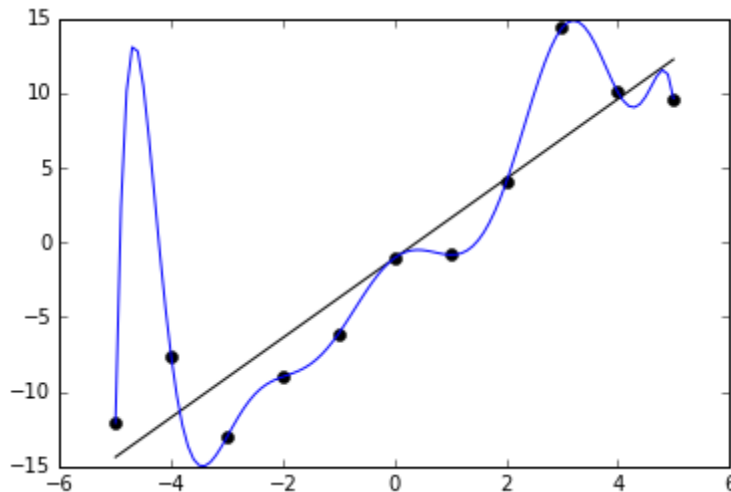
Types of Regression

Tradition Types of Regression Models

- Linear Regression
 - Simple Linear Regression
 - Multiple Linear Regression
- Polynomial Regression
- Regularization Methods
 - **Ridge Regression (L2 Regularization)**
 - Lasso Regression (L1 Regularization)

Ridge Regression

Overfitting



An example of overfitting

Although the high-order polynomial function is a perfect fit for the data, the fitting can not be generalized to new data points (**overfitting**).

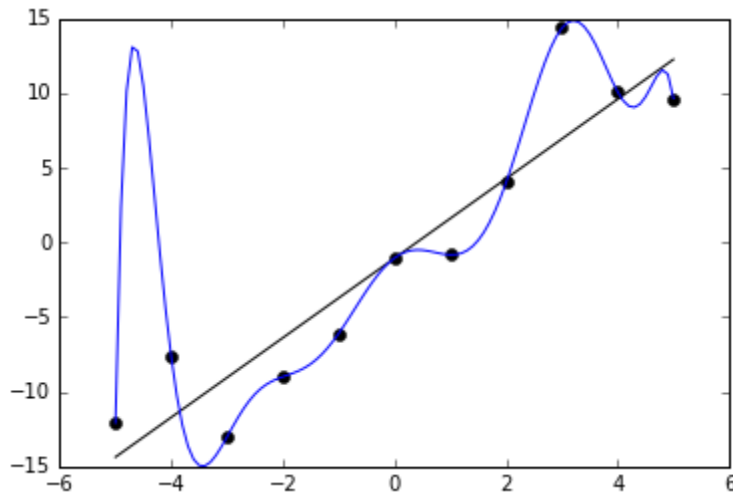
- The model is too complex
- The data samples may not be independent

In the overfitting case, **some parameters are “extremely” activated**.

- The fitted curve can have more “wiggles” to capture more of the data points. Then the model may need to use very large positive and negative parameters, especially for higher-order terms.
- The model is too complex such that some data “noise” are also modelled.

Ridge Regression

Regularization



An example of overfitting

Regularization is introduced to allow for some errors for the data to be fit, while balancing the activation of the different model parameters.

Adds a penalty term to the ordinary least squares (OLS) regression to regularize the coefficient estimates.

It helps to capture the underlying trend rather than fitting the training data very closely, including its noise and outliers.

Ridge Regression

Adds the L2-norm penalty on the value of coefficients to the original OLS.

$$\|\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_2^2$$

Allow some errors, but keep the overall values of coefficients to be small

λ : Tuning parameter that controls the strength of the penalty term.

$\|\boldsymbol{\beta}\|_2^2 = \sum_{j=1}^p \beta_j^2$ is the L2 norm.

The L2 penalty encourages smaller, more diffuse coefficient values, leading to a model that is less likely to overfit to the training data.

The L2 norm, also known as the Euclidean norm, is a measure of the magnitude of vectors in a Euclidean space.

General k-norm $\|\boldsymbol{\beta}\|_k = \left(\sum_{j=1}^p |\beta_j|^k\right)^{1/k}$

Ridge Regression

Choosing λ

$$\|Y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

λ : Tuning parameter that controls the strength of the penalty term.

- Small λ leads to a solution closer to OLS, which might have high variance.
- Large λ Increases the penalty, potentially leading to high bias but reduced variance.
- Typically selected via cross-validation to find the optimal balance that minimizes prediction error.

Ridge Regression

Solving

$$\|Y - X\beta\|_2^2 + \lambda\|\beta\|_2^2$$

Differentiating the objective function with respect to β and set the derivative equal to zero, we have

$$(X^T X + \lambda I)\beta = X^T Y$$

Then

$$\beta = (X^T X + \lambda I)^{-1} X^T Y$$

Types of Regression

Tradition Types of Regression Models

- Linear Regression
 - Simple Linear Regression
 - Multiple Linear Regression
- Polynomial Regression
- Regularization Methods
 - Ridge Regression (L2 Regularization)
 - **Lasso Regression (L1 Regularization)**

Lasso Regression

Now let us recheck the example

Consider an example where we want to predict the price of houses based on various features

- Total square footage of the house
- Number of bedrooms
- Number of bathrooms
- Age of the house (in years)
- Distance to the nearest city center
- Age of owner

We use the multiple linear regression to estimate the price based on different factors

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

However, some factors (e.g., age of owner) are not useful! In many cases, we have designed “over-complicated” models by considering all different factors, which will lead to overfitting.

Lasso Regression

Lasso regression aims to address the overfitting by adding regularization, while at the same time, finds out the key input factors.

- Different with the ridge regression, Lasso regression uses “L1” norm.

$$\|\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}\|_2^2 + \lambda\|\boldsymbol{\beta}\|_1$$

$$\|\boldsymbol{\beta}\|_1 = \sum_{j=1}^p |\beta_j| \quad \text{is the L1 norm.}$$

L1 norm generates sparser solutions!

Lasso Regression

$$\|Y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

L1 norm generates sparser solutions!

- Being sparse, it means only a few vector elements are large values, while the other elements are close to zero.

[0, 0, 0, 3, 0, 0, 0, 0, 0, 2] Sparse Vector

[1, 4, 2, 3, 3, 5, 3.1, 0, 2, 2] Dense Vector

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \dots + \beta_p X_p + \varepsilon$$

If sparse solutions can be obtained for β , we can identify the really useful factors, and make the weights to non-useful factors to zero.

Lasso Regression

Choosing λ

$$||\mathbf{Y} - \mathbf{X}\boldsymbol{\beta}||_2^2 + \lambda ||\boldsymbol{\beta}||_1$$

- λ : Tuning parameter that controls the strength of the penalty term.
- Small λ leads to a solution closer to OLS, which might have high variance.
 - Large λ Increases the penalty, potentially leading to high sparsity.
 - Typically selected via cross-validation to find the optimal balance that minimizes prediction error.

Lasso Regression

Solving

$$\|Y - X\beta\|_2^2 + \lambda\|\beta\|_1$$

There is no closed-form analytic solution for Lasso Regression.

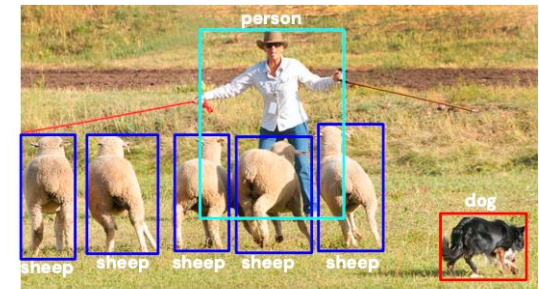
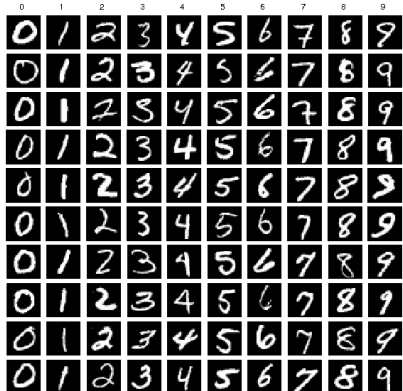
A lasso regression problem can be solved using coordinate descent in an iterative manner.

Details will not be introduced. We can solve it using scikit-learn and will revisit this part later.

What is Supervised Learning

Supervised Learning can be used for both regression and classification
Classification

- Building a model based on input-output pairs, where the output has categorical values.



Types of Classification Problems

- Based on the classification target, the classification can be

Binary Classification



Spam or Not?

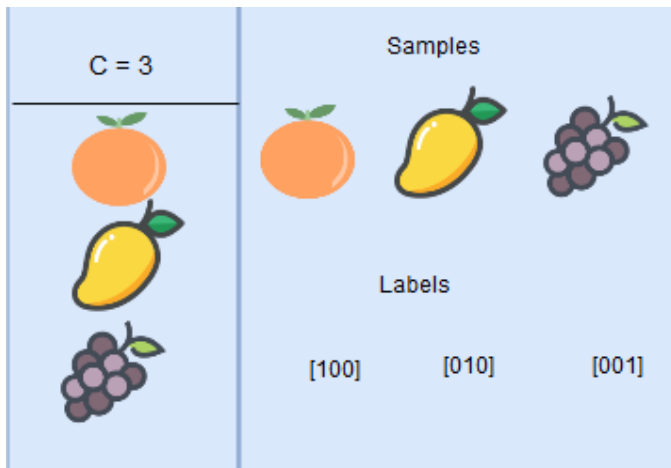
Other Examples?

- The number of classes is two
 - True/False
 - Existing or Not
 - Hot/Not Hot
 -
- Face verification
- Speech verification

Types of Classification Problems

- Based on the classification target, the classification can be

Multi-class Classification



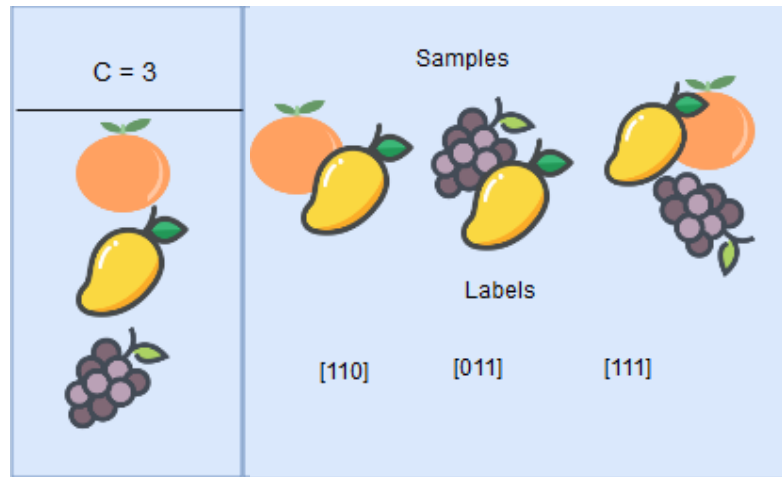
- The number of classes is more than two, and the sample is classified into only one of these categories.
 - Face recognition
 - Speech recognition
 - Object classification

Types of Classification Problems

Types of classification Problems

- Based on the classification target, the classification can be

Multi-label Classification



- The number of classes is more than two, and the sample can be assigned to multiple labels simultaneously.
 - Text Categorization

Performance Metrics for Classification

Confusion Matrix

- Considering the email classification problem, the classification results would be

- True Positive (TP):
 - The email is spam, and the machine predicts it to be spam.
 - Correct Result**

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Performance Metrics for Classification

Confusion Matrix

- Considering the email classification problem, the classification results would be

- False Positive (FP):
 - The email is NOT spam, and the machine predicts it to be spam. (false alarm)
 - Wrong Result**

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Performance Metrics for Classification

Confusion Matrix

- Considering the email classification problem, the classification results would be

- False Negative (FN):
 - The email is spam, and the machine predicts it to be NOT spam. (missed detection)
 - Wrong Result**

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Performance Metrics for Classification

Confusion Matrix

- Considering the email classification problem, the classification results would be
- True Negative (TN):
 - The email is NOT spam, and the machine predicts it to be NOT spam.
 - Correct Result**

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Performance Metrics for Classification

Accuracy

- The ratio of correctly predicted samples (**diagonal**) to the total samples
- Not always a reliable indicator, especially for imbalanced datasets

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

How many emails are correctly classified?

Performance Metrics for Classification

Precision

- The ratio of correctly predicted positive samples to the total predicted positives
- Important when the cost of **false positives** is high

$$\text{Precision} = \frac{TP}{TP + FP}$$

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Among detected spam emails, how many are real spam? (to make sure the detection is real)

Performance Metrics for Classification

Recall

- The ratio of correctly predicted positive samples to the all samples in actual class
- Important when the cost of **false negatives** is high

$$\text{Recall} = \frac{TP}{TP + FN}$$

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Among all **cancer patients**, how many are detected? (to cover the target samples as many as possible)

Performance Metrics for Classification

F1 Score

- The weighted average of Precision and Recall.
- Takes both false positives and false negatives into account.
- Particularly useful when the class distribution is uneven.

$$F1 = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

		Ground Truth	
		Positive	Negative
Prediction	Positive	TP	FP
	Negative	FN	TN

Performance Metrics for Classification

Multi-Class Cases

		Predicted			
		Cat	Dog	Pig	
Actual	Cat	40	20	10	
	Dog	35	85	40	
	Pig	0	10	20	
		Precision			
		Recall			

A large confusion matrix can be generated for multi-class classification problems.

- Overall accuracy can be computed by computing the ratio of **diagonal** samples over all samples
- The accuracy of each class can be computed by assigning different classification to TP/FP/TN/FN.
- Precision, Recall and F1 can be calculated for each class.

Performance Metrics for Classification

Multi-Label Cases

Ground Truth

Prediction	Ground Truth	
	Label 1 Present	Label 1 Absent
	Label 1 Present	Label 1 Absent
Label 1 Present	TP1	FP1
Label 1 Absent	FN1	TN1

A confusion matrix can be generated for each class for the multi-label classification.

- The Accuracy, Precision, Recall and F1 can be calculated for each class.

Supervised Learning Methods

Typical Supervised Learning Methods

For Regression

- Traditional Regression Methods and DNN-based Methods

For Classification

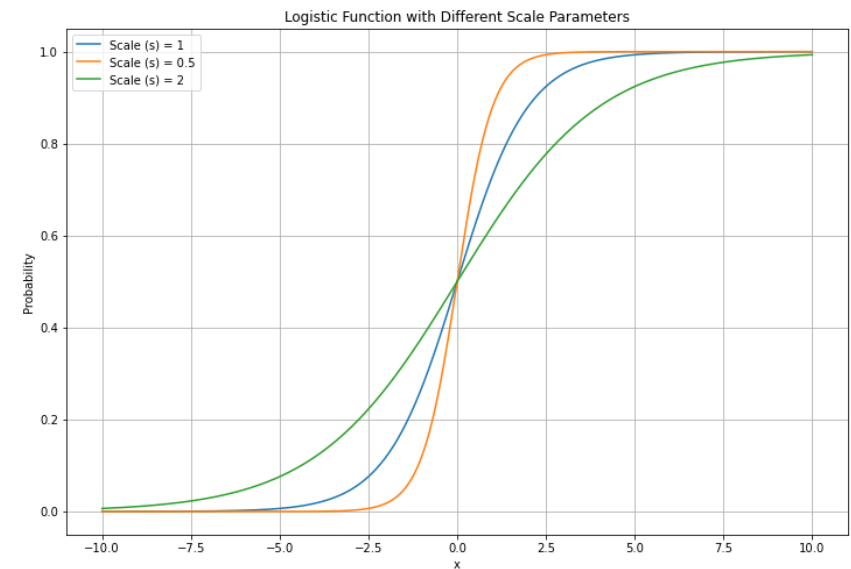
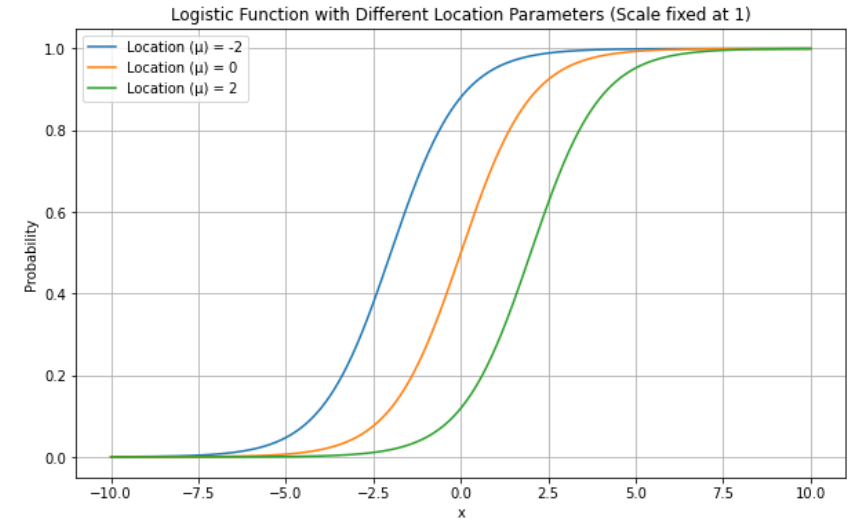
- Logistic Regression
- Naive Bayes
- k-Nearest Neighbors (k-NN)
- Decision Trees
- Support Vector Machines (SVM)

Logistic Regression

- The target is binary
- Outputs probabilities between 0 and 1
- Special design of the logistic function pushes the output values towards 0 and 1 (**S-shaped Curve**).

$$p(\mathbf{X}) = \frac{1}{1 + e^{-(\mathbf{X}-\mu)/s}}$$

- μ is the location parameter
- s is the scale parameter



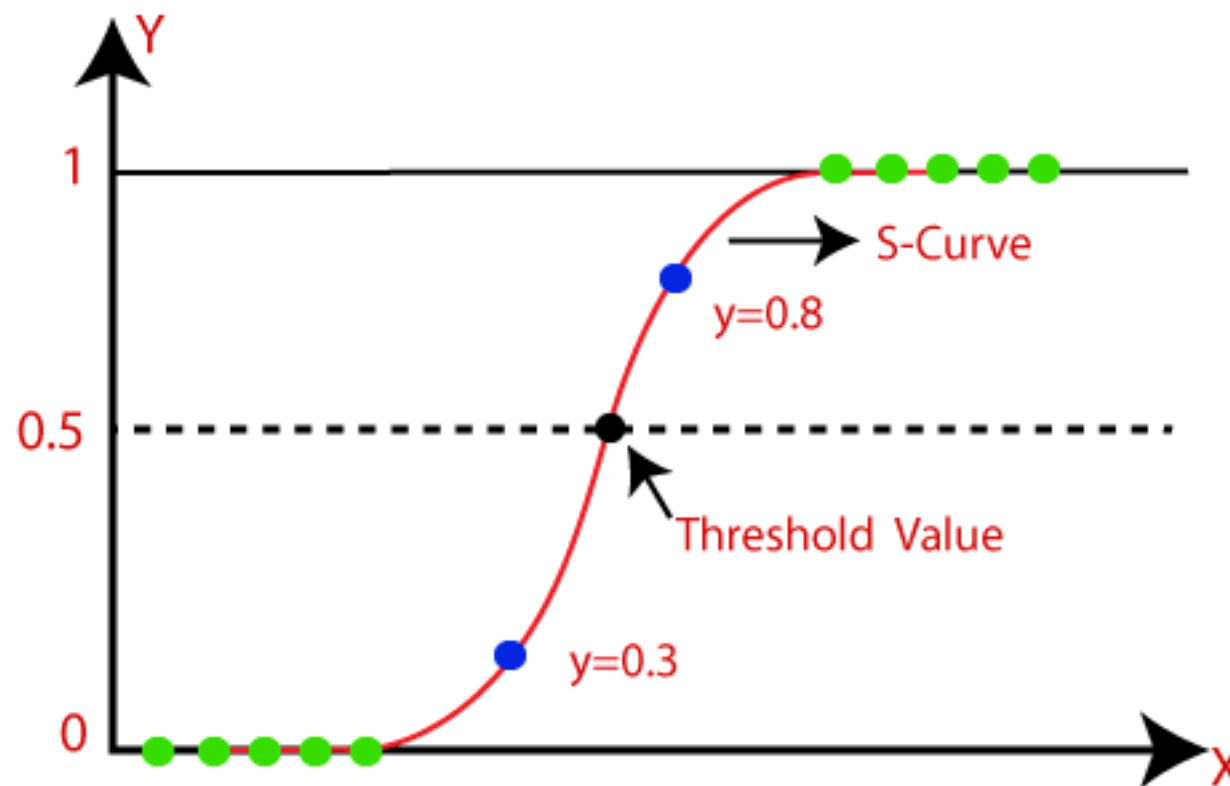
Logistic Regression

Estimating the parameters given the dataset.

$$p(\mathbf{X}) = \frac{1}{1 + e^{-(\mathbf{X} - \boldsymbol{\mu})/s}} \quad \frac{\mathbf{X}}{s} - \frac{\boldsymbol{\mu}}{s} = \ln\left(\frac{p}{p-1}\right)$$

We cannot apply OLS to solve this problem. Rather, we apply Maximum Likelihood Estimation to solve it, which will be studied in advanced courses.

Logistic Regression



Once we solve this problem, we can employ it for classification

Supervised Learning Methods

Typical Supervised Learning Methods

For Regression

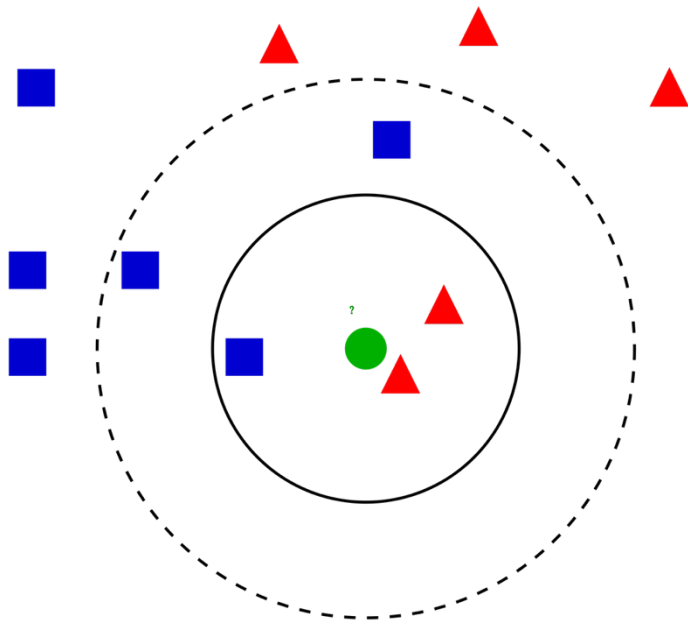
- Traditional Regression Methods and DNN-based Methods

For Classification

- Logistic Regression
- **k-Nearest Neighbors (k-NN)**
- Naive Bayes
- Decision Trees
- Support Vector Machines (SVM)

k-Nearest Neighbors (k-NN)

Given a dataset with features and labels, how to quickly determine the label for a new sample?



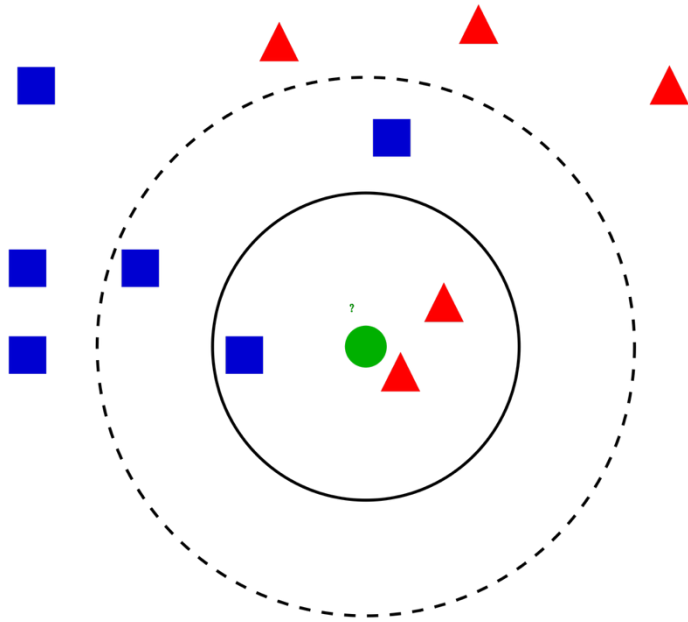
- Simple, non-parametric, and lazy learning algorithm
- Makes predictions simply based on the distance between samples.

Algorithm

- Choose the number of k and a distance metric.
- Find the k nearest neighbors of the sample that we want to classify.
- Predict the class of the sample by majority voting among neighbors.

k-Nearest Neighbors (k-NN)

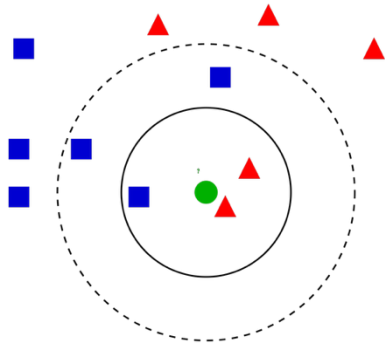
Choice of k



- Small k : less smoothing, more sensitive to noise
- Large k : more smoothing, but may include points from other classes (including irrelevant voters)
- Cross-Validation for choosing the suitable k .

k-Nearest Neighbors (k-NN)

Distance Measures



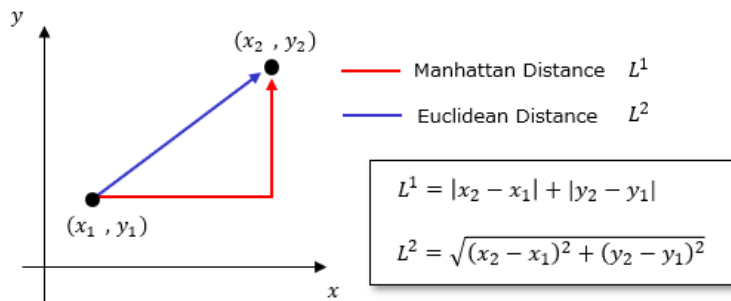
How to measure the similarity between samples?

Euclidean Distance: Most common distance metric (L2 Norm of Error)

$$d(\mathbf{A}, \mathbf{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

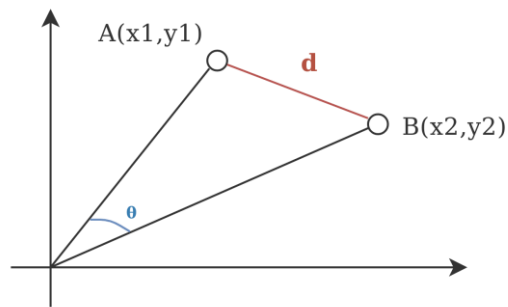
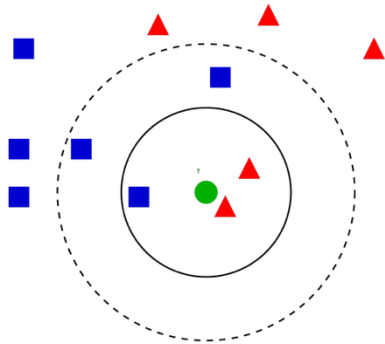
Manhattan Distance: Sum of absolute differences (L1 Norm of Error)

$$D_1(\mathbf{A}, \mathbf{B}) = \sum_{i=1}^n |A_i - B_i|$$



k-Nearest Neighbors (k-NN)

Distance Measures

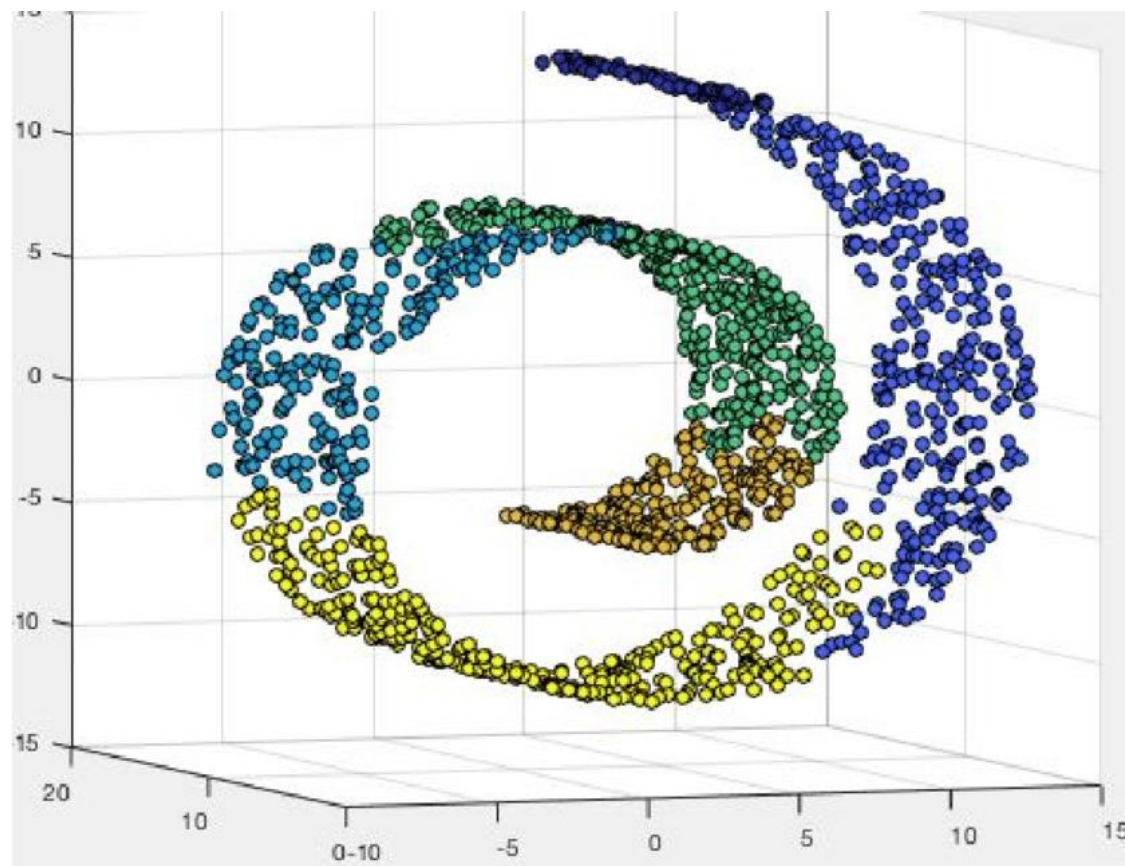


Cosine Similarity: Measures cosine of the angle between two vectors

$$\text{Cosine Similarity} = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|}$$

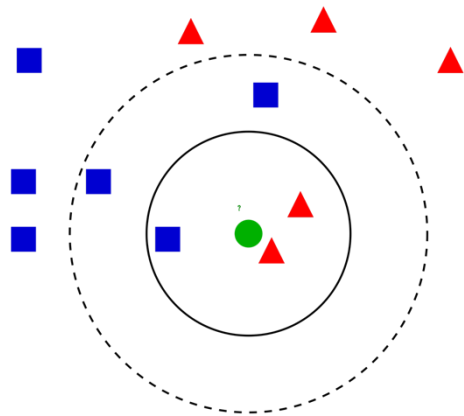
k-Nearest Neighbors (k-NN)

Advanced Topics: Local Linear Embedding (LLE)



k-Nearest Neighbors (k-NN)

Drawbacks



Computationally Intensive

- For each sample, you need to compute the distances to all samples to find out the neighbors!

Sensitivity

- The algorithm cannot distinguish between useful features and irrelevant features.

Supervised Learning Methods

Typical Supervised Learning Methods

For Regression

- Traditional Regression Methods and DNN-based Methods

For Classification

- Logistic Regression
- k-Nearest Neighbors (k-NN)
- **Naive Bayes**
- Decision Trees
- Support Vector Machines (SVM)

Naive Bayes

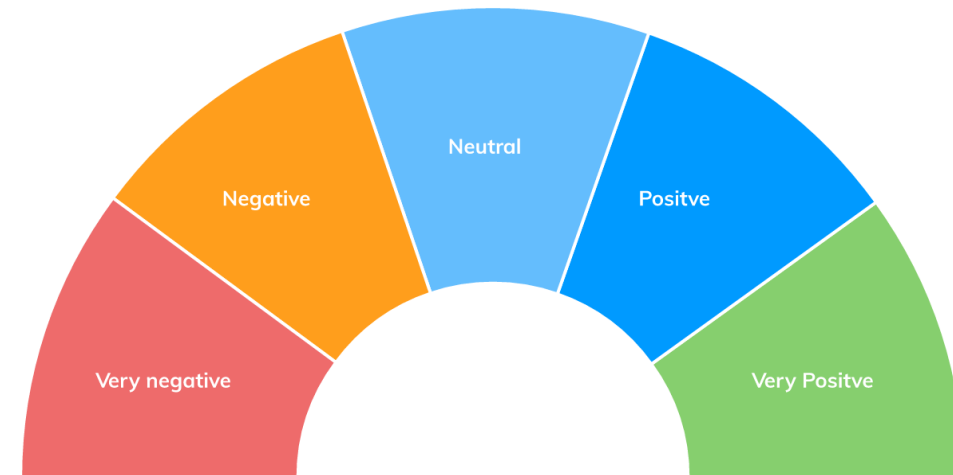
Based on **Bayes' Theorem** with an assumption of independent features

Key Features

- Simple and easy to implement
- Performs well with a small amount of training data.

Applications

- Spam filtering, sentiment analysis, etc.



Naive Bayes

Bayes' Theorem

Foundation

- Describes the probability of a class C based on input data X .

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)} \quad \frac{\text{Number of cases that both } C \text{ and } X \text{ appear}}{\text{Number of cases that } X \text{ appears}}$$

$P(C|X)$: Posterior probability, probability of class given the data

$P(X|C)$: Likelihood

$P(C)$: Prior probability

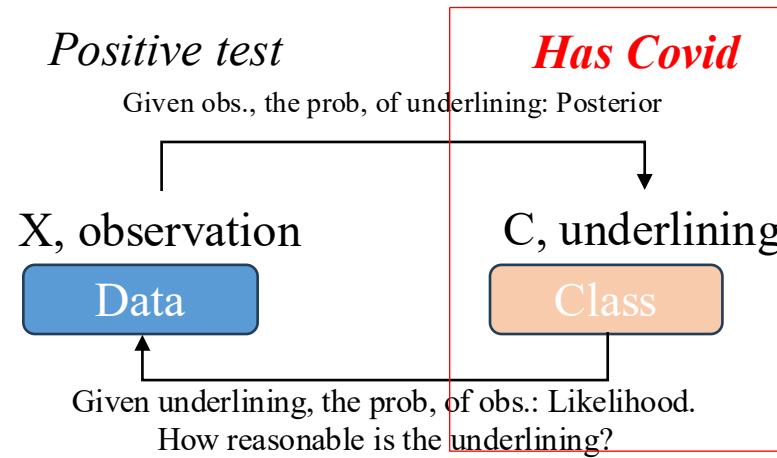
$P(X)$: Marginal probability

Let us see the real-world examples!

Naive Bayes

Bayes' Theorem

Real-World Examples



$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

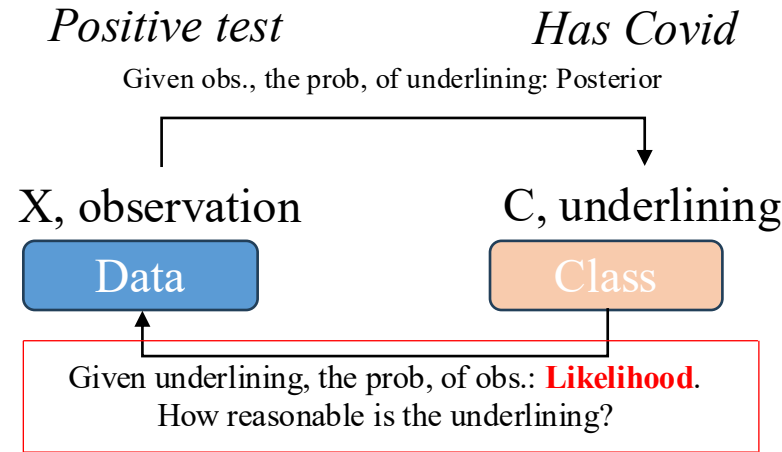
- Prior probability $P(C)$** The probability that a randomly chosen person has a certain disease (e.g., 2% of the population has COVID-19)
- Likelihood $P(X|C)$** The probability that a person with disease tests positive on a specific medical test (e.g., 99% of those with COVID-19 will have a positive test result).
- Marginal probability $P(X)$** The probability that a randomly chosen person **tests positive** on this medical test (considering both, healthy and sick people, includes **True Positive and False Positive**).
- Posterior probability $P(C|X)$** After seeing the test result, the probability that a person who tested positive **actually** has disease (e.g., given a positive test result, there is a 98.5% chance the person has COVID-19)

Naive Bayes

Bayes' Theorem

Real-World Examples

- Prior probability $P(C)$
The probability that a randomly chosen person has a certain disease (e.g., 2% of the population has COVID-19)
- Likelihood $P(X|C)$
The probability that a person with disease tests positive on a specific medical test (e.g., 99% of those with COVID-19 will have a positive test result).
- Marginal probability $P(X)$
The probability that a randomly chosen person **tests positive** on this medical test (considering both, healthy and sick people, includes **True Positive and False Positive**).
- **Posterior probability $P(C|X)$**
After seeing the test result, the probability that a person who tested positive **actually** has disease (e.g., given a positive test result, there is a 98.5% chance the person has COVID-19)



$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Naive Bayes

Bayes' Theorem

Real-World Examples

- Prior probability $P(C)$

The probability that a randomly chosen person has a certain disease (e.g., 2% of the population has COVID-19)

- Likelihood $P(X|C)$

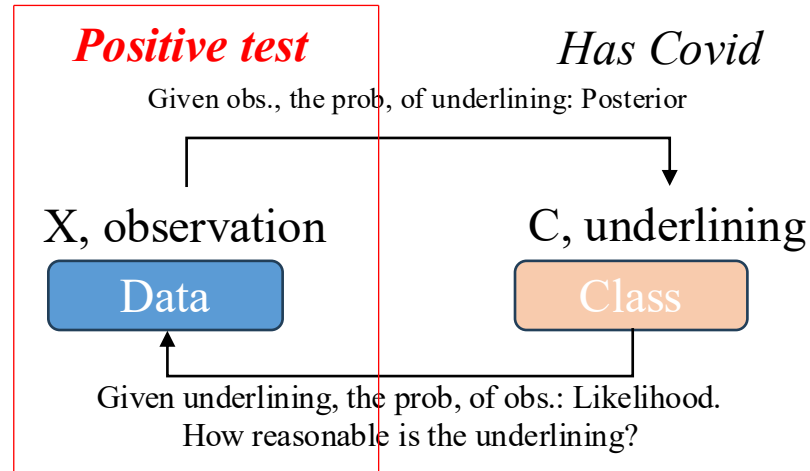
The probability that a person with disease tests positive on a specific medical test (e.g., 99% of those with COVID-19 will have a positive test result).

- Marginal probability $P(X)$

The probability that a randomly chosen person **tests positive** on this medical test (considering both, healthy and sick people, includes **True Positive and False Positive**).

- **Posterior probability $P(C|X)$**

After seeing the test result, the probability that a person who tested positive **actually** has disease (e.g., given a positive test result, there is a 98.5% chance the person has COVID-19)



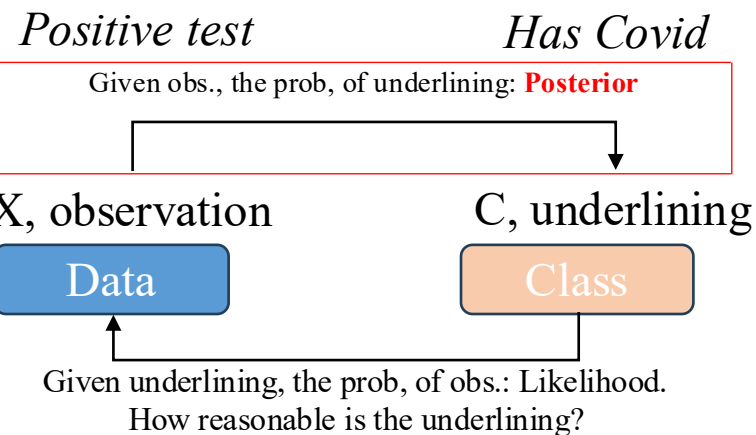
$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

Naive Bayes

Bayes' Theorem

Real-World Examples

- Prior probability $P(C)$
- Likelihood $P(X|C)$
- Marginal probability $P(X)$



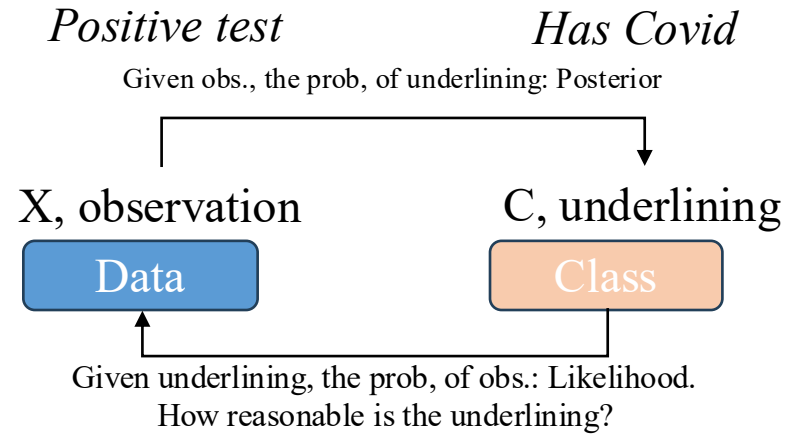
$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

- **Posterior probability $P(C|X)$** After seeing the test result, the probability that a person who tested positive **actually** has disease (e.g., given a positive test result, there is a 98.5% chance the person has COVID-19)

Naive Bayes

Bayes' Theorem

Real-World Examples



$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)}$$

- Let's consider the situation with COVID-19, **we want to determine the probability that a person actually has COVID-19 given a positive test.**
 - The probability of having COVID-19 (Prior probability, $P(C)$) is 0.02 (2% of the tested population is infected)
 - The probability of testing positive given having the disease (True Positive Rate, Likelihood, $P(X|C)$) is 0.99.
 - The probability of testing positive in the whole population (Marginal probability, $P(X)$) is 2.01%.

$$P(C|X) = \frac{P(X|C) \cdot P(C)}{P(X)} = \frac{0.99 \cdot 0.02}{0.0201} \approx 0.985$$

Naive Bayes

Naive Assumption

- Computing $P(X|C)$ can be difficult.
- Assumes all features are independent from each other given the class label
- Simplifies computation
 - The joint probability of a set of features can be computed as the product of their individual probabilities
- Naive Bayes Equation

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

- Where C is the class, X is the feature set, and x_i is a specific feature

Naive Bayes

Solving Naive Bayes Equation

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

- Given the dataset with C the class, X the n-dim feature set, then $P(C)$ can be computed directly by counting the labels.
- For $P(x_i|C)$, to compute the probability for any value of x_i , we need to build parametric statistical models, like Gaussian distribution.

Naive Bayes

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

Examples

- Now we have a dataset with two features and a binary class label.
- We want to predict the class of a new sample **(2.0,1.5)**

Step 1: Calculate $P(C)$

From the dataset, we have

Computing Prior

$$P(C = 0) = \frac{3}{5} = 0.6$$

$$P(C = 1) = \frac{2}{5} = 0.4$$

X_1	X_2	y
2.3	1.9	0
1.8	1.0	0
3.2	2.5	1
2.1	0.5	0
3.0	2.9	1

Naive Bayes

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

Examples

- Now we have a dataset with two features and a binary class label.
- We want to predict the class of a new sample (2.0,1.5)

Step 2: Calculate $P(X_i|C)$

Building Statistical Models

Considering samples with $C = 0$, we have

$$\mu_{[x_1, x_2]|C=0} = \text{mean}\left(\begin{bmatrix} 2.3 & 1.9 \\ 1.8 & 1.0 \\ 2.1 & 0.5 \end{bmatrix}, \text{axis} = 0\right) = [2.067, 1.133]$$

$$\sigma_{[x_1, x_2]|C=0}^2 = \text{var}\left(\begin{bmatrix} 2.3 & 1.9 \\ 1.8 & 1.0 \\ 2.1 & 0.5 \end{bmatrix}, \text{axis} = 0\right) = [0.042, 0.336]$$

X_1	X_2	y
2.3	1.9	0
1.8	1.0	0
3.2	2.5	1
2.1	0.5	0
3.0	2.9	1

Computing the mean and variance

Naive Bayes

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

Examples

- Now we have a dataset with two features and a binary class label.
- We want to predict the class of a new sample (2.0,1.5)

Step 2: Calculate $P(X_i|C)$

Building Statistical Models

Then compute $P(X_1 = 2|C = 0)$ and $P(X_2 = 1.5|C = 0)$

$$P(x_1 = 2|C = 0) = \frac{1}{\sqrt{2\pi\sigma_{x_1|C=0}^2}} \exp\left(-\frac{(x_1 - \mu_{x_1|C=0})^2}{2\sigma_{x_1|C=0}^2}\right) = 1.845$$

$$P(x_2 = 1.5|C = 0) = \frac{1}{\sqrt{2\pi\sigma_{x_2|C=0}^2}} \exp\left(-\frac{(x_2 - \mu_{x_2|C=0})^2}{2\sigma_{x_2|C=0}^2}\right) = 0.563$$

X_1	X_2	y
2.3	1.9	0
1.8	1.0	0
3.2	2.5	1
2.1	0.5	0
3.0	2.9	1

Computing the probability density

Naive Bayes

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

Examples

- Now we have a dataset with two features and a binary class label.
- We want to predict the class of a new sample (2.0,1.5)

Step 2: Calculate $P(X_i|C)$

Similarly for $C = 1$

Building Statistical Models

X_1	X_2	y
2.3	1.9	0
1.8	1.0	0
3.2	2.5	1
2.1	0.5	0
3.0	2.9	1

Computing the mean and variance

$$\mu_{[x_1, x_2]|C=1} = \text{mean}\left(\begin{bmatrix} 3.2 & 2.5 \\ 3.0 & 2.9 \end{bmatrix}, \text{axis} = 0\right) = [3.1, 2.7]$$

$$\sigma_{[x_1, x_2]|C=1}^2 = \text{var}\left(\begin{bmatrix} 3.2 & 2.5 \\ 3.0 & 2.9 \end{bmatrix}, \text{axis} = 0\right) = [0.01, 0.04]$$

Computing the probability density

$$P(x_1 = 2|C = 1) = \frac{1}{\sqrt{2\pi\sigma_{x_1|C=1}^2}} \exp\left(-\frac{(x_1 - \mu_{x_1|C=1})^2}{2\sigma_{x_1|C=1}^2}\right) \approx 0$$

$$P(x_2 = 1.5|C = 1) = \frac{1}{\sqrt{2\pi\sigma_{x_2|C=1}^2}} \exp\left(-\frac{(x_2 - \mu_{x_2|C=1})^2}{2\sigma_{x_2|C=1}^2}\right) \approx 0$$

Naive Bayes

$$P(C|X) = \frac{P(C)P(X|C)}{P(X)} \propto P(C) \prod_{i=1}^n P(x_i|C)$$

Examples

- Now we have a dataset with two features and a binary class label.
- We want to predict the class of a new sample (2.0,1.5)

Step 3: Calculate $P(C|X)$

Computing Posterior for Each Class

For $C = 0$

$$P(C = 0|X) = P(C = 0)P(x_1|C = 0)P(x_2|C = 0) = 0.6 * 1.845 * 0.563 = 0.624$$

For $C = 1$

$$P(C = 1|X) = P(C = 1)P(x_1|C = 1)P(x_2|C = 1) \approx 0.4 * 0 * 0 = 0$$

Belongs to $C = 0$

X_1	X_2	y
2.3	1.9	0
1.8	1.0	0
3.2	2.5	1
2.1	0.5	0
3.0	2.9	1

Again, because we use probability density functions here, the summation of probabilities is not 1.

Supervised Learning Methods

Typical Supervised Learning Methods

For Regression

- Traditional Regression Methods and DNN-based Methods

For Classification

- Logistic Regression
- k-Nearest Neighbors (k-NN)
- Naive Bayes
- **Decision Trees**
- Support Vector Machines (SVM)

What is Decision Tree?

You may usually write codes like this (using a set of if-else decisions)

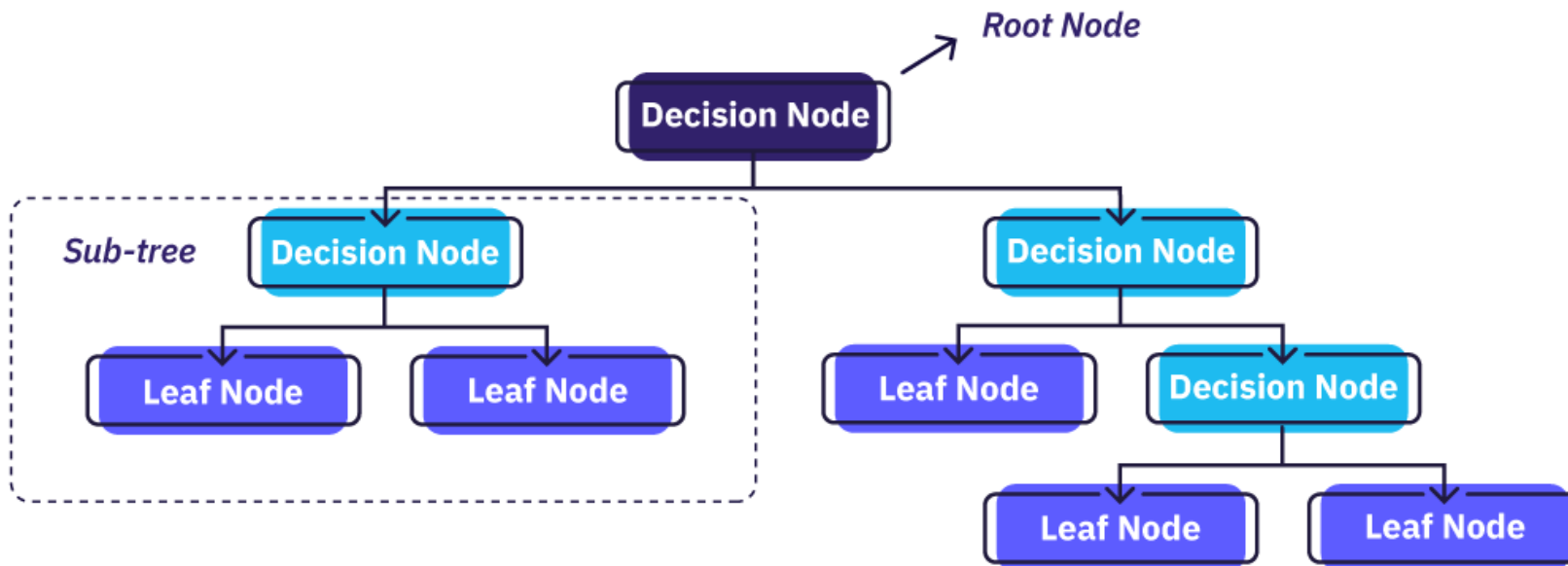
```
if(finalScore >= 90)
    cout << "You got A!" << endl;
else if(finalScore >= 80 && finalScore < 90)
    cout << "You got B!" << endl;
else if(finalScore >= 70 && finalScore < 80)
    cout << "You got C!" << endl;
else if(finalScore >= 60 && finalScore < 70)
    cout << "You got D!" << endl;
else
    cout << "You got F!" << endl;
```

Performing classification according to a set of decisions.

What is Decision Tree?

Decision Tree:

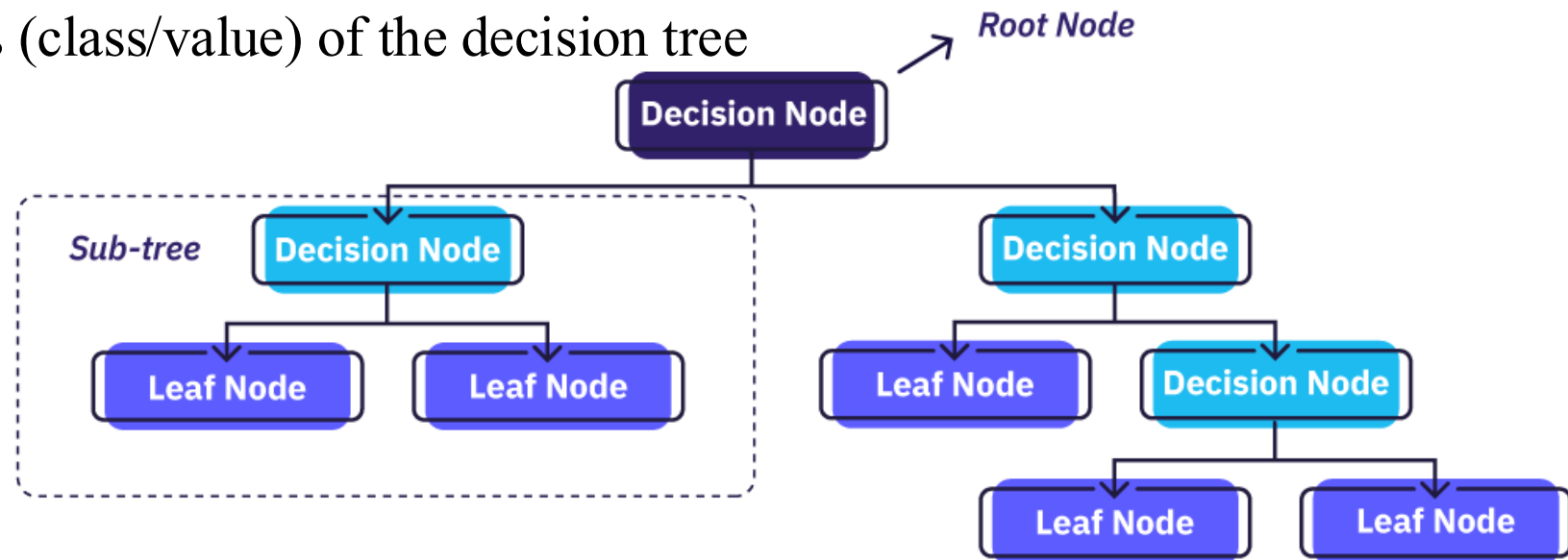
- Automates the process of building the decision conditions based on data
- Consists of a hierarchical tree-like model of decisions and their possible consequences



What is Decision Tree?

Components of Decision Tree

- Root Node: the starting point of the decision tree
- Branches: paths between nodes, representing decision rules
- Leaves: final outcomes (class/value) of the decision tree



What is Decision Tree?

Key Features of Decision Tree

- Interpretability: Easy to understand and interpret
- Non-Parametric: No need to assume the distribution of data (v.s. Naïve Bayes)
- Feature Selection: Important features are placed at the top of the tree.

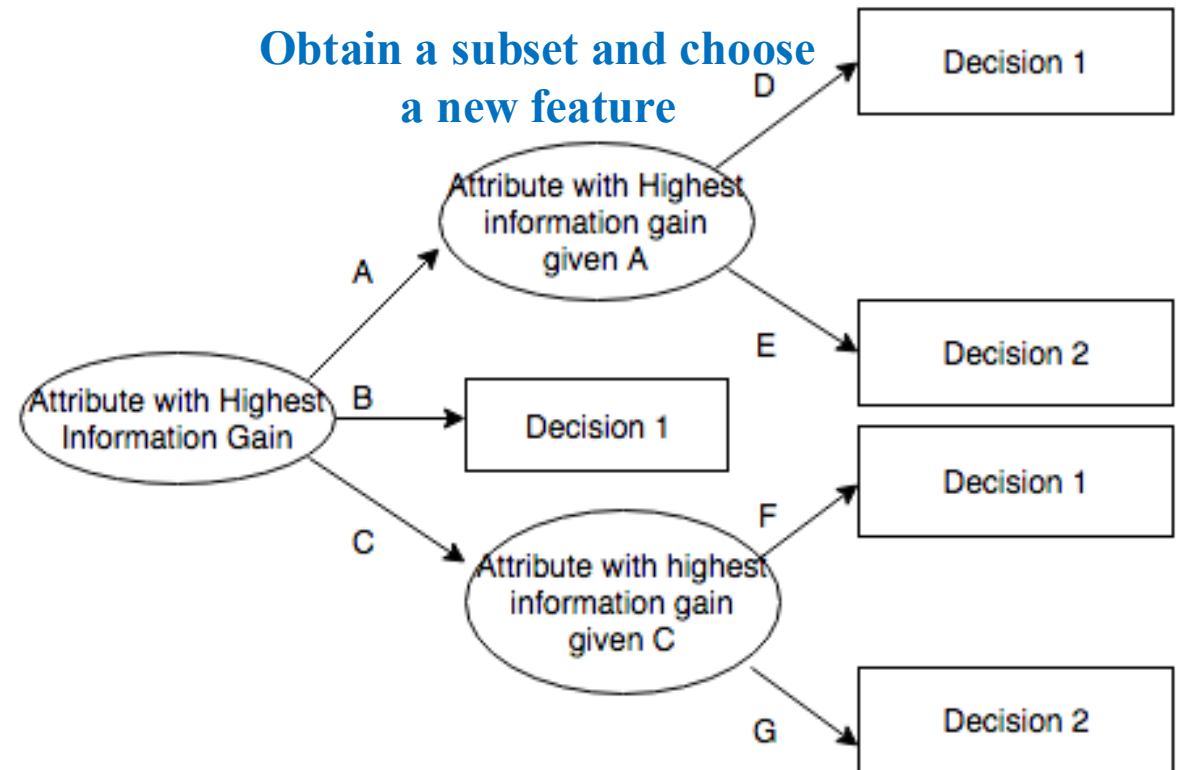
Building a Decision Tree

- ID3 (Iterative Dichotomiser 3)
 - Uses Entropy and Information Gain
- ~~C4.5~~
 - ~~Can handle both discrete and continuous attributes~~

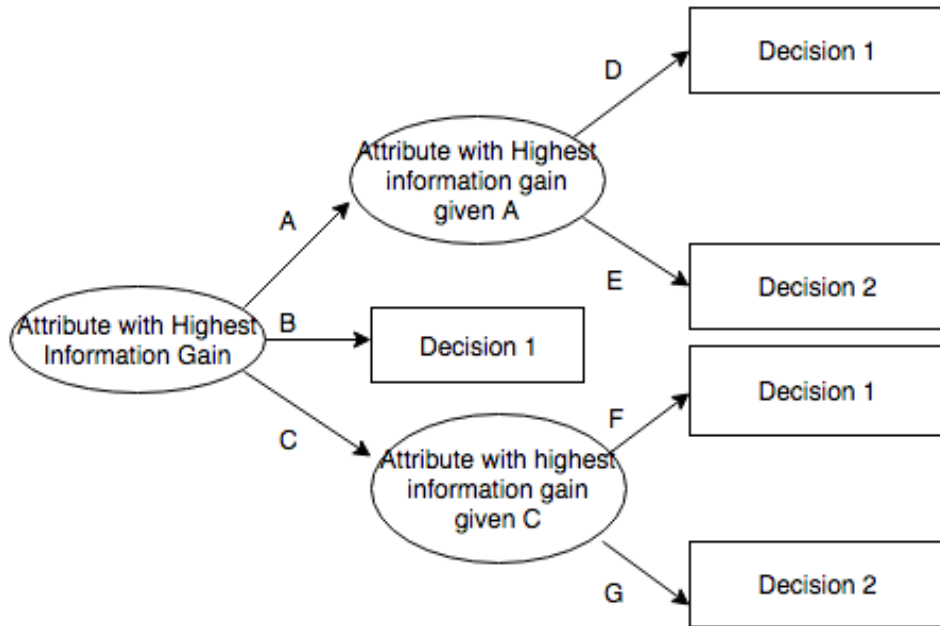
ID3 (Iterative Dichotomiser 3)

ID3 builds the decision tree

- By selecting the most powerful feature for classification at each level
- The process is iterative and continues until all data is classified or the stopping condition is met.



ID3 (Iterative Dichotomiser 3)



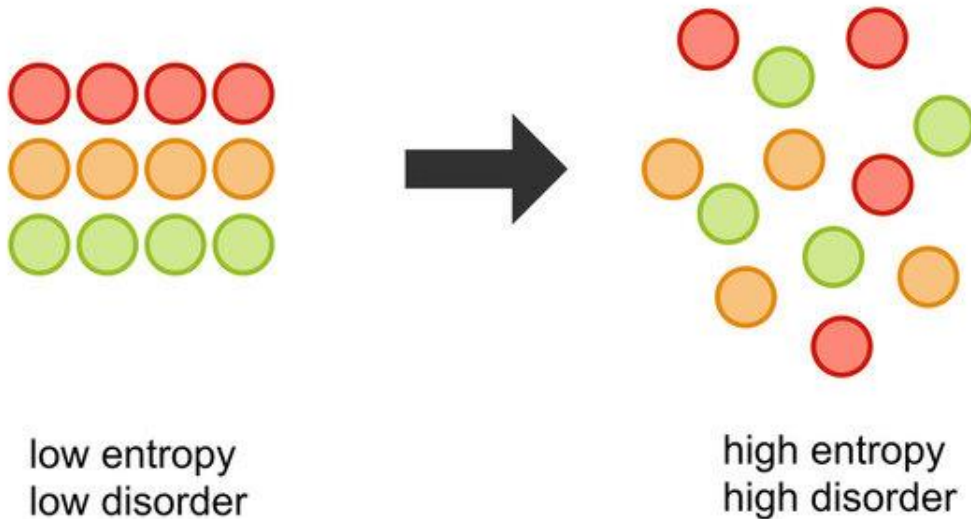
Information Gain

- Measuring the effectiveness of feature for classification
- To calculate the information gain, we need to understand “Entropy” first
 - Entropy: a measure of disorder or uncertainty

If a split reduces the disorder significantly, the split should be chosen.

ID3 (Iterative Dichotomiser 3)

Measure the disorder of data



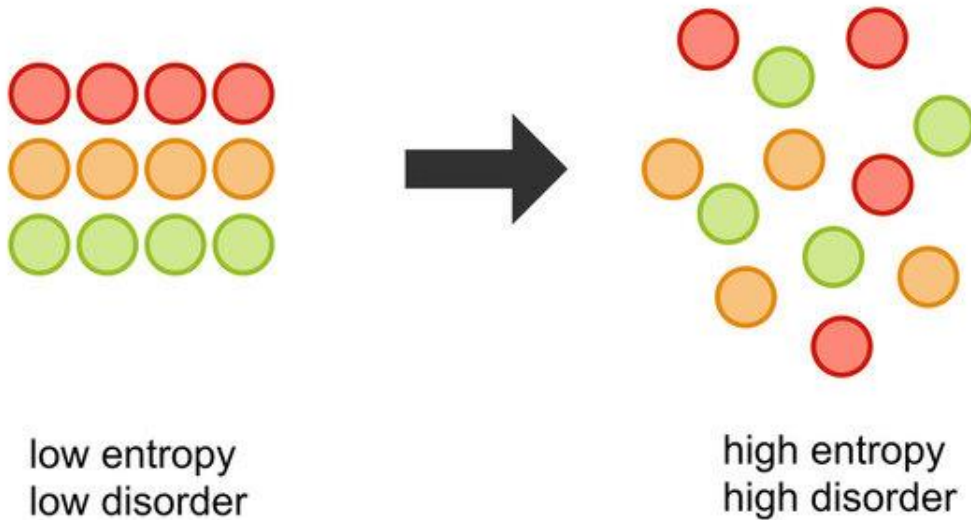
Entropy

$$H(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

- How to define the “information” of each event?
 - $-\log_2 p_i$ is zero if something will certainly happen ($p_i = 1$) – If it happens, it gives no information
 - If something rare (p_i is small) happens, the information $-\log_2 p_i$ is high!

ID3 (Iterative Dichotomiser 3)

Measure the disorder of data

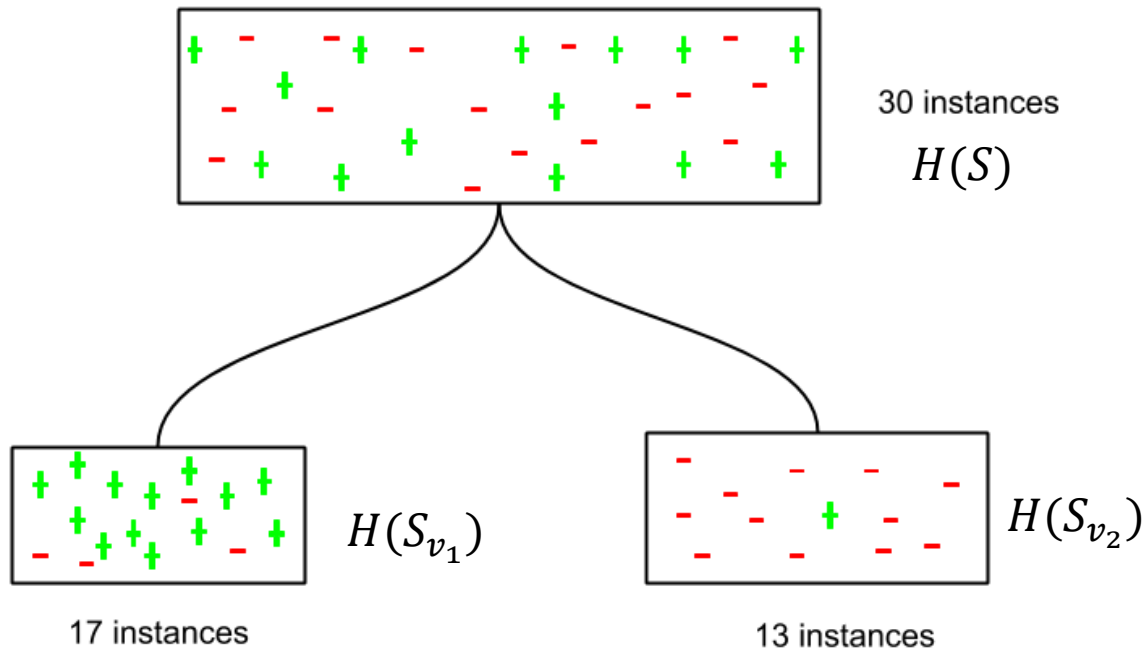


Entropy

$$H(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

- Entropy can be seen as the expectation of the defined “information” for each event (class label)
 - Weighted sum by p_i
 - n is the number of classes
- Measures the disorder of the dataset

ID3 (Iterative Dichotomiser 3)



An illustration of split over attribute A

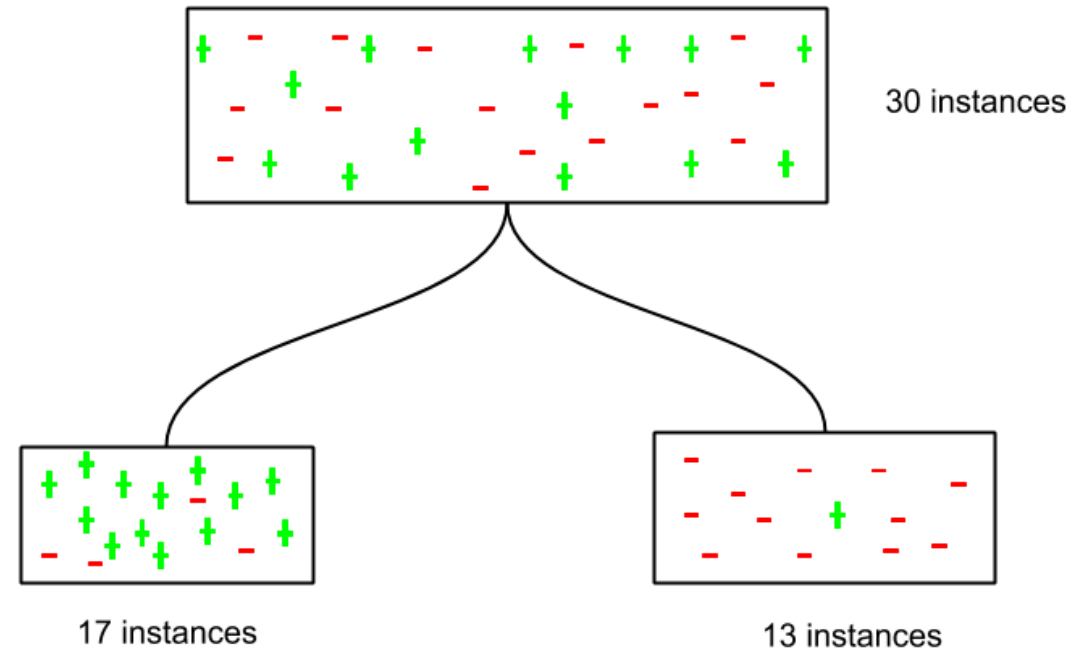
Information Gain

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

- For each category v in the attribute A , calculate the entropy $H(S_v)$
- Get the expectation over different possible values, which can be seen as the “entropy” of classes after splitting on A
- The **information gain** is calculated as the **decrease of entropy** after splitting on A

ID3 (Iterative Dichotomiser 3)

- A leaf node is created if all subset belongs to a single class.
- The process is repeated for remaining attributes.
- Top attributes are more important.



ID3 (Iterative Dichotomiser 3)

Example Data

- Calculate the entropy $H(S)$
 - $P(C = No) = 0.5, P(C = Yes) = 0.5$
 - $H(S) = -0.5 * \log_2 0.5 - 0.5 * \log_2 0.5 = 1$

A_1	A_2	C
0	0	No
1	0	No
0	1	Yes
1	1	Yes

ID3 (Iterative Dichotomiser 3)

Example Data

Now we split according to A_1

- Calculate the information gain on A_1

For $A_1 = 0$

- $P(C = No) = 0.5, P(C = Yes) = 0.5$
- $H(S_{A_1=0}) = -0.5 * \log_2 0.5 - 0.5 * \log_2 0.5 = 1$

For $A_1 = 1$

- $P(C = No) = 0.5, P(C = Yes) = 0.5$
- $H(S_{A_1=1}) = -0.5 * \log_2 0.5 - 0.5 * \log_2 0.5 = 1$

$$\text{Then } IG(S, A_1) = H(S) - \left(\frac{2}{4} * H(S_{A_1=0}) + \frac{2}{4} * H(S_{A_1=1}) \right) = 1 - (1) = 0$$

A_1	A_2	C
0	0	No
1	0	No
0	1	Yes
1	1	Yes

ID3 (Iterative Dichotomiser 3)

Example Data

Now we split according to A_2

- Calculate the information gain on A_2

For $A_2 = 0$

- $P(C = No) = 1, P(C = Yes) = 0$
- $H(S_{A_2=0}) = -1 * \log_2 1 - 0 * \log_2 0 = 0$

For $A_2 = 1$

- $P(C = No) = 0, P(C = Yes) = 1$
- $H(S_{A_2=1}) = -0 * \log_2 0 - 1 * \log_2 1 = 0$

$$\text{Then } IG(S, A_2) = H(S) - \left(\frac{2}{4} * H(S_{A_2=0}) + \frac{2}{4} * H(S_{A_2=1}) \right) = 1 - 0 = 1$$

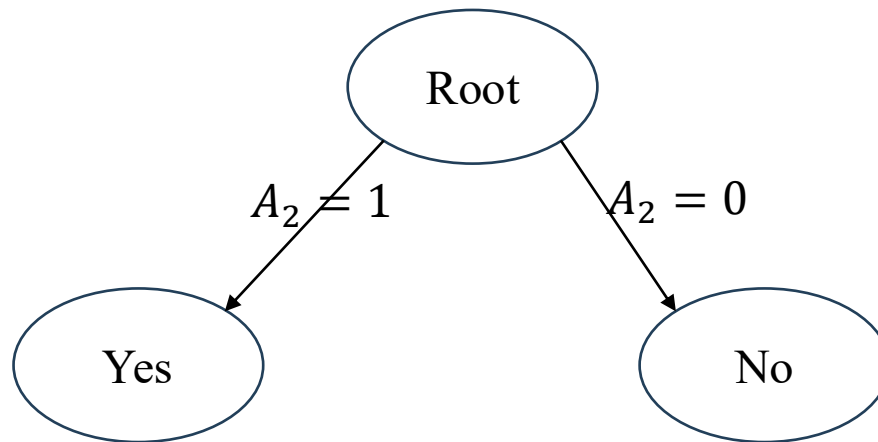
The entropy is 0, meaning all samples belong to the same class

A_1	A_2	C
0	0	No
1	0	No
0	1	Yes
1	1	Yes

ID3 (Iterative Dichotomiser 3)

Example Data

Comparing the information gains, we split on A_2



A_1	A_2	C
0	0	No
1	0	No
0	1	Yes
1	1	Yes

- This simple shows how to perform splitting on an “arbitrary dataset”, which may be a subset of one branch.
- The process can be performed recursively.

Building a Decision Tree

- ID3 (Iterative Dichotomiser 3)
 - Uses Entropy and Information Gain
- ~~C4.5~~
 - ~~Can handle both discrete and continuous attributes~~

C4.5 Algorithm

Gain Ratio

- C4.5 uses the Gain Ratio to select the attribute, which normalizes the Information Gain by the intrinsic split information of an attribute.

$$\text{Gain Ratio}(A) = \frac{\text{Information Gain}(A)}{\text{Split Information}(A)}$$

where

$$\text{Split Information}(A) = - \sum_{i=1}^v \frac{|S_i|}{|S|} \log_2 \frac{|S_i|}{|S|}$$

S_i is the subset of S that takes the i -th value of attribute A . The split information can be seen as the “entropy” of the attribute A itself.

C4.5 Algorithm

Another improvement of C4.5 is handling continuous values

House	Square (Cont. Val.)	Price Category
A	1200	Low
B	1300	Low
C	1500	High
D	1600	High
E	1700	Low
F	1800	High
G	2000	High

Define a threshold t for splitting

1. Sort all the unique values of the continuous attribute.
2. For each adjacent pair of values, compute a threshold at their midpoint.
3. Calculate the Gain Ratio for each threshold.
4. Choose the threshold with the highest Gain Ratio.

C4.5 Algorithm

Another improvement of C4.5 is handling continuous values

House	Square (Cont. Val.)	Price Category
A	1200	Low
B	1300	Low
C	1500	High
D	1600	High
E	1700	Low
F	1800	High
G	2000	High

Three potential split thresholds:

- $(1300+1500)/2 = 1400$
- $(1600+1700)/2 = 1650$
- $(1700+1800)/2 = 1750$

C4.5 Algorithm

Another improvement of C4.5 is handling continuous values

House	Square (Cont. Val.)	Price Category
A	1200	Low
B	1300	Low
C	1500	High
D	1600	High
E	1700	Low
F	1800	High
G	2000	High

$$H(S) = Entropy([0\ 0\ 1\ 1\ 0\ 1\ 1]) = 0.9852$$

For $t = 1400$

$$\begin{aligned}
 IG(t = 1400) &= H(S) - \frac{2}{7} * 0 - \frac{5}{7} * Entropy([1\ 1\ 0\ 1\ 1]) \\
 &= 0.4696
 \end{aligned}$$

C4.5 Algorithm

Another improvement of C4.5 is handling continuous values

House	Square (Cont. Val.)	Price Category
A	1200	Low
B	1300	Low
C	1500	High
D	1600	High
E	1700	Low
F	1800	High
G	2000	High

$$H(S) = Entropy([0\ 0\ 1\ 1\ 0\ 1\ 1]) = 0.9852$$

For $t = 1650$

$$\begin{aligned}
 IG(t = 1650) &= H(S) - \frac{4}{7} * Entropy([0\ 0\ 1\ 1]) - \frac{3}{7} \\
 &\quad * Entropy([0\ 1\ 1]) = 0.02
 \end{aligned}$$

C4.5 Algorithm

Another improvement of C4.5 is handling continuous values

House	Square (Cont. Val.)	Price Category
A	1200	Low
B	1300	Low
C	1500	High
D	1600	High
E	1700	Low
F	1800	High
G	2000	High

$$H(S) = Entropy([0\ 0\ 1\ 1\ 0\ 1\ 1]) = 0.9852$$

For $t = 1750$

$$\begin{aligned}
 IG(t = 1750) &= H(S) - \frac{5}{7} * Entropy([0\ 0\ 1\ 1\ 0]) - \frac{2}{7} \\
 &\quad * Entropy([1\ 1]) = 0.2917
 \end{aligned}$$

C4.5 Algorithm

Another improvement of C4.5 is handling continuous values

House	Square (Cont. Val.)	Price Category
A	1200	Low
B	1300	Low
C	1500	High
D	1600	High
E	1700	Low
F	1800	High
G	2000	High

Comparing between the three candidates, the best threshold for the attribute is 1400.

The best gain ratio is calculated by

$$\frac{IG(t = 1400)}{\text{Split Info}} = \frac{0.4696}{\text{Entropy}([0 \ 0 \ 1 \ 1 \ 1 \ 1 \ 1])} = 0.544$$

This gain ratio is compared with other attributes to determine whether splitting on the Square Attribute.

Brief Summary on Decision Tree

- Decision making tree based on data
- ID3
 - Find the attribute with the maximum information gain, a measure based on entropy
 - Entropy: measure of uncertainty or disorder
- C4.5
 - Performs normalization over the split information of the attribute, to avoid too many distinct values
 - Handles continuous values by comparing between different candidate thresholds

Supervised Learning Methods

Typical Supervised Learning Methods

For Regression

- Traditional Regression Methods and DNN-based Methods

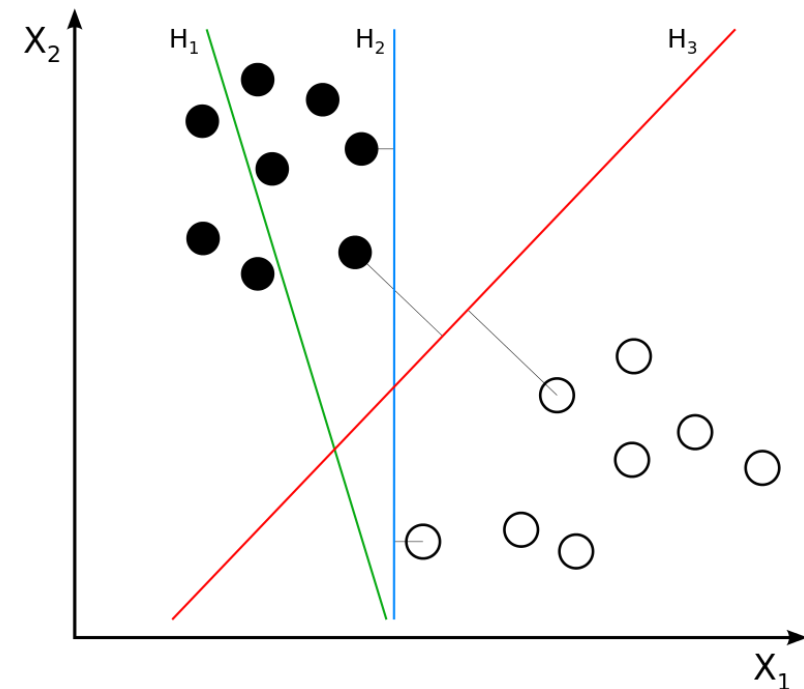
For Classification

- Logistic Regression
- k-Nearest Neighbors (k-NN)
- Naive Bayes
- Decision Trees
- Support Vector Machines (SVM)

Support Vector Machines (SVM)

SVM is widely regarded as one of the most important classifiers for machine learning

- SVM works by finding the hyperplane that best separates classes in the feature space



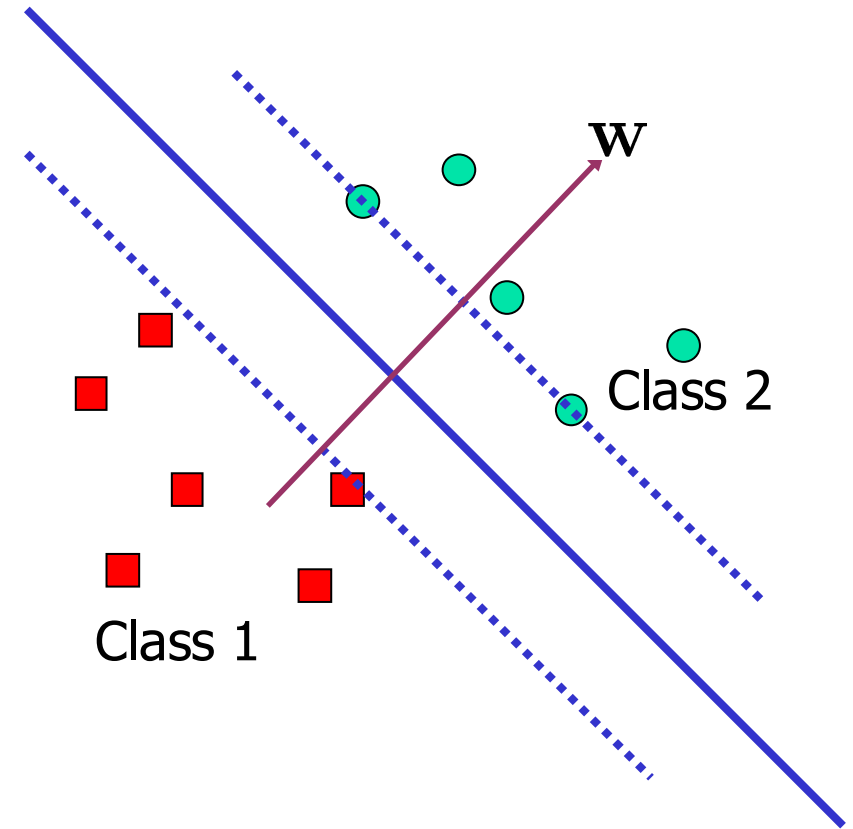
Which hyperplane is better?

Support Vector Machines (SVM)

SVM works by

1. Finding the data points that are **closest** to the hyperplane
2. **Maximize** the distances between the data points to the hyperplane

Data points of each class that are closest to the hyperplane: **support vectors**



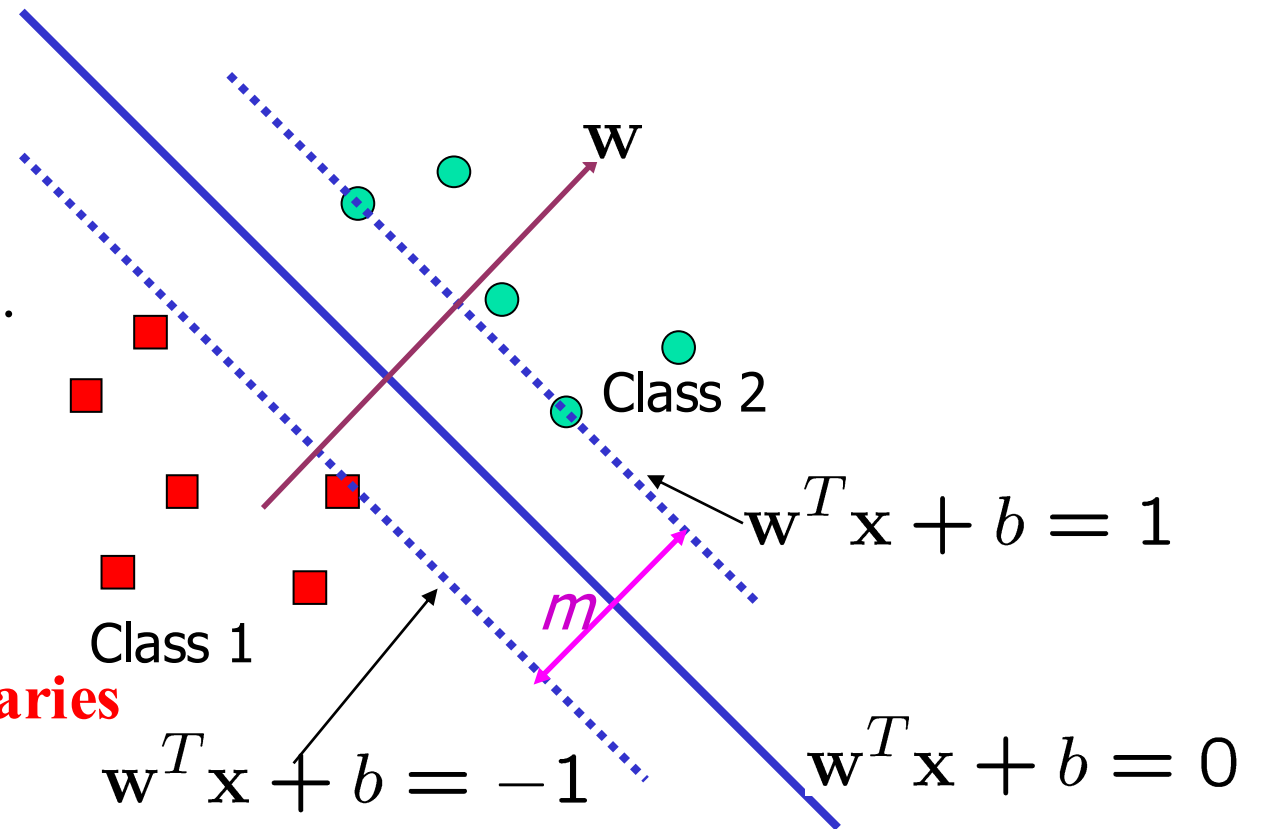
Linear SVM

Finding the optimal hyperplane

Consider the binary classification

- Defining the target class to be 1 and -1.
- The sample belongs to class 1 if
 - $\mathbf{w}^T \mathbf{x} + b \leq -1$
- The sample belongs to class 2 if
 - $\mathbf{w}^T \mathbf{x} + b \geq 1$
- The **margin between decision boundaries** is calculated by

$$m = \frac{2}{\|\mathbf{w}\|}$$



Linear SVM

Finding the optimal hyperplane

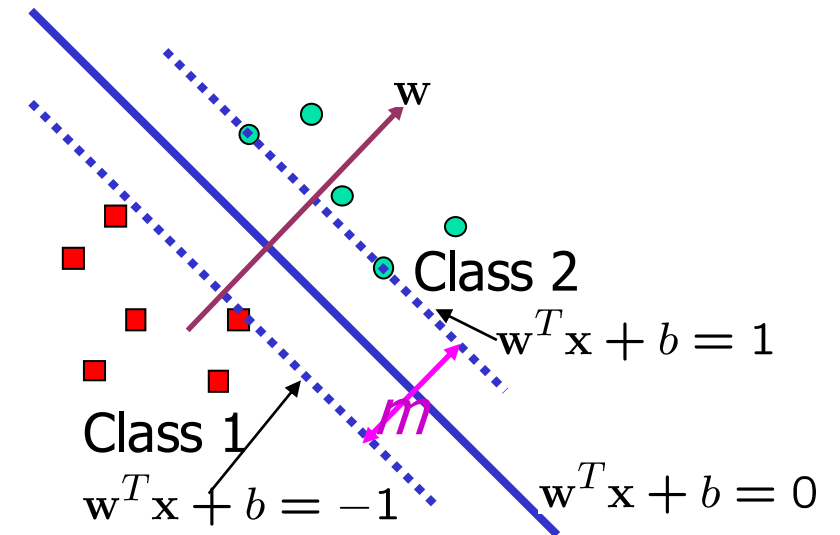
Consider the binary classification

$$\text{Minimize } \frac{1}{2} \|\mathbf{w}\|^2$$

$$\text{subject to } y_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 \quad \forall i$$

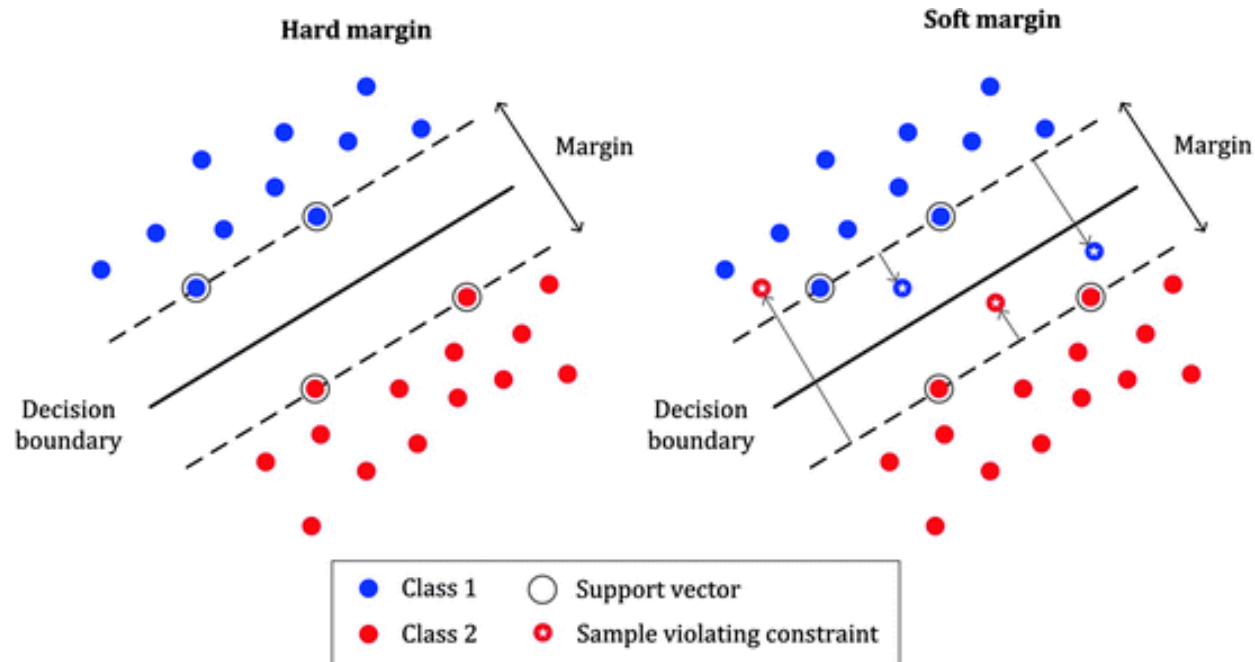
There is no closed-form solution because the optimization problem involves both inequality constraints and a convex quadratic objective.
Dual form, derived from Lagrange multipliers, can be solved (Skipped here).

Max-margin hyperplane is completely determined by those Support vectors that lie nearest to it.



Soft Margin SVM

Soft margin SVM is an extension of the basic SVM that allows for some misclassifications

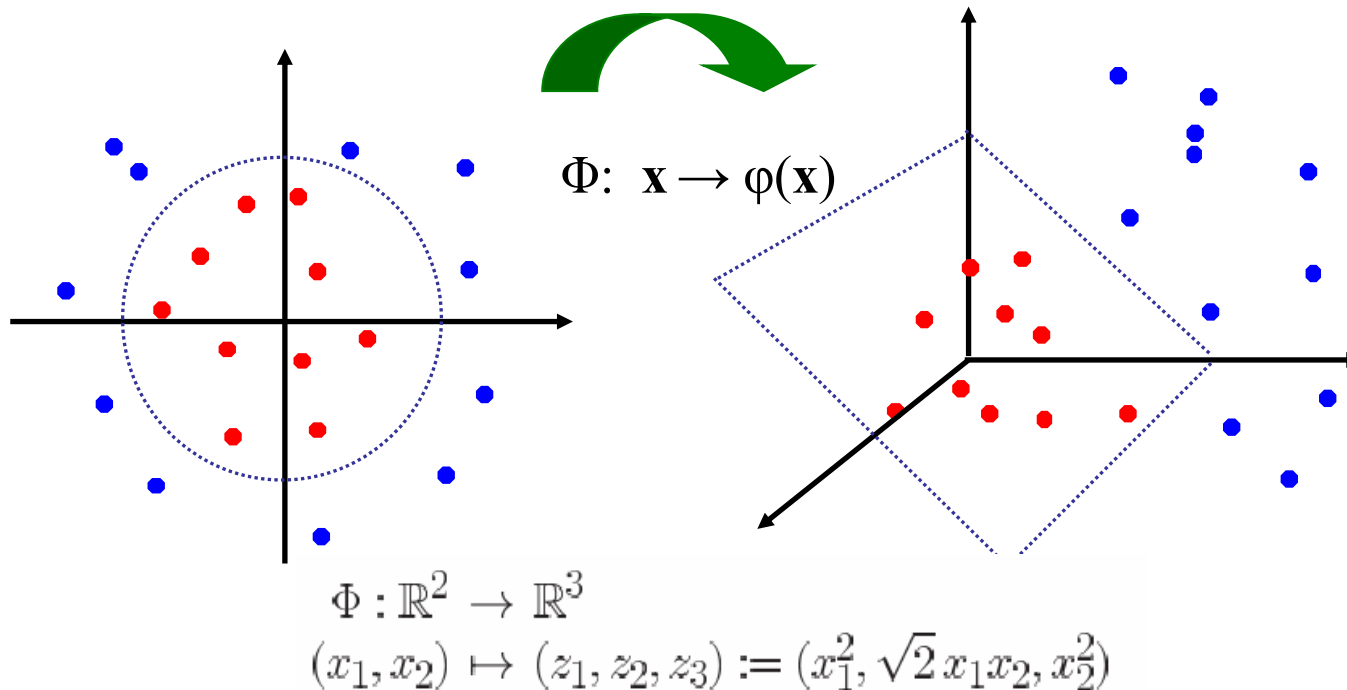


Non-negative Slack Variables

- $\xi_i = 0$ on the correct side of the margin
- $\xi_i \in (0,1]$ correctly classified but within the margin
- $\xi_i > 1$ misclassified.

Non-Linear SVM

- Many real-world data cannot be separated by a line or hyperplane.
- Non-linear SVM maps the input features into a higher-dimensional space where it is possible to find a hyperplane that separates the classes.



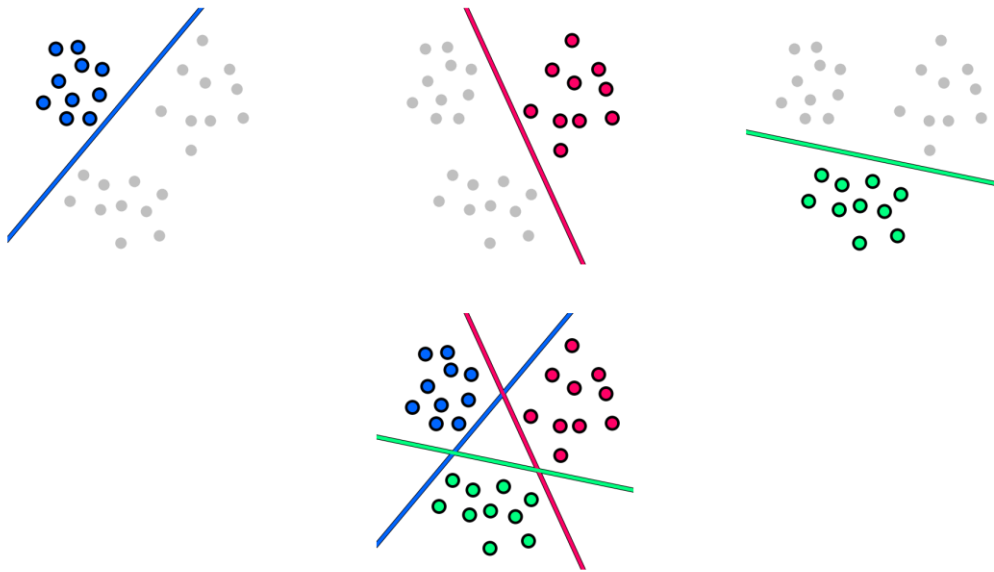
Input space: $\mathbf{x}$

Feature space: $\varphi(\mathbf{x})$

Feature mapping or transformation: φ

Multi-Class SVM

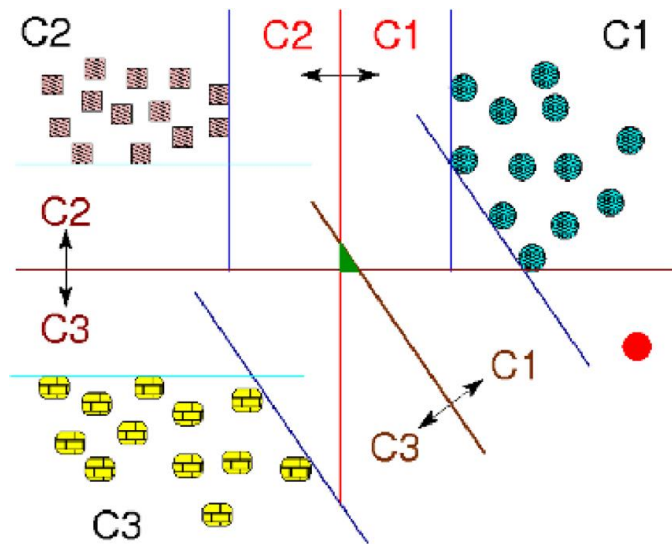
- SVM is originally designed for binary classification. For multi-class cases, we can have
 - One-vs-All / One-vs-Rest Strategy



- Train one SVM per class, where each classifier separates one class from the rest.
- During prediction, **the classifier with the highest score** determines the class label.

Multi-Class SVM

- SVM is originally designed for binary classification. For multi-class cases, we can have
 - One-vs-One Strategy



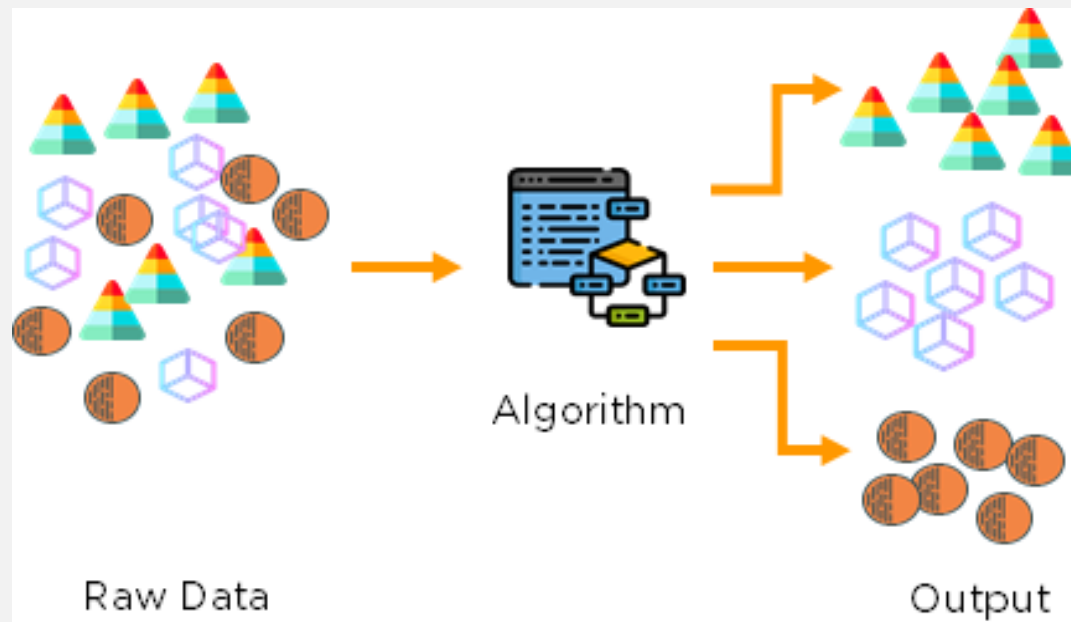
- Train one SVM for **each pair of classes**.
- There will be $N(N-1)/2$ classifiers for N classes.
- During prediction, a voting scheme is used where **each classifier votes for a class**, and the class with the most votes determines the instance's label.

Brief Summary on SVM

- Finding the separation hyperplane based on the support vectors
- Linear SVM
 - The concept of support vectors, the idea of optimization target.
- Soft Margin SVM
 - Allow errors by introducing slack variables
- Non-Linear SVM
 - Feature mapping to the high-dimension separable space
 - Extension to multi-class

What is Unsupervised Learning

Unsupervised Learning: Algorithms that identify patterns in data **without any labels**. The data difference can be automatically found.

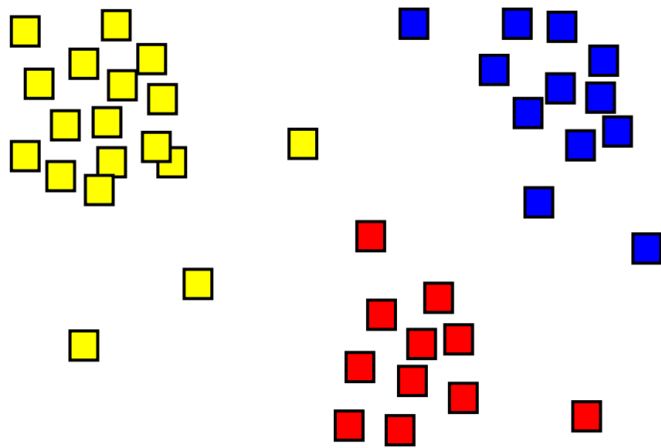


One key feature of unsupervised learning as compared with supervised learning, is **label-free**.

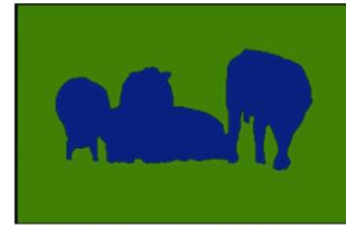
Types of Unsupervised Learning

Clustering

Partitioning a dataset into groups, such that the instances within each group are more similar to each other.



a



b

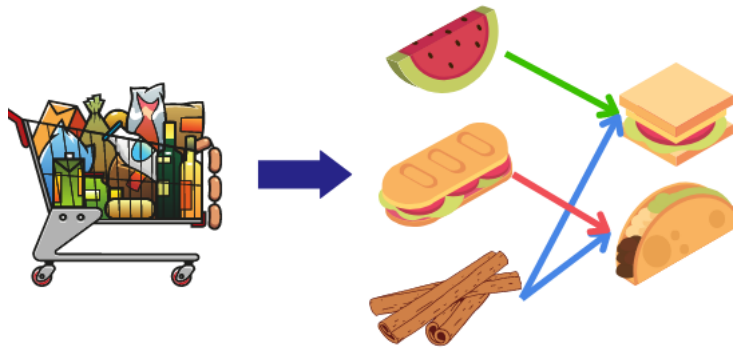


Clustering for image segmentation

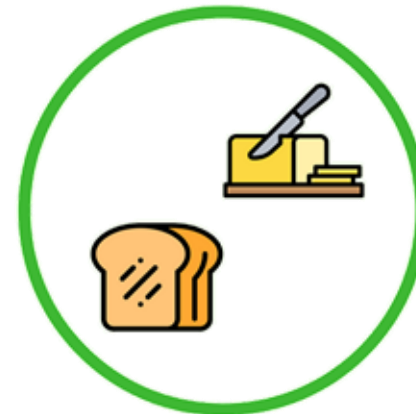
Types of Unsupervised Learning

Association Rule Learning

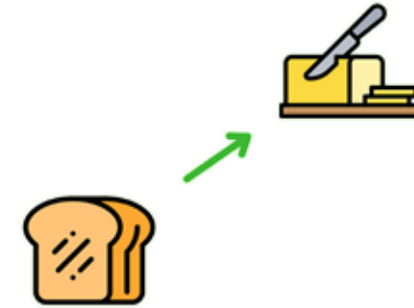
Discovering interesting relations between variables in large databases.



*"93% of people who purchased item A
also purchased item B"*



CLUSTERING

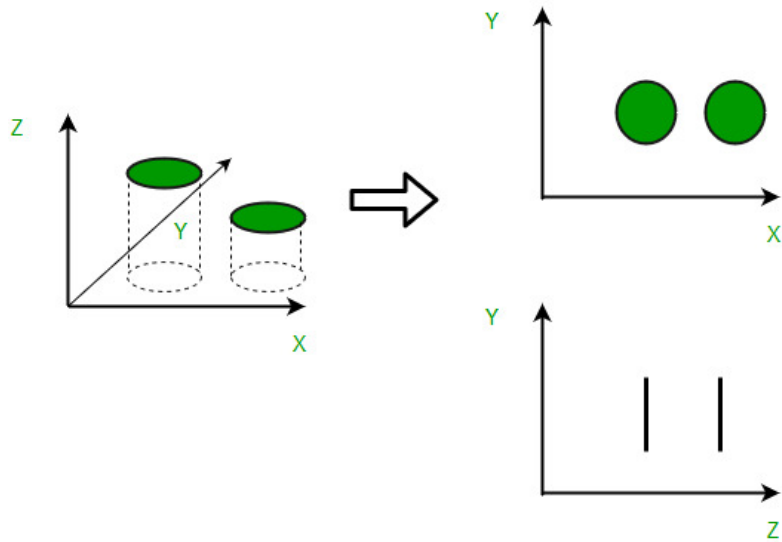


**ASSOCIATION RULE
MINING**

Types of Unsupervised Learning

Dimensionality Reduction

Reducing the number of variables to consider, which includes feature selection and feature extraction.



Feature selection picks up
useful features



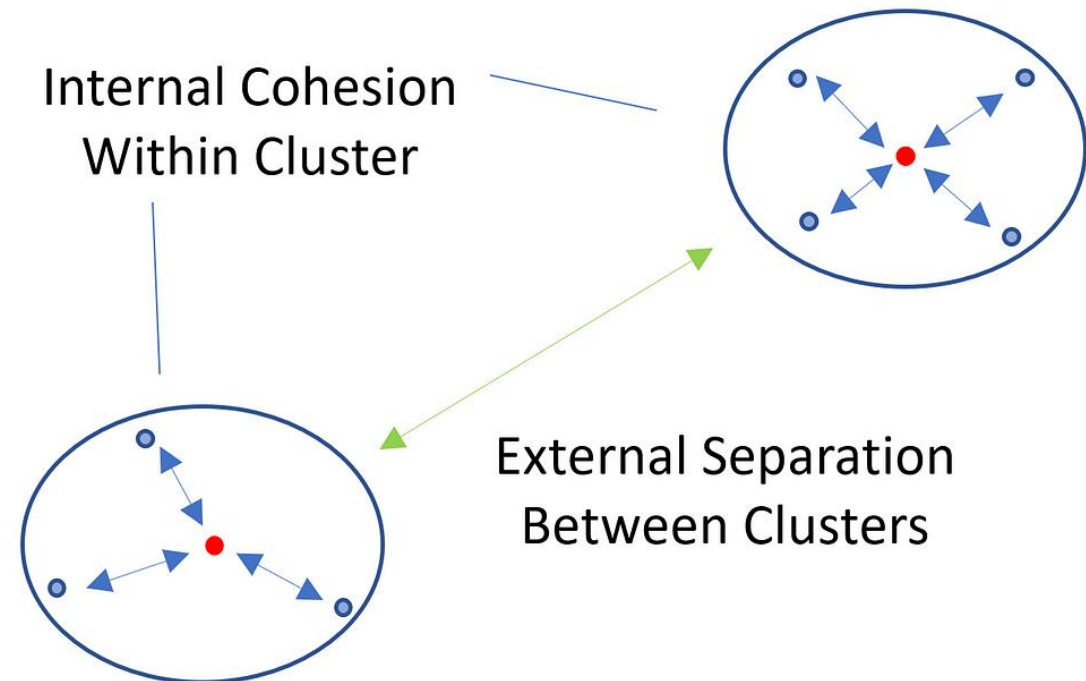
Feature extraction transforms
original features

Unsupervised Learning Outline

- **Clustering**
 - **Evaluation**
 - K-Means Clustering
 - Hierarchical Clustering
 - Data Normalization
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA

Clustering Evaluation

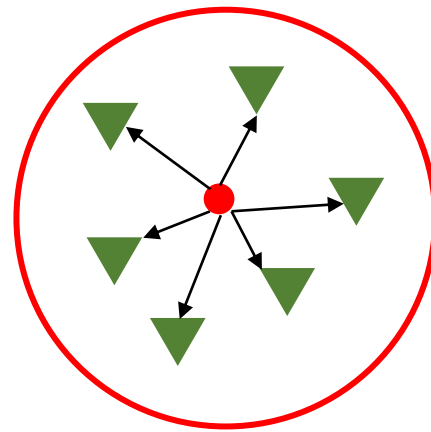
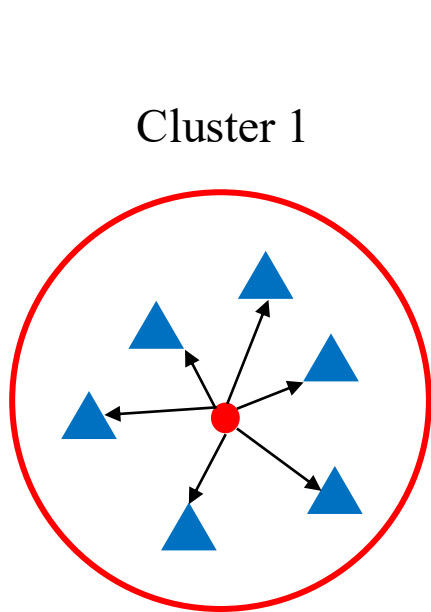
- Partitioning a dataset into clusters, such that the instances within each cluster are more similar to each other.
- Two goals of clustering:
 - Internal Compactness
 - External Separation



Clustering Evaluation

- **Inertia**

- The sum of distances of all the points within a cluster from the centroid of that cluster. The smaller the better.



Cluster 2

Intra-cluster Distance of One Cluster

$$\text{Inertia} = \sum_{i=1}^k \sum_{\mathbf{x} \in S_i} ||\mathbf{x} - \mu_i||^2$$

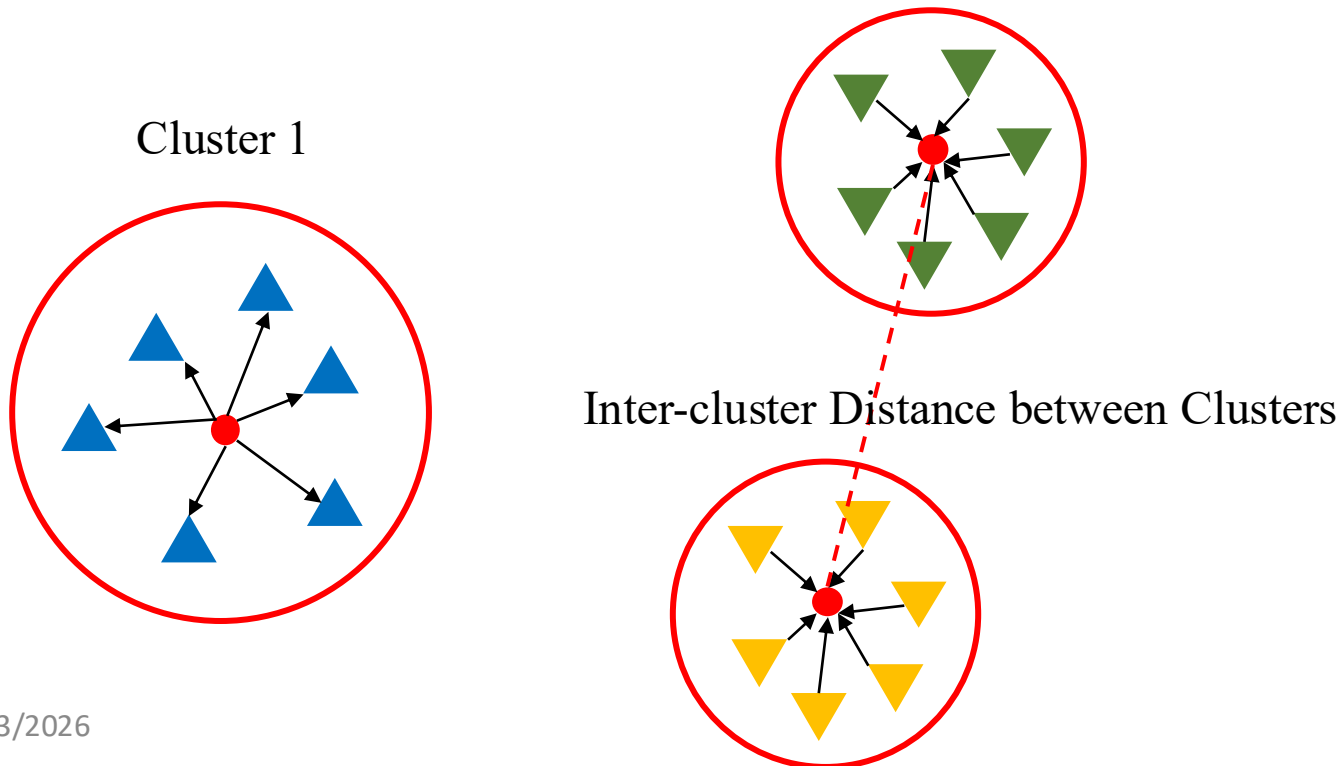


Intra-cluster Distance for Cluster i

Clustering Evaluation

- **Dunn Index**

- The ratio between the smallest inter-cluster distance and the largest intra-cluster distance. The larger the better.



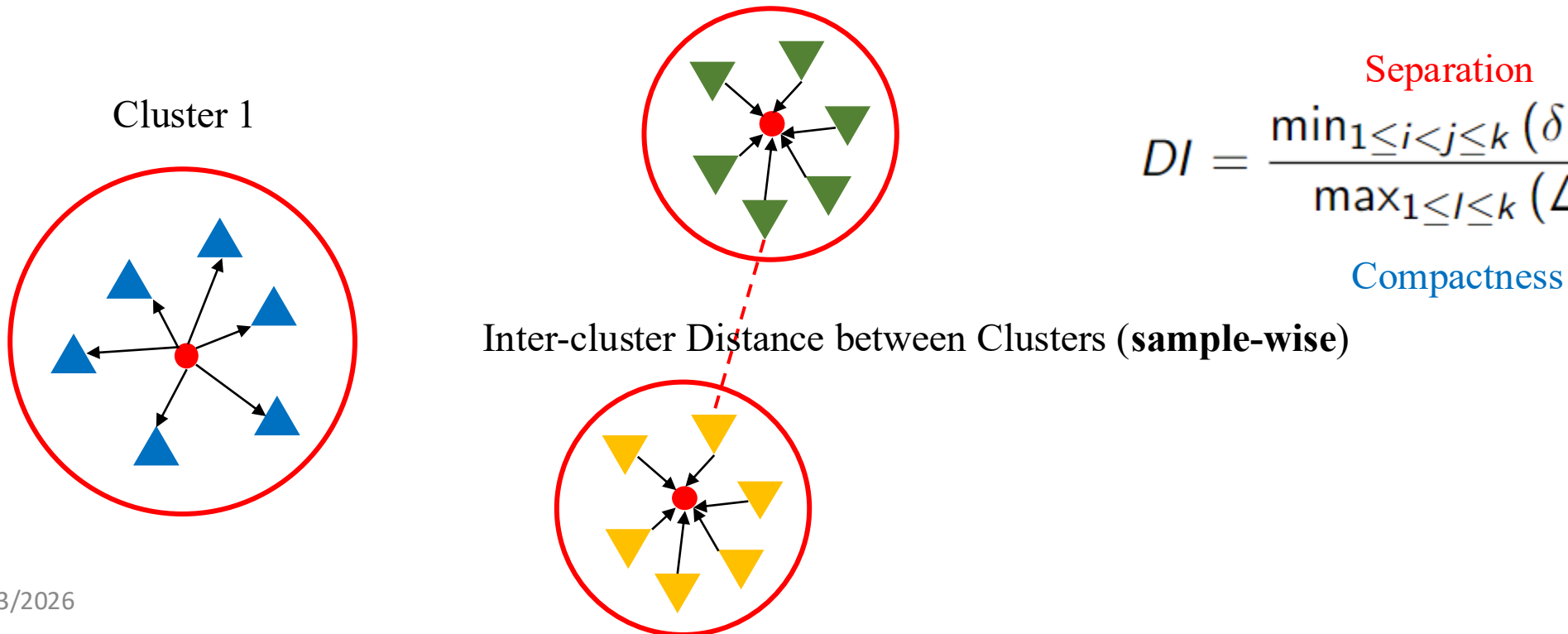
$$DI = \frac{\text{Separation}}{\text{Compactness}}$$

$$DI = \frac{\min_{1 \leq i < j \leq k} (\delta(S_i, S_j))}{\max_{1 \leq l \leq k} (\Delta(S_l))}$$

Clustering Evaluation

- **Dunn Index**

- The definitions of inter-cluster and intra-cluster distance can vary, but the principles are the same.

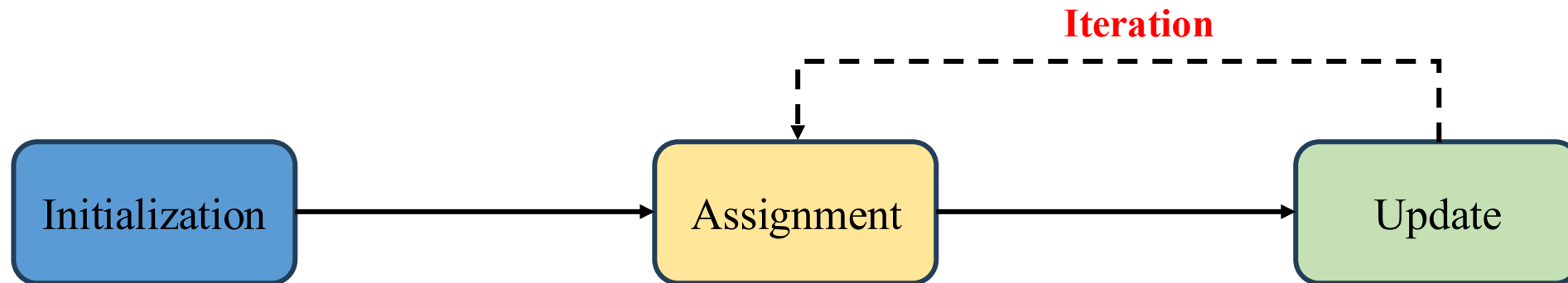


Unsupervised Learning Outline

- **Clustering**
 - Evaluation
 - **K-Means Clustering**
 - Hierarchical Clustering
 - Data Normalization
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA

K-Means Clustering

- K-Means classifies n samples into $k < n$ clusters such that each sample is assigned to the cluster with the nearest mean.
- K-Means is an iterative algorithm.



Select k initial centroids randomly from the dataset, where k is the desired number of clusters.

Assign each data point to the nearest centroid based on distance measure

Recalculate the centroids as the mean of all points in that cluster

K-Means Clustering

- The goal of K-Means is to classify n samples into $k < n$ clusters such that each sample is assigned to the cluster with the nearest mean.
- K-Means is an iterative algorithm.



Initialization

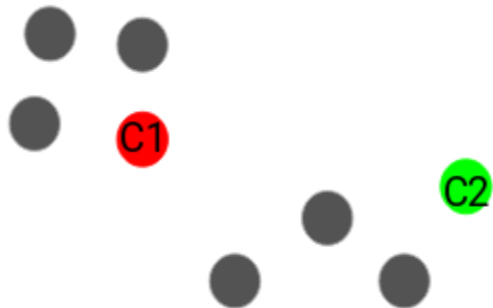
Choose the number of clusters k

We choose $k = 2$ here.

K-Means Clustering

- The goal of K-Means is to classify n samples into $k < n$ clusters such that each sample is assigned to the cluster with the nearest mean.
- K-Means is an iterative algorithm.

Initialization

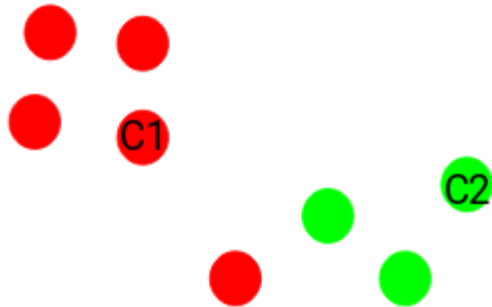


Select $k = 2$ random points from the data as centroids

K-Means Clustering

- The goal of K-Means is to classify n samples into $k < n$ clusters such that each sample is assigned to the cluster with the nearest mean.
- K-Means is an iterative algorithm.

Assignment



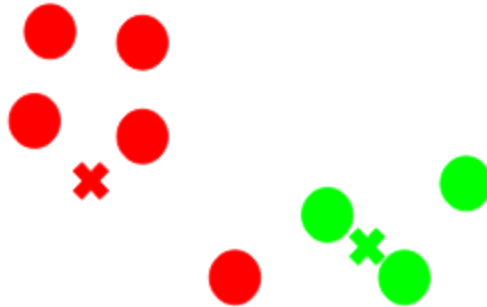
Assign each point to the **closest** cluster centroid

K-Means Clustering

- The goal of K-Means is to classify n samples into $k < n$ clusters such that each sample is assigned to the cluster with the nearest mean.
- K-Means is an iterative algorithm.

Update

Compute the centroids of newly formed clusters.



K-Means Clustering

- The goal of K-Means is to classify n samples into $k < n$ clusters such that each sample is assigned to the cluster with the nearest mean.
- K-Means is an iterative algorithm.

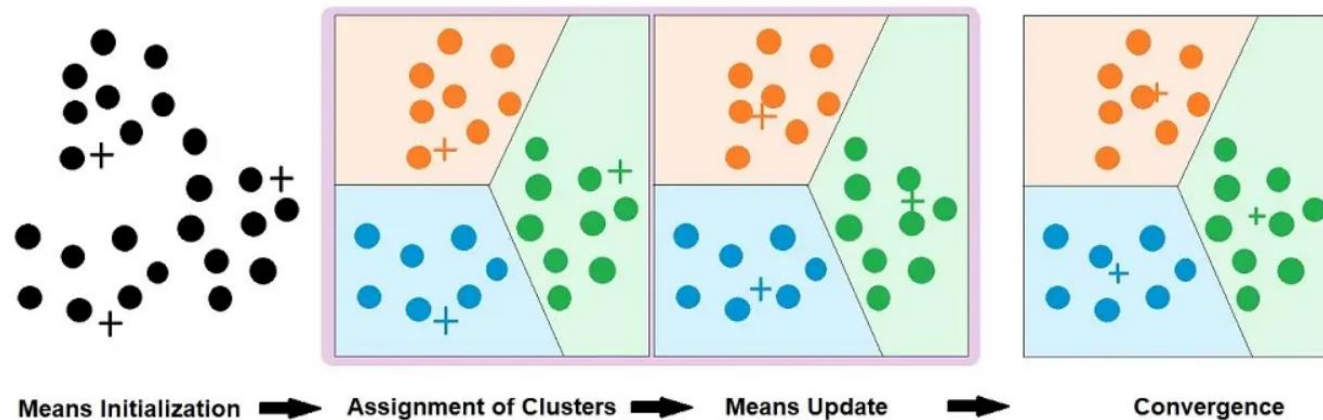


Repeat

Repeat Assignment step, then we have a better clustering result.

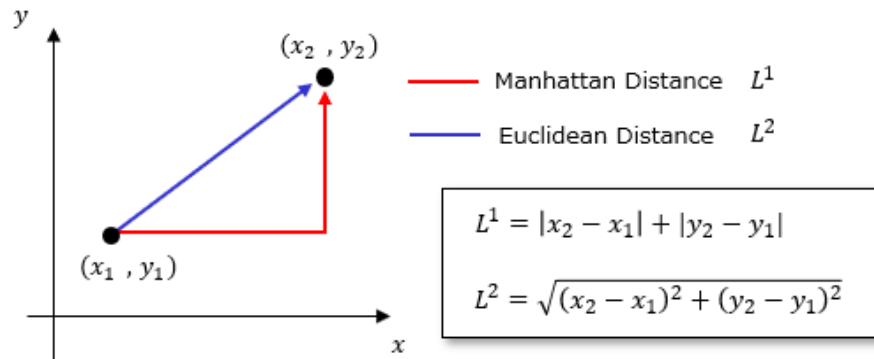
K-Means Clustering

- Stopping Criteria
 - **Centroid Convergence:** Centroids of newly formed clusters do not change significantly between iterations
 - **Change in Labels:** There are no changes in the assignment of data points to clusters between iterations.
 - **Maximum Iterations:** The algorithm stops after a predefined number of iterations.



K-Means Clustering

- Distance Measures (Learned in k-NN algorithm)



How to measure the similarity between samples?

Euclidean Distance: Most common distance metric (L2 Norm of Error)

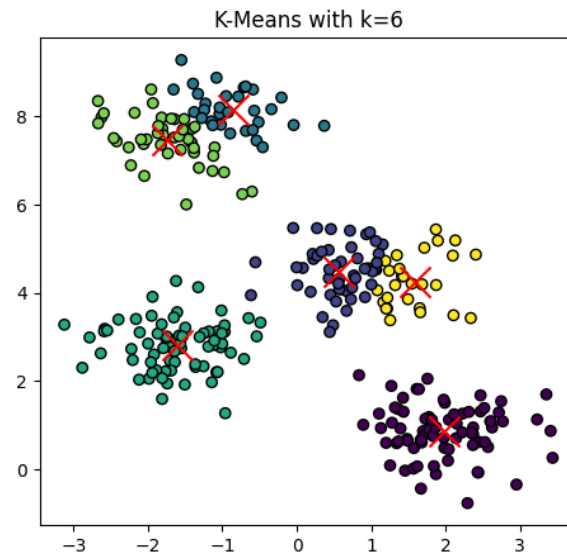
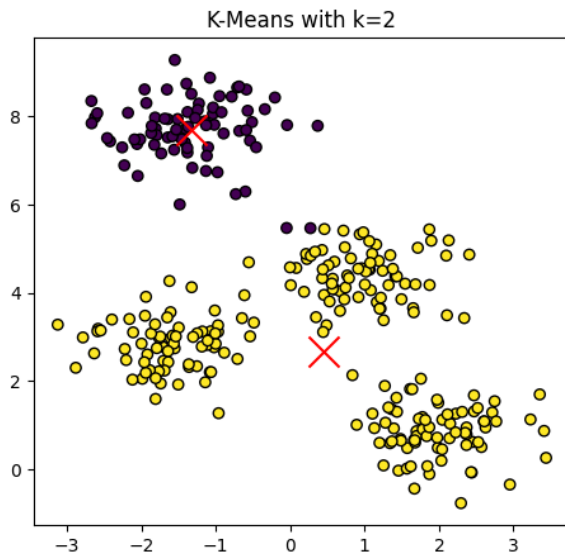
$$d(\mathbf{A}, \mathbf{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2}$$

Manhattan Distance: Sum of absolute differences (L1 Norm of Error)

$$D_1(\mathbf{A}, \mathbf{B}) = \sum_{i=1}^n |A_i - B_i|$$

K-Means Clustering

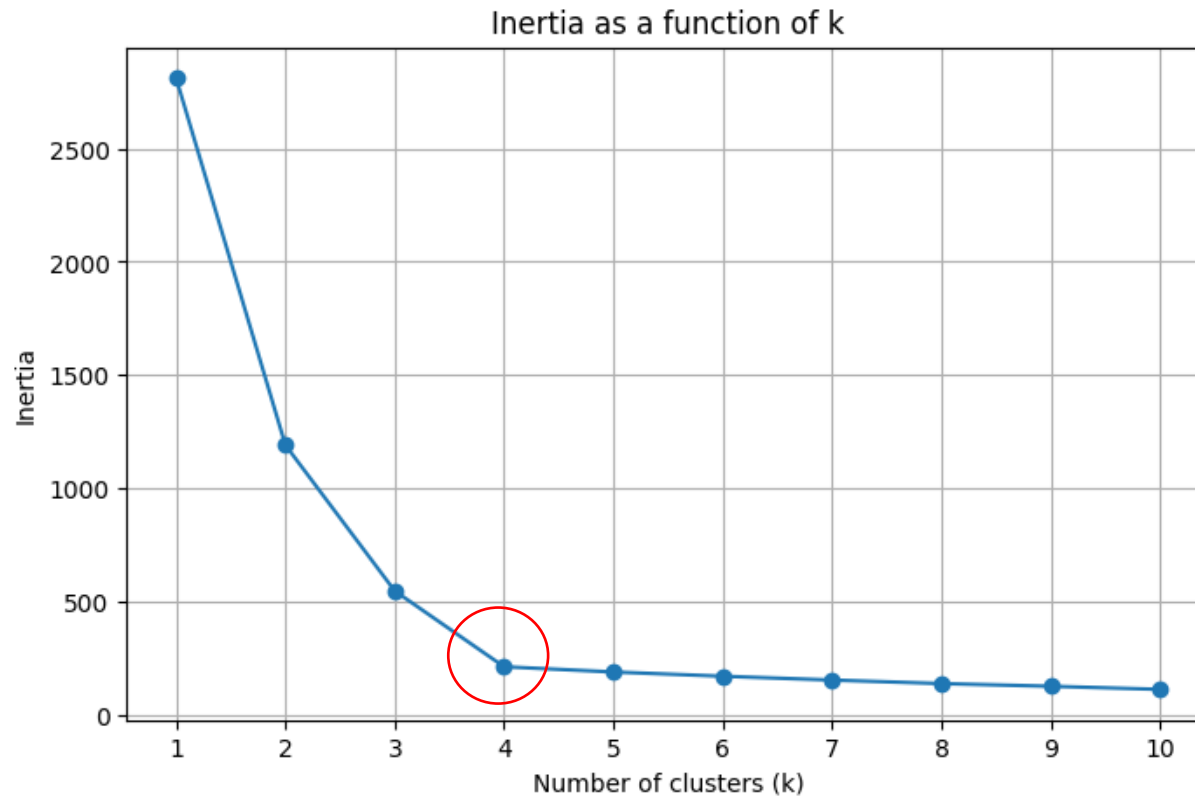
- Choosing the right k
 - A small k leads to a larger size of clusters, which can cause the algorithm to underfit the data, leading to less meaningful distinctions.
 - A large k results in smaller clusters, which can cause the algorithm to overfit the data. Clusters may capture noise and outliers as separate groups.



Examples for data with $k = 4$ clusters

K-Means Clustering

- Choosing the right k
 - Elbow curve: a function of evaluation metric as a function of k .

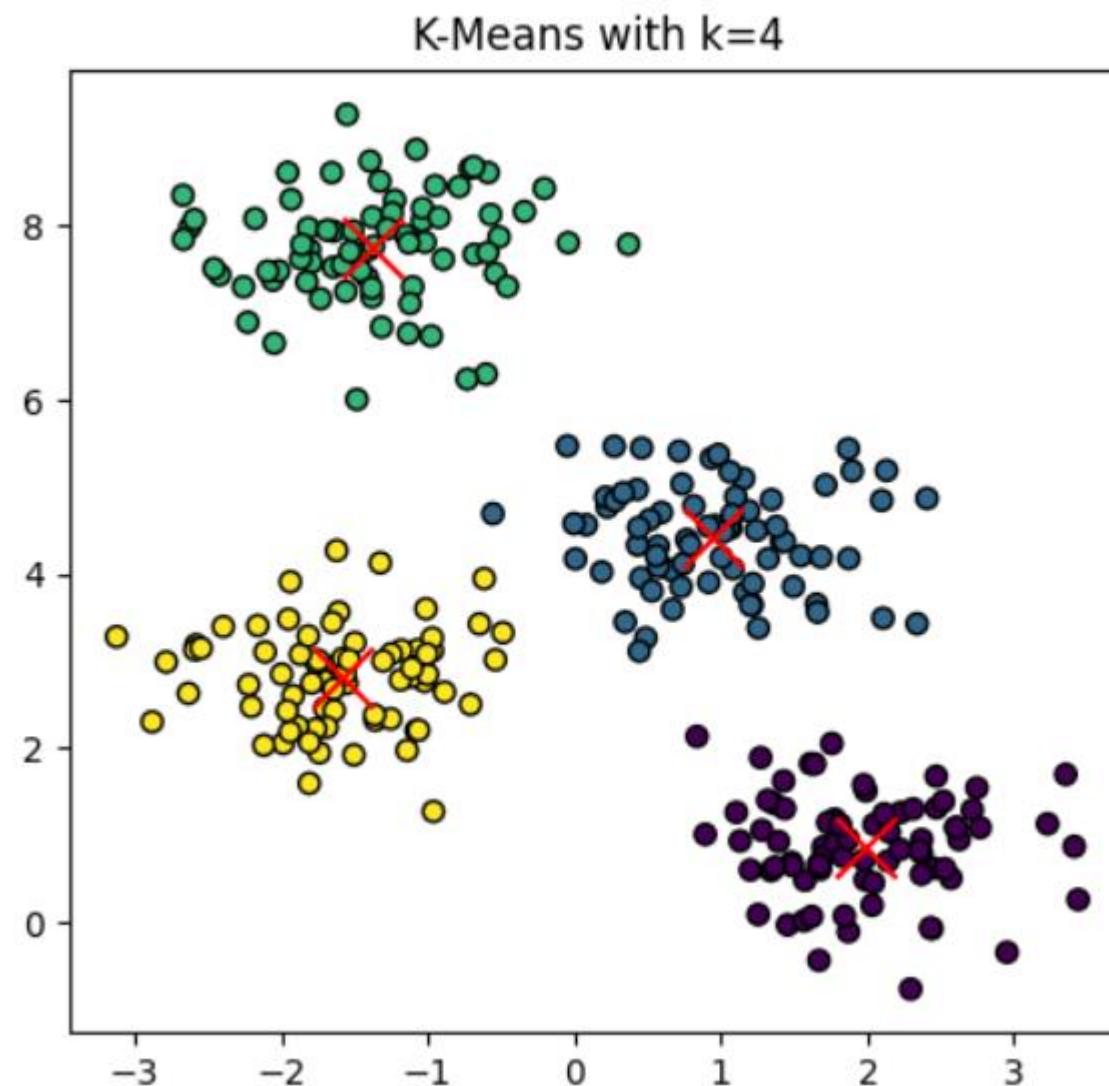


The point where the evaluation metric stops decreasing significantly.

K-Means Clustering

- Choosing the right k

Results for choosing $k = 4$



Unsupervised Learning Outline

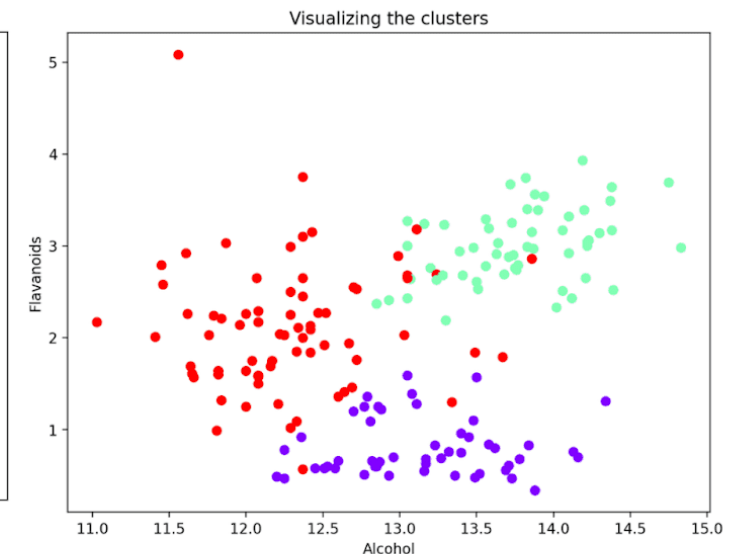
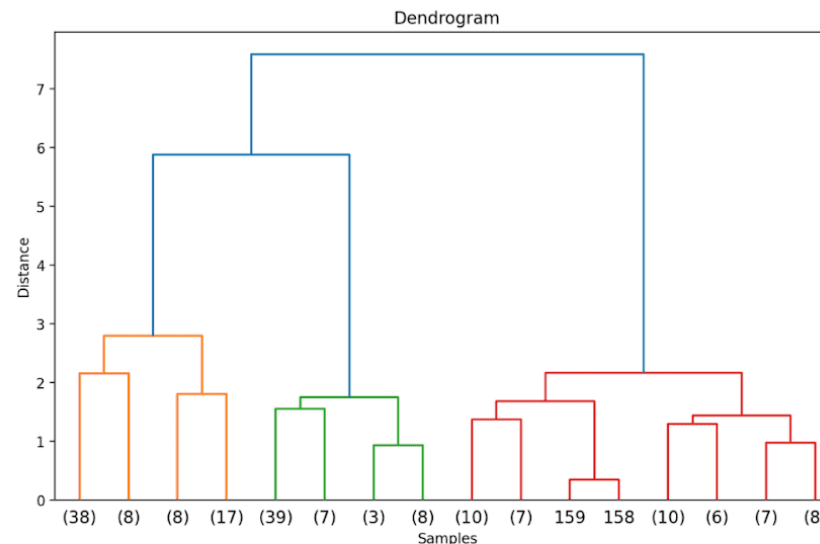
- **Clustering**
 - Evaluation
 - K-Means Clustering
 - **Hierarchical Clustering**
 - Data Normalization
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA

Hierarchical Clustering

- Hierarchical clustering builds a hierarchy of clusters using either an agglomerative (bottom-up) or divisive (top-down) approach.
- Produces a tree structure called dendrogram.

Advantages over K-Means

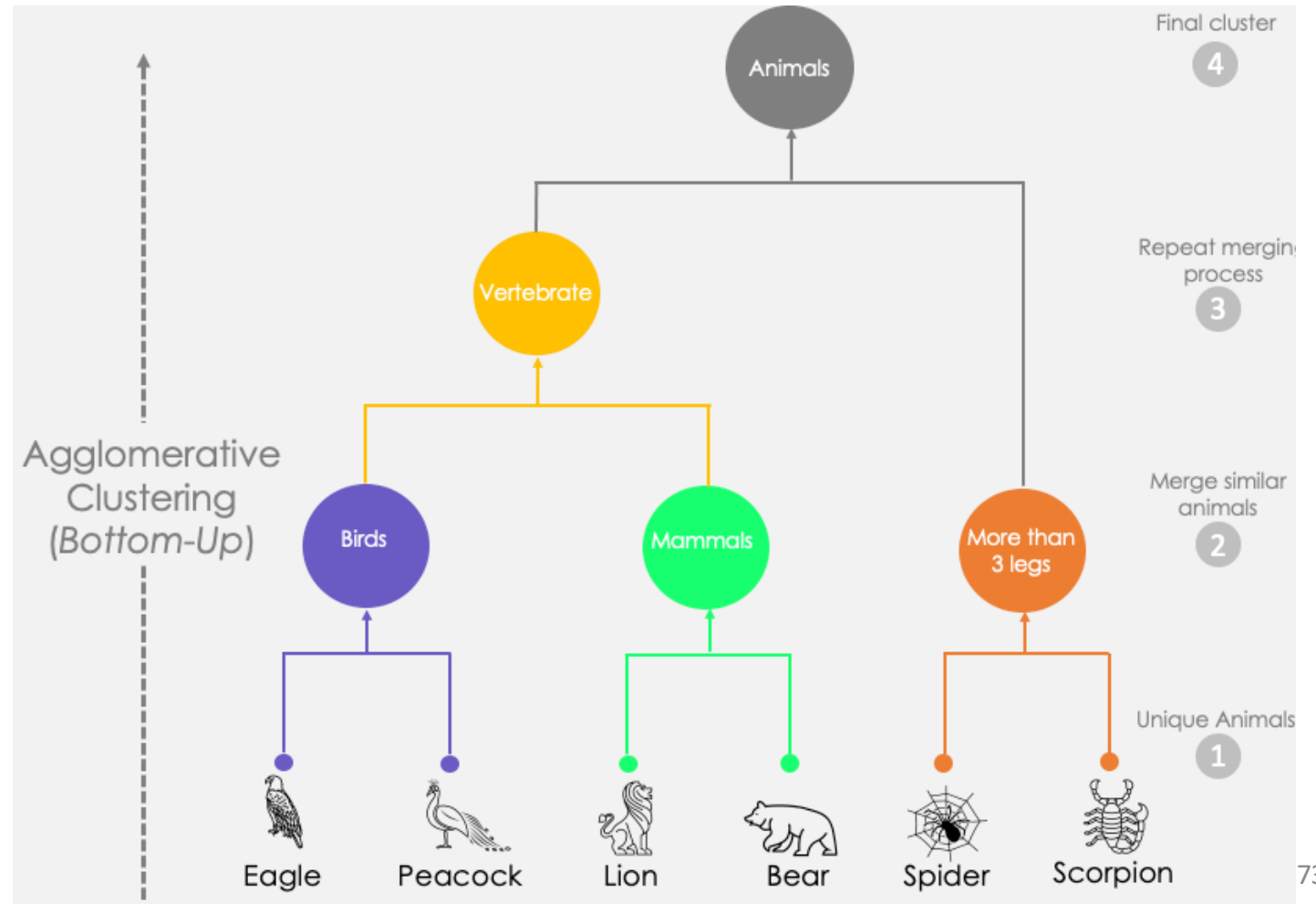
- No need to specify the number of clusters
- Visualization
- Capturing cluster hierarchy



Hierarchical Clustering

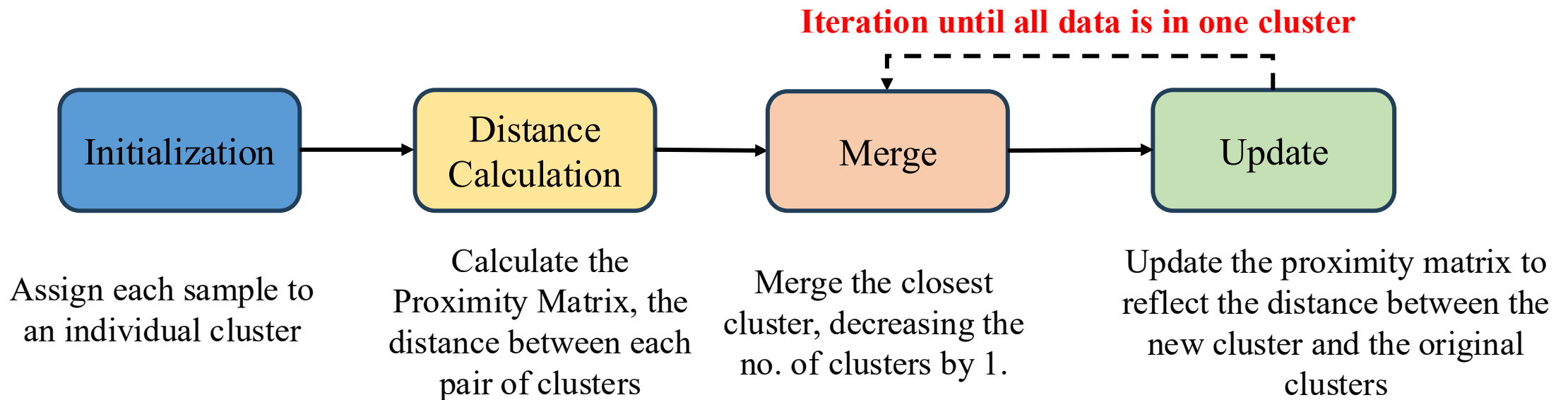
Advantages over K-Means

- No need to specify the number of clusters
- Visualization
- Capturing cluster hierarchy



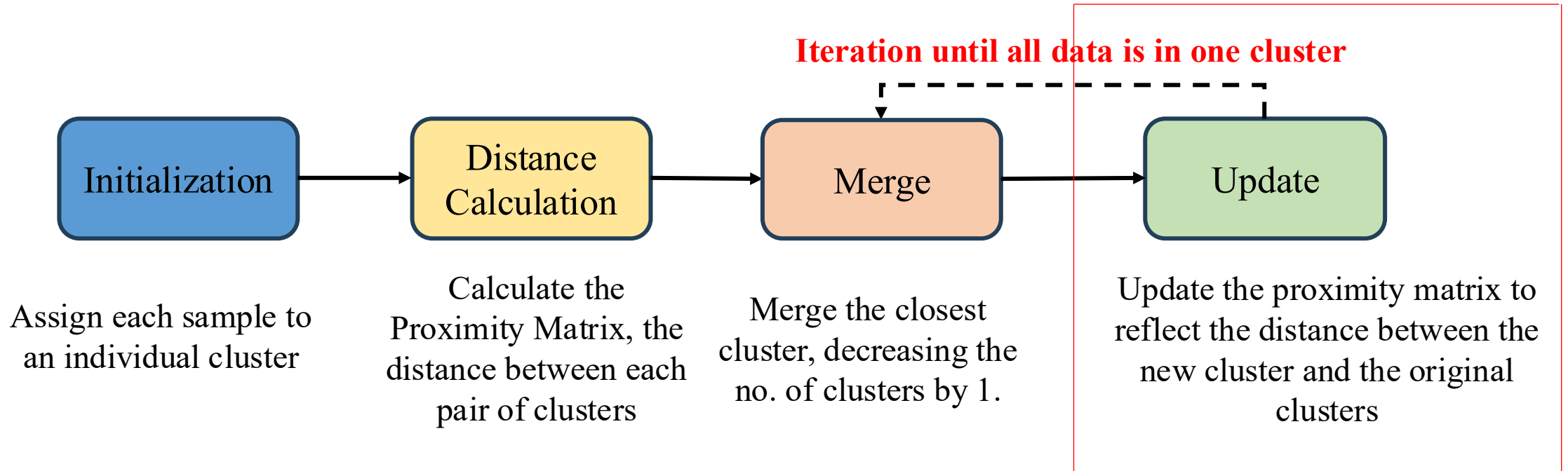
Agglomerative Hierarchical Clustering

- Bottom-up Approach



Agglomerative Hierarchical Clustering

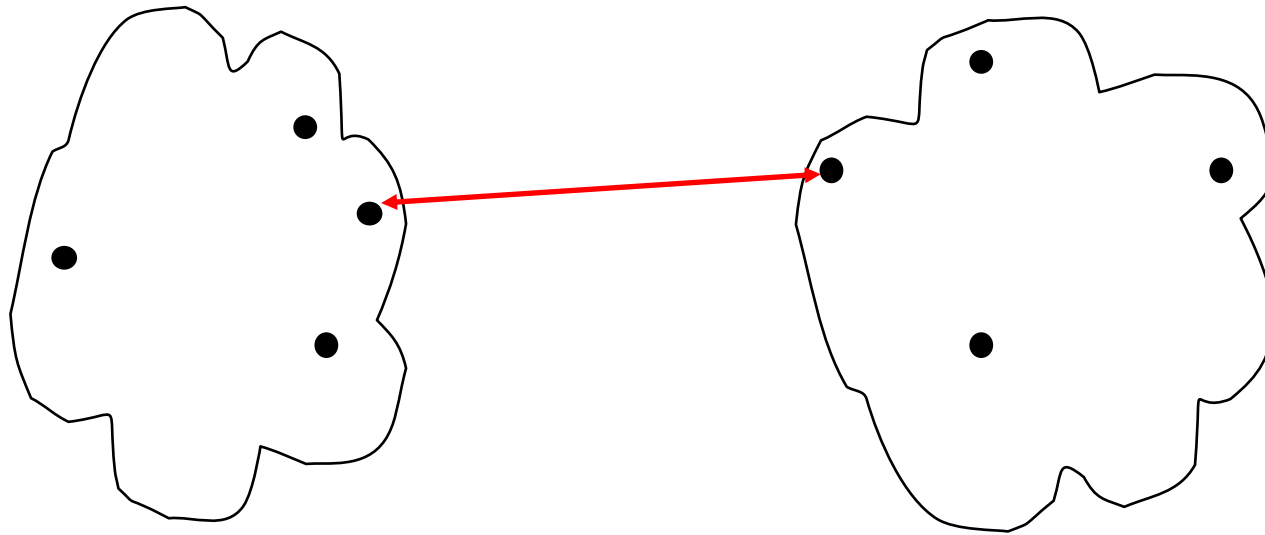
- Bottom-up Approach



How to recompute the cluster distances?

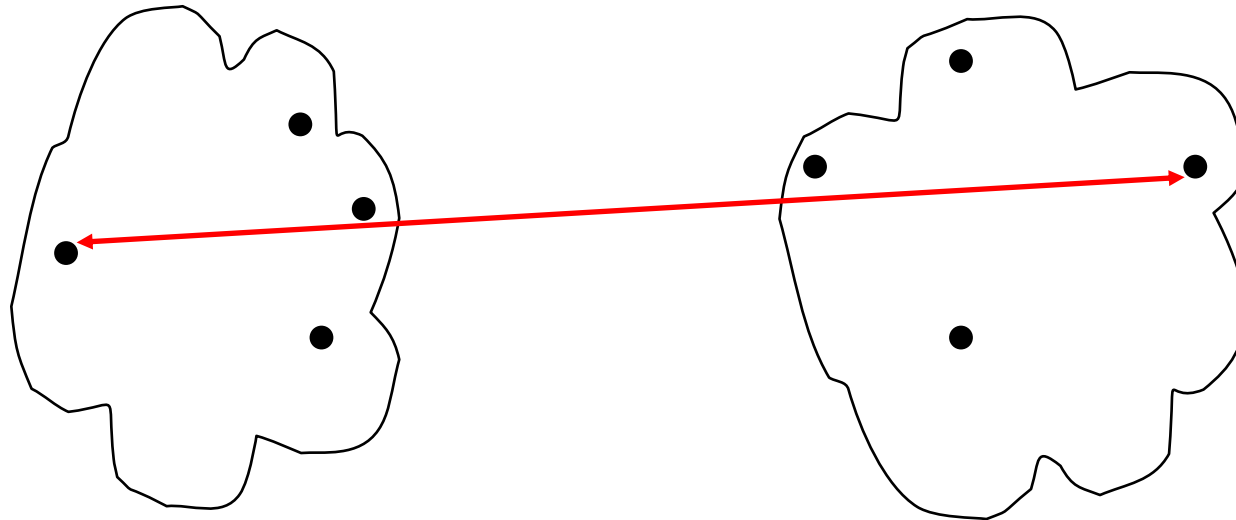
Agglomerative Hierarchical Clustering

- Update the Proximity Matrix
 - **Single linkage (minimum linkage):** Minimum distance between any single member of one cluster and any single member of the other cluster.



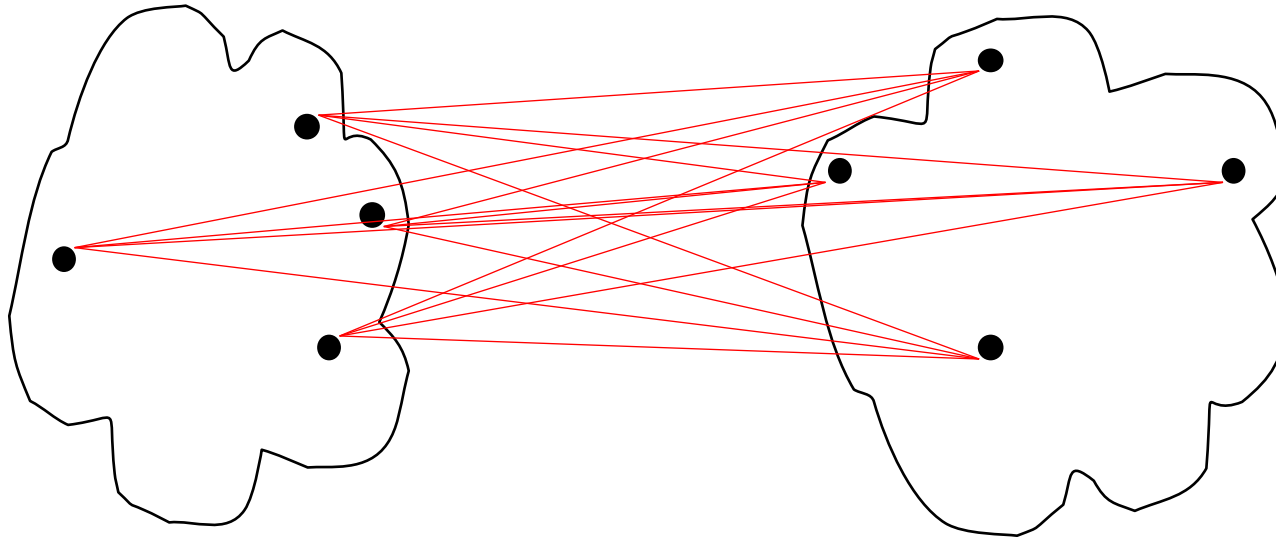
Agglomerative Hierarchical Clustering

- Update the Proximity Matrix
 - **Complete linkage (maximum linkage):** Maximum distance between any member of one cluster and any member of the other cluster.



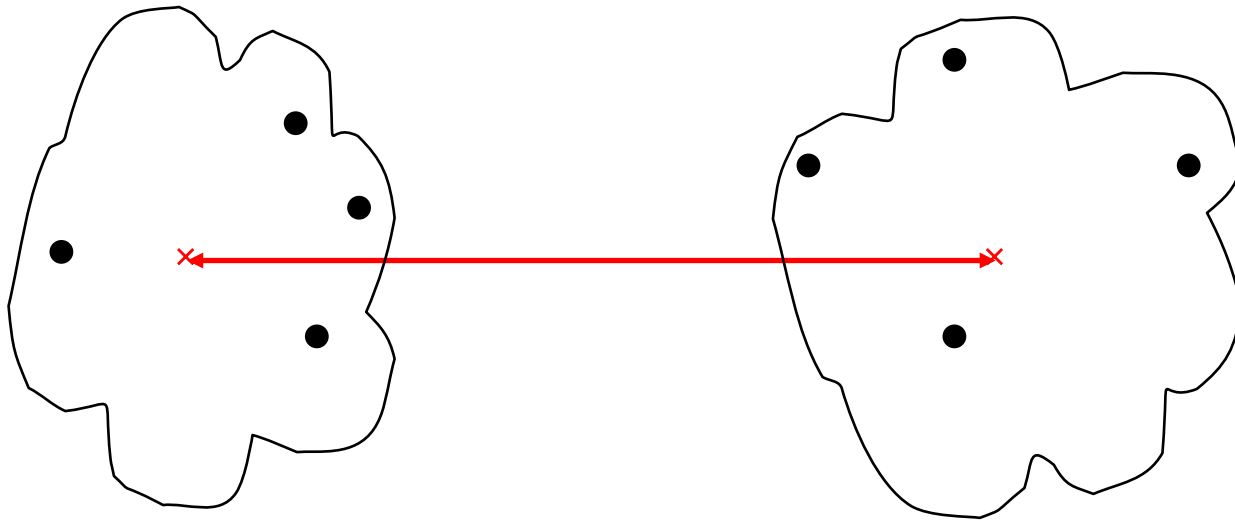
Agglomerative Hierarchical Clustering

- Update the Proximity Matrix
 - **Average linkage:** Average distance between all pairs of individuals in the two clusters.



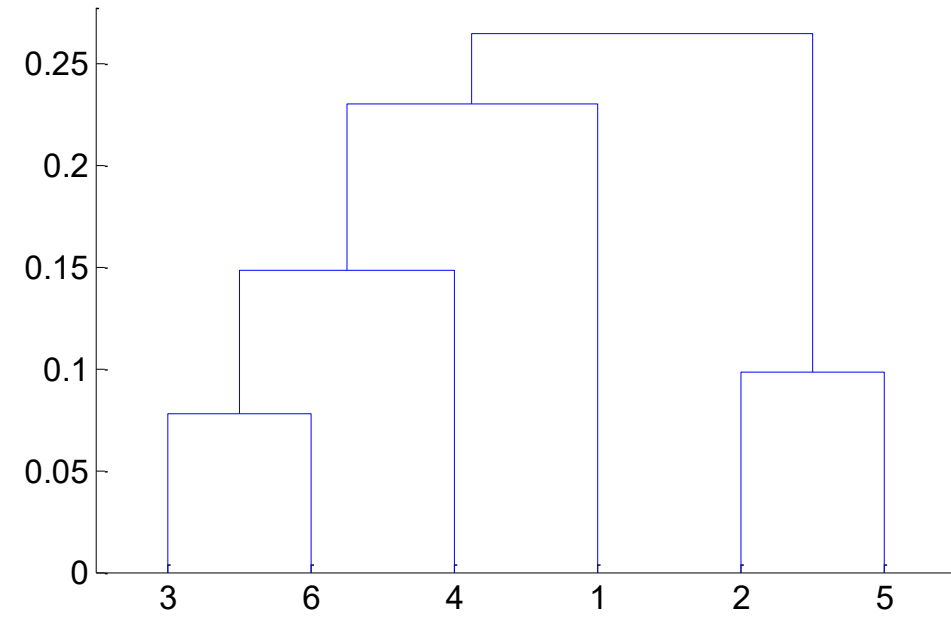
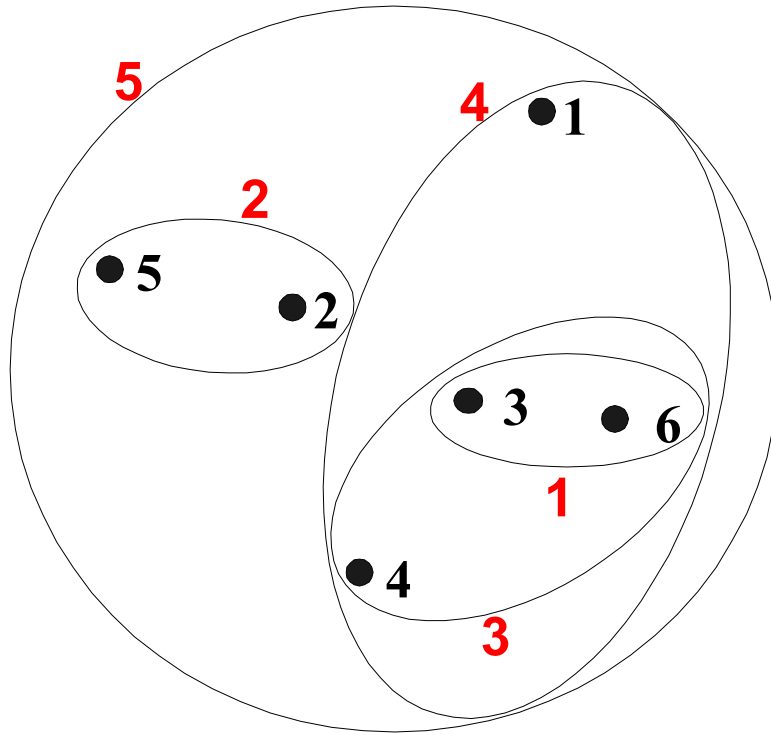
Agglomerative Hierarchical Clustering

- Update the Proximity Matrix
 - **Centroid linkage:** Distance between the centroids of the clusters.



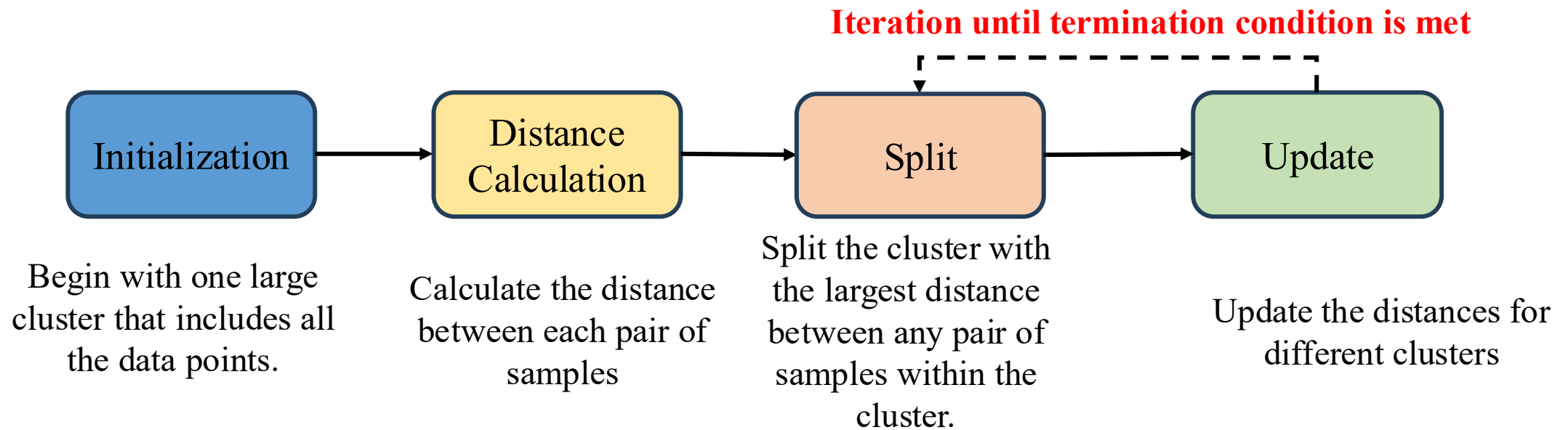
Agglomerative Hierarchical Clustering

- Bottom-up Approach



Divisive Hierarchical Clustering

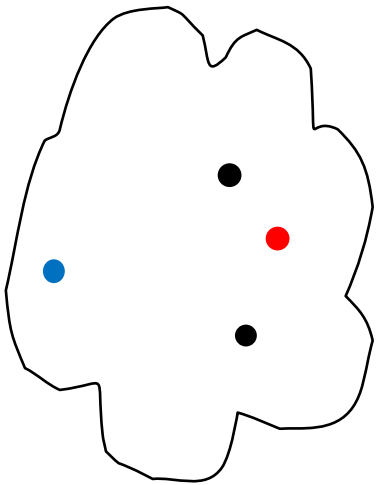
- Top-down Approach



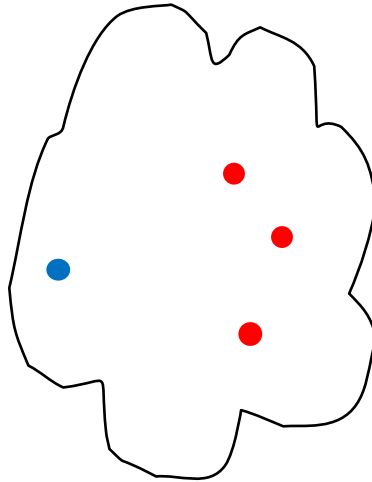
Divisive Hierarchical Clustering

- Top-down Approach

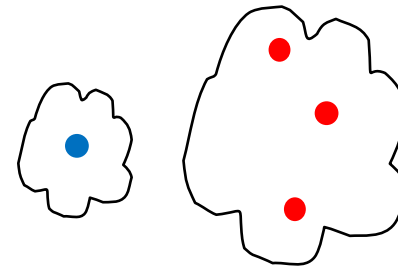
Cluster Splitting



Identify the farthest
pair of samples



Assign to different
centroids



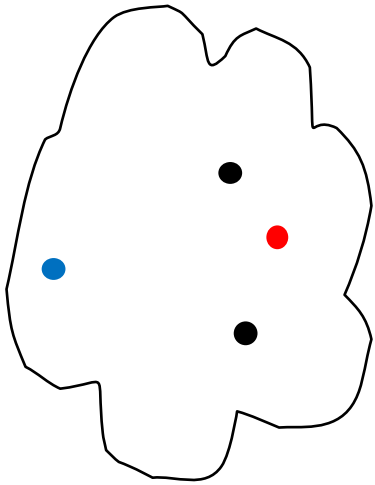
Split into two
clusters

Divisive Hierarchical Clustering

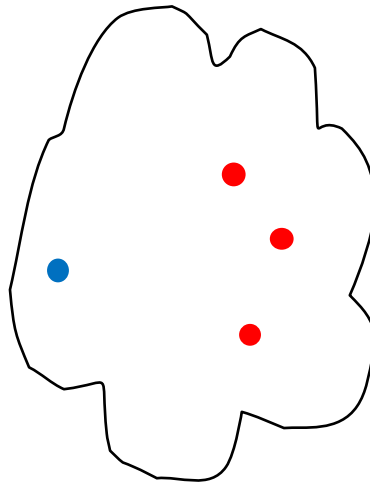
- Top-down Approach

Termination Conditions

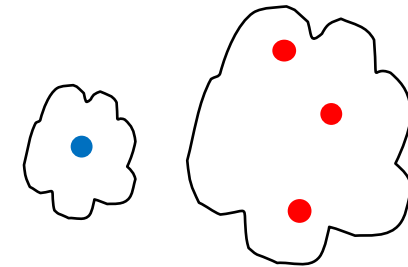
- Reaching expected number of clusters
- The longest linkage for all clusters is below a threshold



Identify the farthest
pair of samples



Assign to different
centroids



Split into two
clusters

Unsupervised Learning Outline

- **Clustering**
 - Evaluation
 - K-Means Clustering
 - Hierarchical Clustering
 - **Data Normalization**
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA

Data Normalization

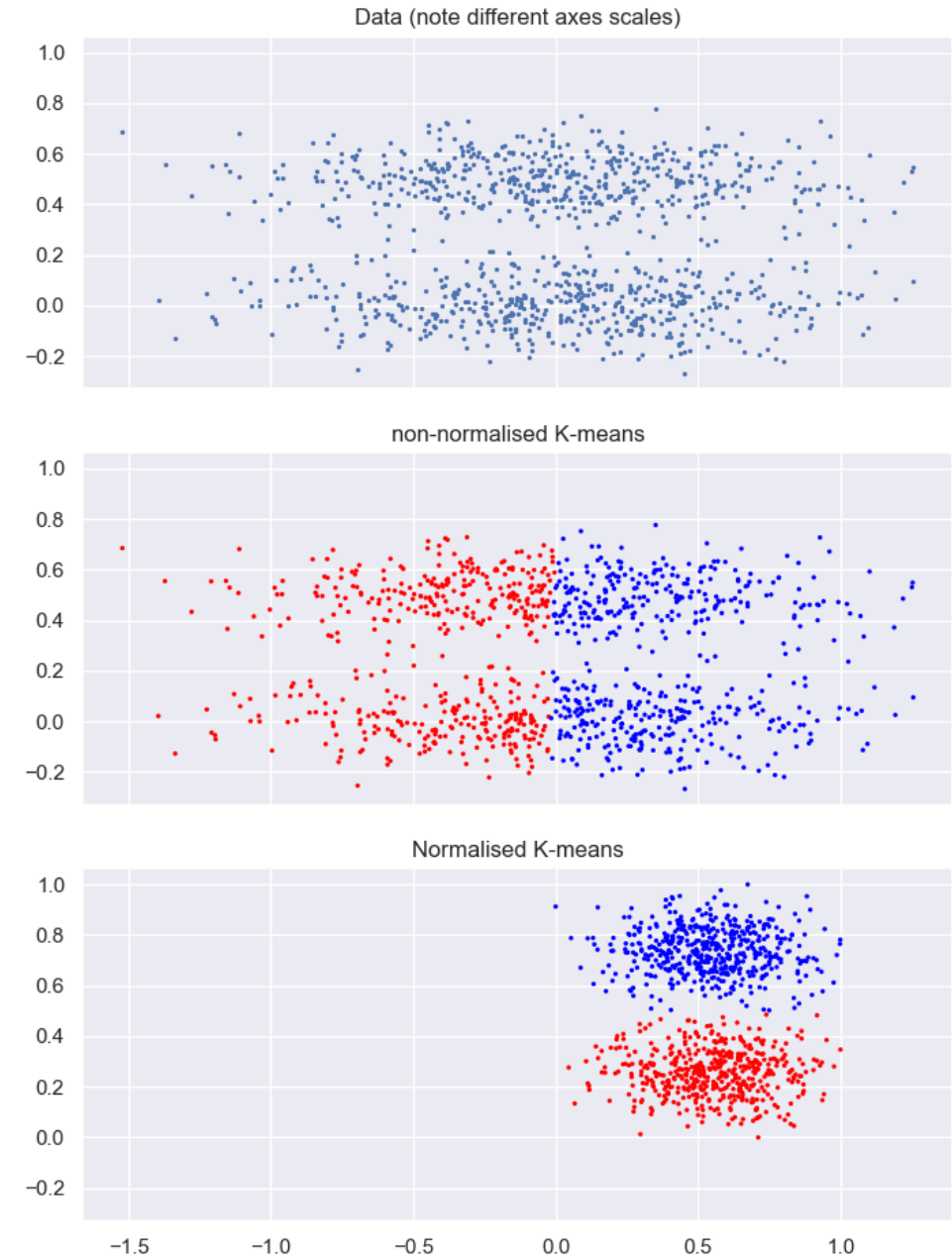
- Data normalization is crucial for clustering because distance metrics (e.g., Euclidean distance) determine the similarity between samples.



- The data scales of different attributes can vary significantly.
 - Weight (40~100 kg), height (150-190 cm)
 - Temperature (0-30 degrees), speed (0-100)
 - Stock Price (0-1000), P/E Ratio (0-100)
- Non-compatible data scale could lead to wrong distance computation.

Data Normalization

- Data normalization makes features having equal importance when performing clustering.
 - Min-Max scaling
 - Standardization

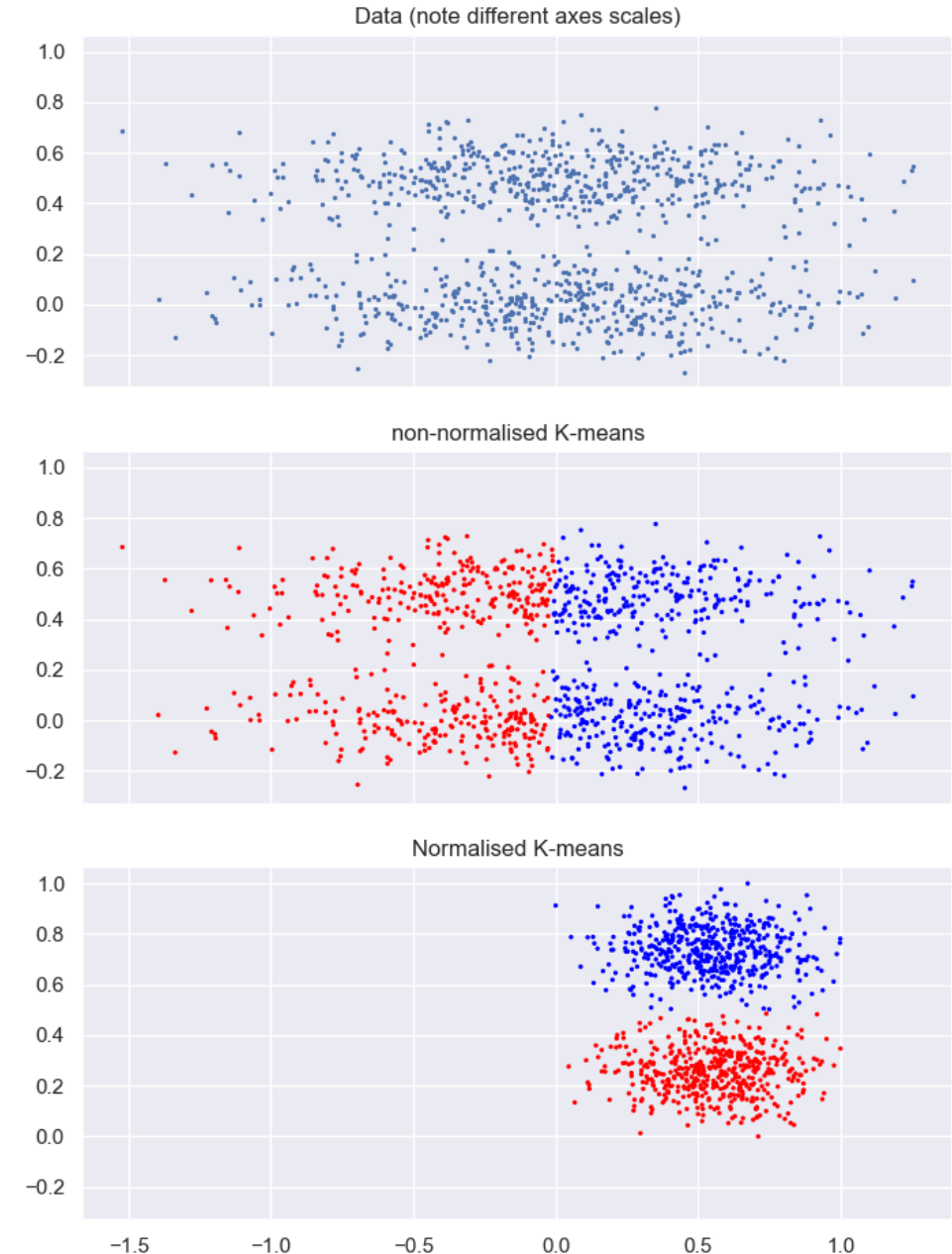


Data Normalization

- Data normalization makes features having equal importance when performing clustering.
 - Min-Max scaling

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Performed for each attribute separately.



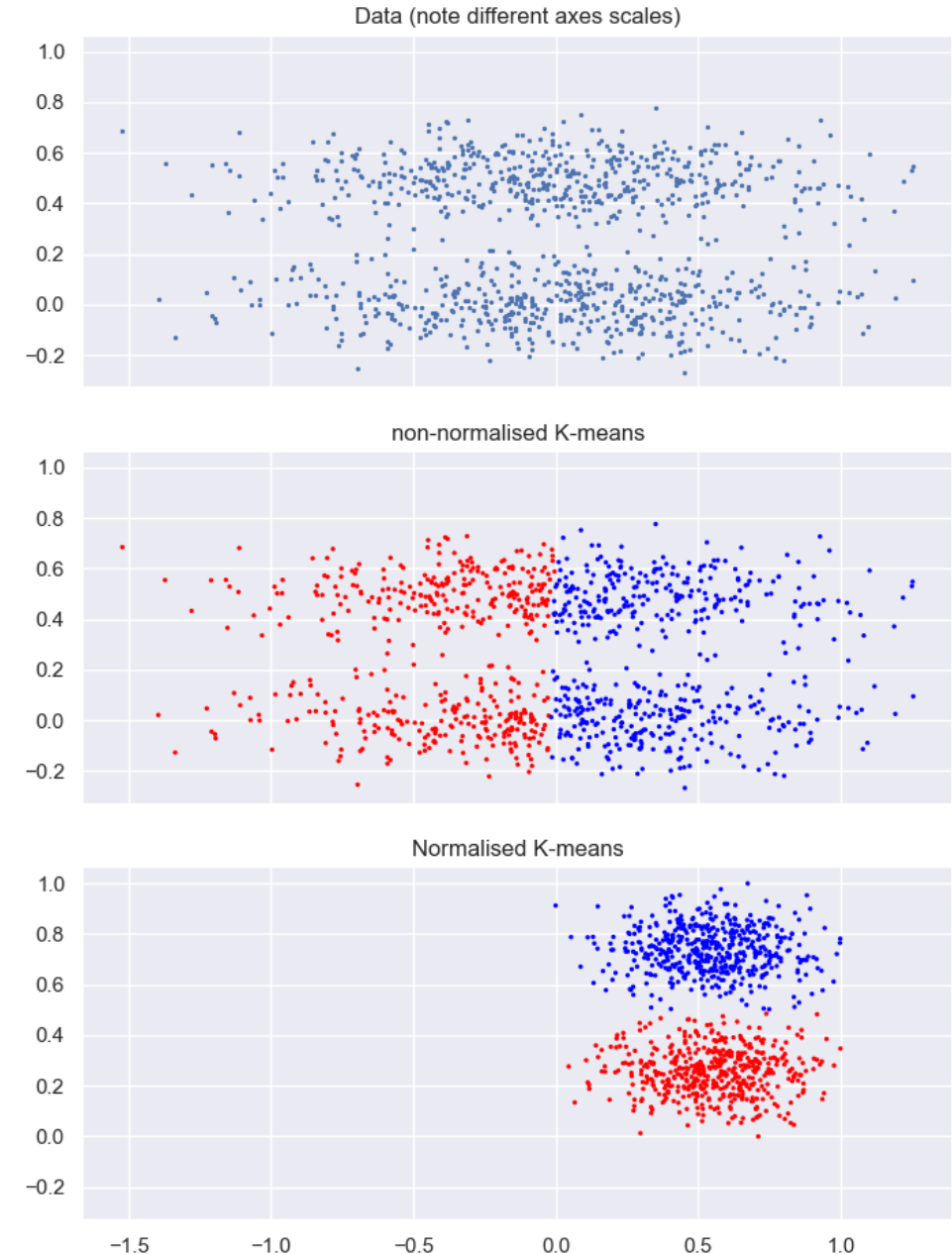
Data Normalization

- Data normalization makes features having equal importance when performing clustering.
 - Standardization

$$x' = \frac{x - \mu}{\sigma}$$

μ is the mean of the attribute

σ is the standard deviation of the attribute

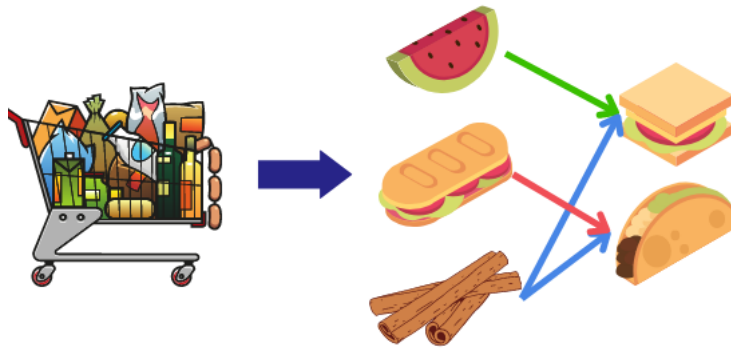


What is Association Rule Learning

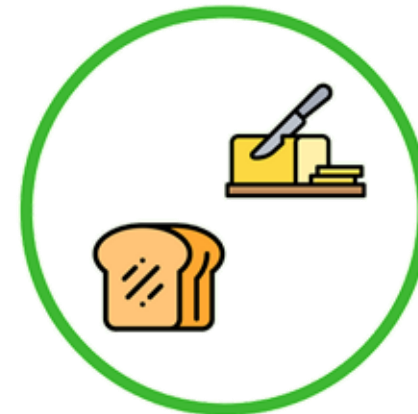
Association Rule Learning

Discovering interesting relations between variables in large databases. (a few things that always happen at the same time)

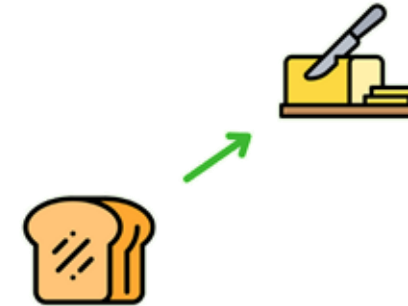
Important: Association rule learning does NOT find causality.



*"93% of people who purchased item A
also purchased item B"*



CLUSTERING



ASSOCIATION RULE
MINING

What is Association Rule Learning

Association Rule Learning

Important to optimize the product placement in supermarkets, and online shopping recommendation.



Customers who bought this item also bought

Page 1 of 15

Strong & Beautiful
> Pamela Reif
★★★★★ 346
Hardcover
€18.00

Feierabend: 101 neue Wohlfühl-Rezepte für einen entspannten & leckeren Feierabend
> Florian Kaminski Foodoh...
★★★★★ 40
Paperback
€24.00

You deserve this: Simple & Natural Recipes For A Healthy Lifestyle: Bowl Cookbook
> Pamela Reif
★★★★★ 310
Hardcover
€20.00

Bowls Kochbuch für Anfänger: 101 Rezepte aus der Schale - für ein gesundes Leben....
> Bon Vivant
★★★★★ 211
Paperback
€8.95

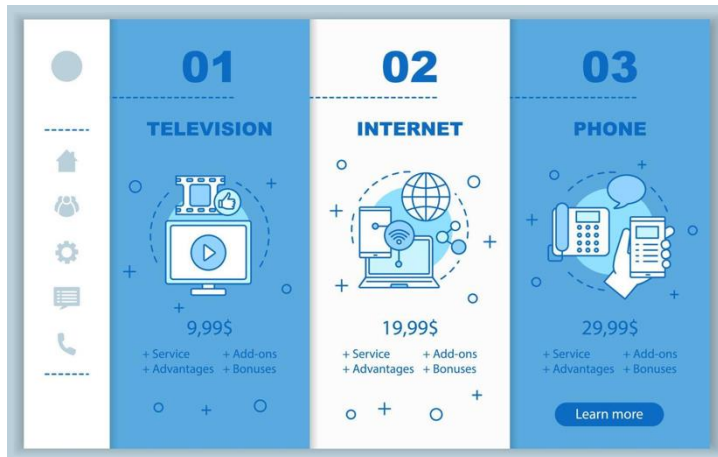
comboo Coconut Bowl - Original Coconut Bowl - Buddha Bowl - Polished
★★★★★ 237
€19.95

pandoo Coconut Bowls Set of 2 - 100 % Natural Product - Plastic-Free Alternative - Handmade and Polished with Coconut Oil
★★★★★ 552
€19.99

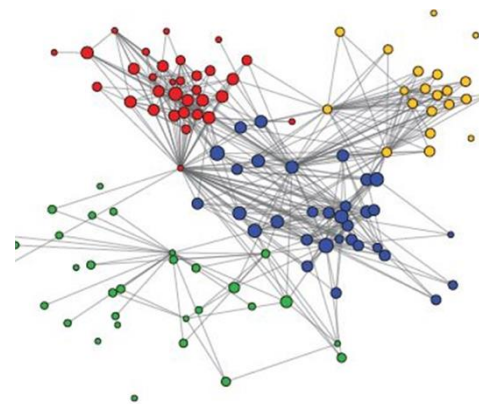
What is Association Rule Learning

Association Rule Learning

Applications other than market basket analysis



Telecommunications: Understanding patterns in service usage to create bundles of services.



Social Network Analysis: Discover associations between individuals, communities, or groups

Tools and techniques	Risk Identification	Risk Assessment Process			Risk Evaluation
		Consequence	Probability	Level of risk	
1 Brainstorming	SA	NA	NA	NA	NA
2 Structured or semi-structured interviews	SA	NA	NA	NA	NA
3 Delphi	SA	NA	NA	NA	NA
4 Check-lists	SA	NA	NA	NA	NA
5 Primary hazard analysis	SA	NA	NA	NA	NA
6 Hazard and operability studies (HAZOP)	SA	A	A	A	A
7 Hazard Analysis and Critical Control Points (HACCP)	SA	SA	NA	NA	SA
8 Environmental risk assessment	SA	SA	SA	SA	SA
9 Structure - What If? (SWIFT)	SA	SA	SA	SA	SA
10 Scenario analysis	SA	SA	A	A	A
11 Business impact analysis	A	SA	A	A	A
12 Root cause analysis	NA	SA	SA	SA	SA
13 Failure mode effect analysis	SA	SA	SA	SA	SA
14 Fault tree analysis	A	NA	SA	A	A
15 Event tree analysis	A	SA	A	A	NA
16 Cause and consequence analysis	A	SA	SA	A	A
17 Cause-and-effect analysis	SA	SA	NA	NA	NA
18 Layer protection analysis (LOPA)	A	SA	A	A	A
19 Decision tree	NA	SA	SA	A	A
20 Human reliability analysis	SA	SA	SA	SA	A
21 Bow tie analysis	NA	A	SA	SA	A
22 Reliability Centred Maintenance (RCM)	SA	SA	SA	SA	SA
23 Shrink circuit analysis	A	NA	NA	NA	NA
24 Markov analysis	A	SA	NA	NA	NA
25 Monte Carlo simulation	NA	NA	NA	NA	SA
26 Bayesian Statistics and Bayes Nets	NA	SA	NA	NA	SA
27 F/N curves	A	SA	SA	A	SA
28 Risk indices	A	SA	SA	A	SA
29 Consequence/probability matrix	SA	SA	SA	SA	A
30 Cost/benefit analysis	A	SA	A	A	A
31 Multi-criteria decision analysis (MCDA)	A	SA	A	SA	A

SA = Strongly Applicable
NA = Not Applicable
A = Applicable

Risk Analysis: Identify associations between risk factors, events, or conditions

Unsupervised Learning Outline

- Clustering
 - Evaluation
 - K-Means Clustering
 - Hierarchical Clustering
 - Data Normalization
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA

Concepts for Association Rule Learning

Key concepts that are important for the association rule learning algorithm

Support

A measure of how frequently the items appear in the dataset

$$\text{Support}(X) = \frac{\text{Number of records containing } X}{\text{Number of records}}$$

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

Only those with support **above a threshold (minimum support)** are considered.
This threshold is crucial to eliminate item sets that are too rare to be useful.

$$\text{Support}(\text{Bread}) = \frac{4}{5} = 0.8$$

Concepts for Association Rule Learning

Key concepts that are important for the association rule learning algorithm

Confidence

A measure of the reliability of the inference made by an association rule.

$X \rightarrow Y$: given that these records contain X , the conditional probability of finding the Y as well. (If X then Y . If X is purchased, then Y is purchased.)

$$\text{Confidence}(X \rightarrow Y) = \frac{\text{Support}(X \& Y)}{\text{Support}(X)}$$

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

However, a high confidence value could be misleading when Y is inherently popular, which does not depend on X . Can be used together with **Lift**.

$$\text{Confidence}(\text{Bread} \rightarrow \text{Beer}) = \frac{2}{4} = 0.5$$

Concepts for Association Rule Learning

Key concepts that are important for the association rule learning algorithm

Lift

A measure of the strength of a rule over the random co-occurrence of the itemset.

What is the “lift” (positive, negative, zero) of the appearance of Y , when there is X

$$\text{Lift}(X \rightarrow Y) = \frac{\text{Confidence}(X \rightarrow Y)}{\text{Support}(Y)} = \frac{\text{Support}(X \& Y)}{\text{Support}(X)\text{Support}(Y)}$$

- Greater than 1 implies that X and Y appear more often together than expected, indicating a positive relationship.
- Equal to 1 implies that X and Y are independent of each other.
- Less than 1 implies that X and Y appear less often together than expected, which can be seen as a negative relationship.

1	Bread , Milk
2	Bread , Diaper, Beer , Eggs
3	Milk, Diaper, Beer , Cola
4	Bread , Milk, Diaper, Beer
5	Bread , Milk, Diaper, Cola

$$\text{Lift}(\text{Bread} \rightarrow \text{Beer}) = \frac{0.5}{3/5} = 0.83$$

Concepts for Association Rule Learning

Key concepts that are important for the association rule learning algorithm

Strong Rules

- Association rules that satisfy both a minimum support threshold and a minimum confidence/lift threshold.
- Rules that meet these thresholds are considered interesting and potentially useful for decision-making.

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

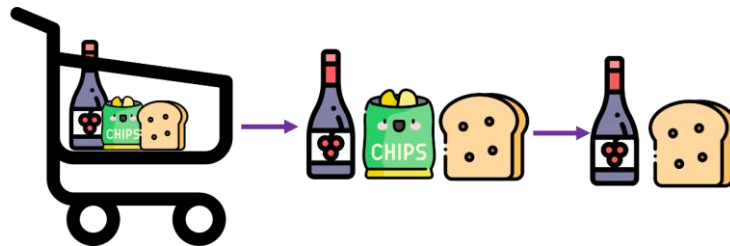
$$\text{Confidence}(Bread \rightarrow Beer) = \frac{2}{4} = 0.5$$

The Apriori Algorithm

Mining frequent item sets and relevant association rules from the dataset.

Basic Ideas

- Frequent Item Sets: Sets of items that appear together with a frequency above a user-specified threshold (minimum support).
- **Apriori Property:**
 - All non-empty subsets of a frequent itemset must also be frequent (Join).
 - Conversely, if an itemset is infrequent, all its supersets will also be infrequent (Pruning).



If the set of beer, chips and bread appears frequently (above a threshold), then, for instance, beer and chips must have appeared frequently.

The Apriori Algorithm

Workflow

Initialization:

- Start by identifying all frequent item sets of size $k = 1$ (individual items)

Set minimum support threshold to **60%**

- Bread**: 4/5 transactions = 80%
- Milk**: 4/5 transactions = 80%
- Diaper**: 4/5 transactions = 80%
- Beer**: 3/5 transactions = 60%
- Eggs: 1/5 transactions = 20%
- Cola: 2/5 transactions = 40%

Prune the item sets and keep **Bread, Milk, Diaper, and Beer**.

Dataset

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

Frequent Item Sets

$k = 1$

1	Bread
2	Milk
3	Diaper
4	Beer

The Apriori Algorithm

Workflow

Iteration:

- Generate candidate item sets of size k by joining frequent item sets of size $k - 1$
- Determine the frequent item sets based on the support threshold.

Set minimum support threshold to 60%

- Bread, Milk: 3/5 transactions = 60%
- Bread, Diaper: 3/5 transactions = 60%
- Bread, Beer: 2/5 transactions = 40%
- Milk, Diaper: 3/5 transactions = 60%
- Milk, Beer: 2/5 transactions = 40%
- Diaper, Beer: 3/5 transactions = 60%

Prune the item sets and keep (Bread, Milk), (Bread, Diaper), (Milk, Diaper) and (Diaper, Beer).

Dataset

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

Frequent Item Sets

$k = 1$

1	Bread
2	Milk
3	Diaper
4	Beer

Candidate Item Sets

$k = 2$

1	Bread, Milk
2	Bread, Diaper
3	Bread, Beer
4	Milk, Diaper
5	Milk, Beer
6	Diaper, Beer

The Apriori Algorithm

Workflow

Iteration:

- Generate candidate item sets of size k by joining frequent item sets of size $k - 1$
- Determine the frequent item sets based on the support threshold.

Termination

- No further successful joins can be made, or no new frequent item sets are found

Set minimum support threshold to **60%**

- Bread, Milk, Diaper: 2/5 transactions = 40%
- Bread, Diaper, Beer: 2/5 transactions = 40%
- Milk, Diaper, Beer: 1/5 transactions = 20%

Prune the item sets and keep **None**.

Dataset

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

Frequent Item Sets

$k = 2$

1	Bread, Milk
2	Bread, Diaper
3	Milk, Diaper
4	Diaper, Beer

Candidate Item Sets

$k = 3$

1	Bread, Milk, Diaper
2	Bread, Diaper, Beer
3	Milk, Diaper, Beer

The Apriori Algorithm

Workflow

Strong Rule Generation:

- Association rules are generated by dividing the frequent item sets into two non-empty disjoint subsets, where one subset implies the other.
- The strength of these rules is then evaluated based on confidence or lift.

For instance,

$$\text{Confidence}(Bread \rightarrow Milk) = \frac{\text{Support}(Bread \& Milk)}{\text{Support}(Milk)} = \frac{0.6}{0.8} = 0.75$$

$$\text{Confidence}(Milk \rightarrow Bread) = \frac{\text{Support}(Bread \& Milk)}{\text{Support}(Bread)} = \frac{0.6}{0.8} = 0.75$$

Dataset

1	Bread, Milk
2	Bread, Diaper, Beer, Eggs
3	Milk, Diaper, Beer, Cola
4	Bread, Milk, Diaper, Beer
5	Bread, Milk, Diaper, Cola

Frequent Item Sets

$$k = 2$$

1	Bread, Milk
2	Bread, Diaper
3	Milk, Diaper
4	Diaper, Beer

They cannot be considered as strong rules if we set the confidence threshold to 0.8

The Apriori Algorithm

Workflow

Initialization:

- Start by identifying all frequent item sets of size $k = 1$ (individual items)

Iteration:

- Generate candidate item sets of size k by joining frequent item sets of size $k - 1$
- Determine the frequent item sets based on the support threshold.

Termination

- No further successful joins can be made, or no new frequent item sets are found

Strong Rule Generation:

- Association rules are generated by dividing the frequent item sets into two non-empty disjoint subsets, where one subset implies the other.
- The strength of these rules is then evaluated based on confidence or lift.

The Apriori Algorithm

Disadvantages

- Computationally expensive due to the need to scan the database multiple times.
- Combinatorial explosion with the increase in the number of transactions or the size of item sets.

The Apriori Algorithm

Evaluation

- Support: How frequency the items appear at the same time
- Confidence: How likely it is for Y happening when X happens
- Lift: Is X improving the appearance of Y ? (a value that is compared to 1)

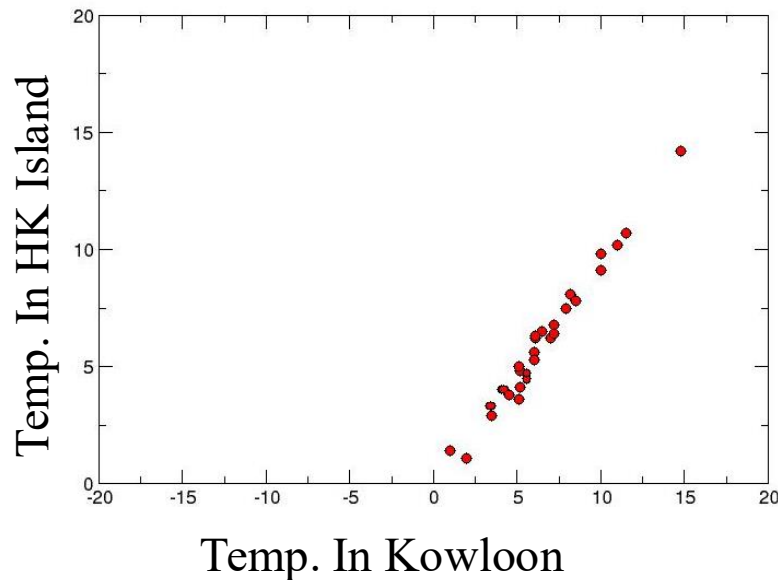
Unsupervised Learning Outline

- Clustering
 - Evaluation
 - K-Means Clustering
 - Hierarchical Clustering
 - Data Normalization
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA (Supervised Learning)

Why Principal Component Analysis is Needed

Redundancy of data features

- The handcrafted data features can always include redundant dimensions, because
 - We may tend to include as many factors as possible.

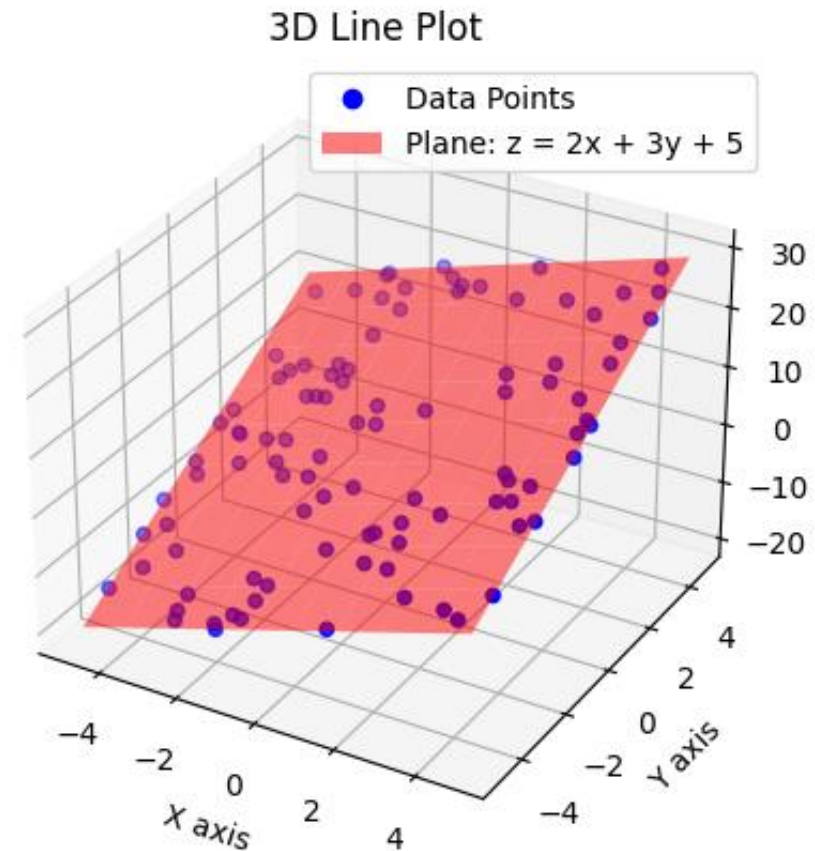


$$y = x + \text{noise}$$

Why Principal Component Analysis is Needed

Redundancy of data features

- The handcrafted data features can always include redundant dimensions, because
 - The inherent correlations between feature dimensions are not obvious

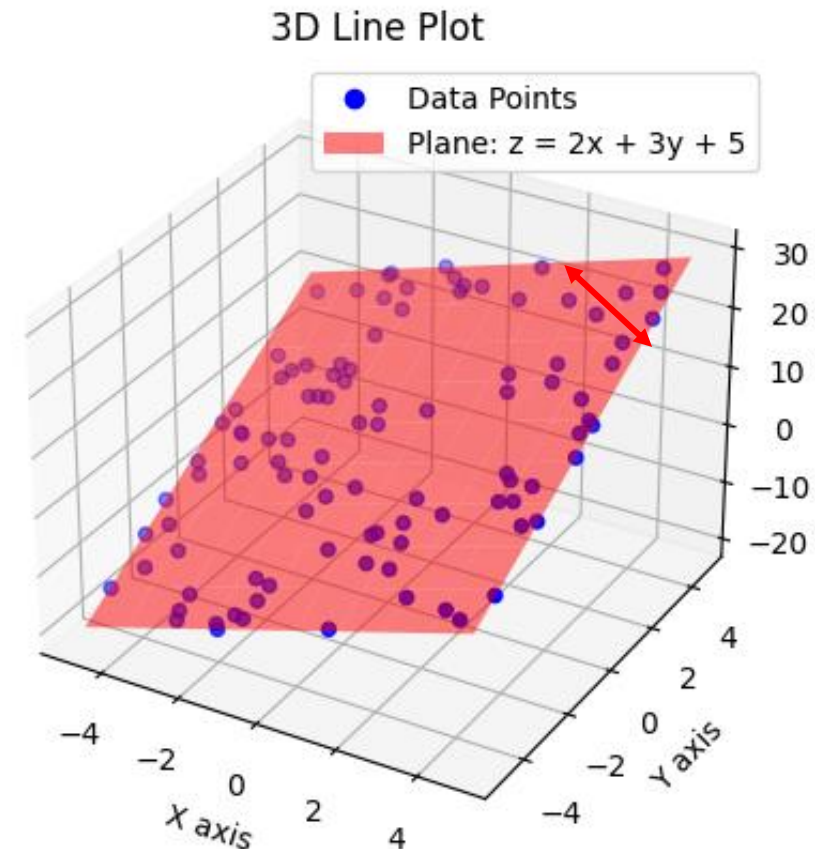


In this case, the z dimension is totally redundant because it is a linear combination of x and y .

Why Principal Component Analysis is Needed

Redundancy of data features

- How to reduce these redundant dimensions?
 - **Check the variance over different directions!**

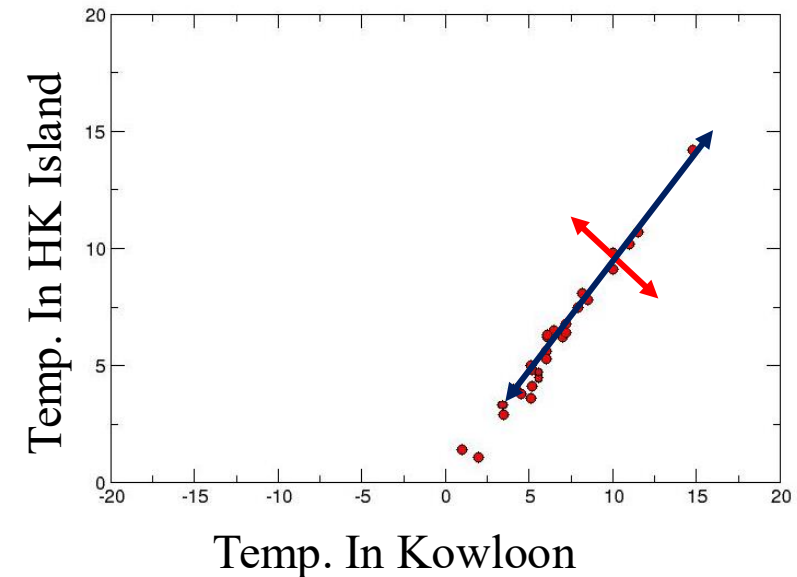


The variance over the **red** arrow is ZERO, i.e., **data would not be “expressive” in this direction.**

Why Principal Component Analysis is Needed

Redundancy of data features

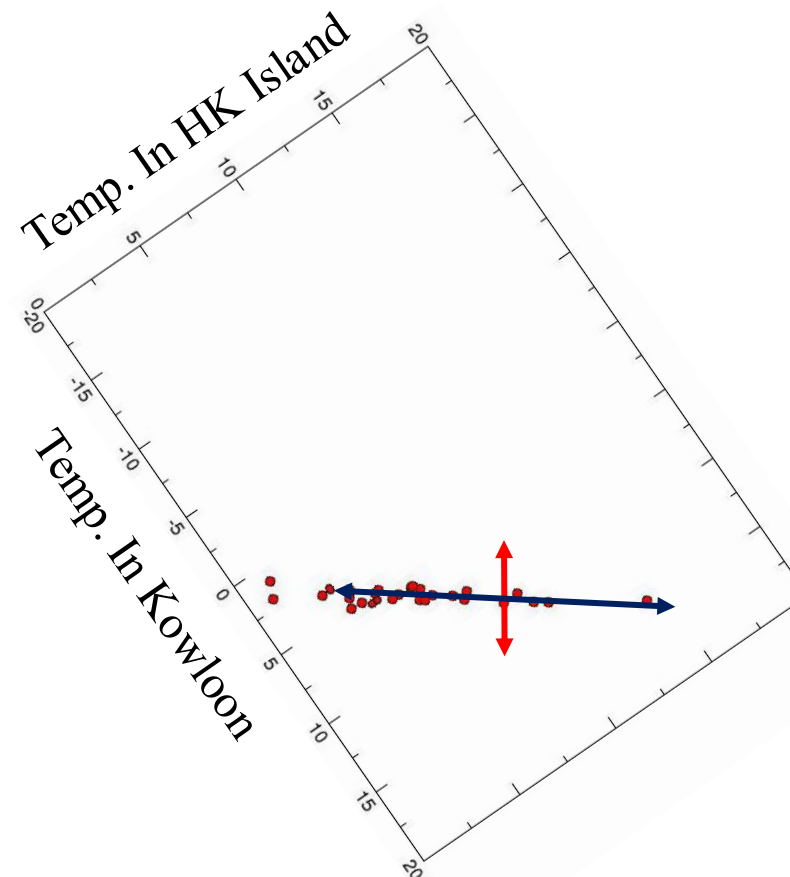
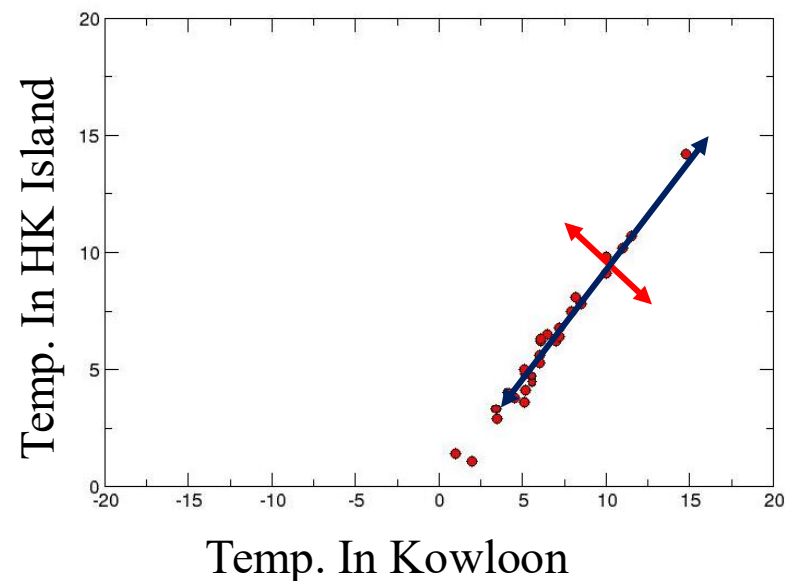
- How to reduce these redundant dimensions?
 - **Check the variance over different directions!**



The variance over the red arrow is much smaller than the variance over the blue arrow, i.e., **data would not be “expressive” in this direction.**

Why PCA is needed

Redundancy of data features

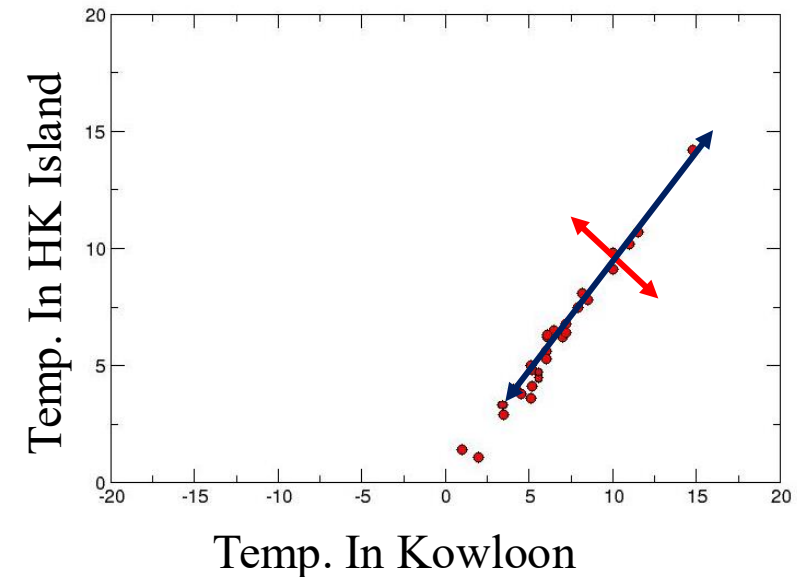


If we only **keep the direction with the major expressiveness (large variance)**, and project all data to this direction, we reduce the dimension.

Why PCA is needed

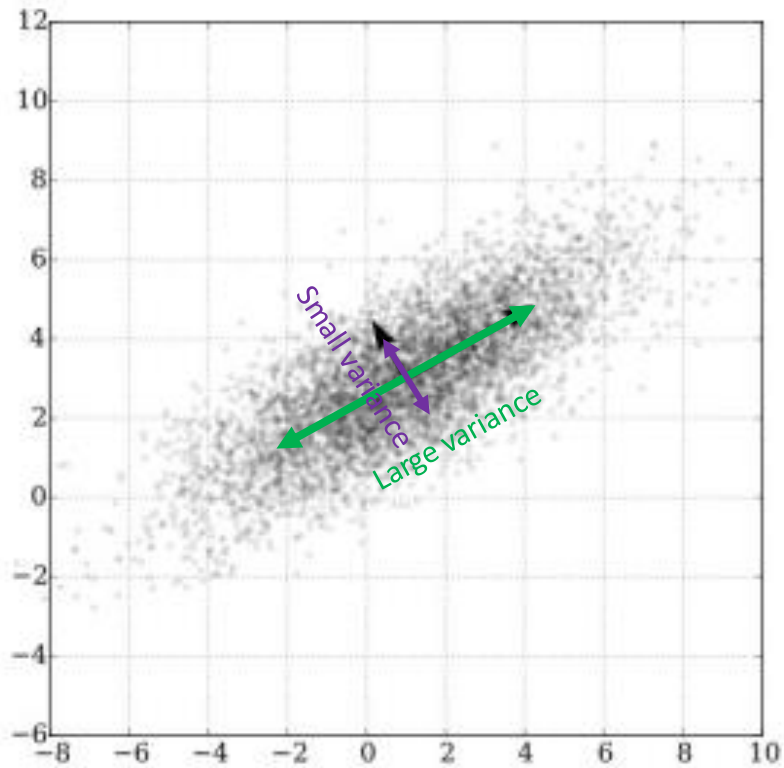
Redundancy of data features

- How to reduce these redundant dimensions?
 - To avoid information linkage, we expect different candidate directions are **orthogonal, yielding independent features**.



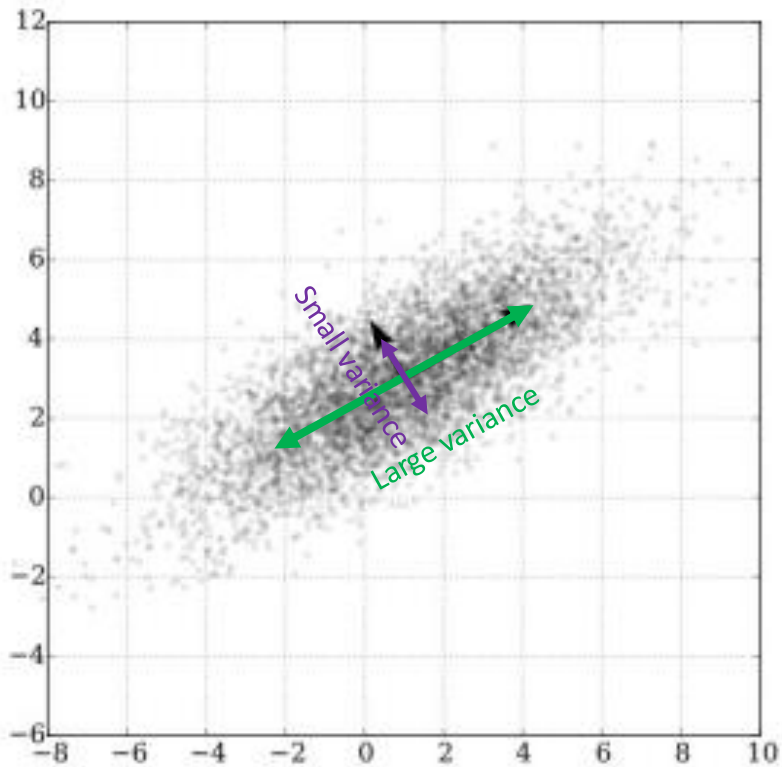
The variance over the red arrow is much smaller than the variance over the blue arrow, i.e., **data would not be “expressive” in this direction.**

Basic Ideas of PCA



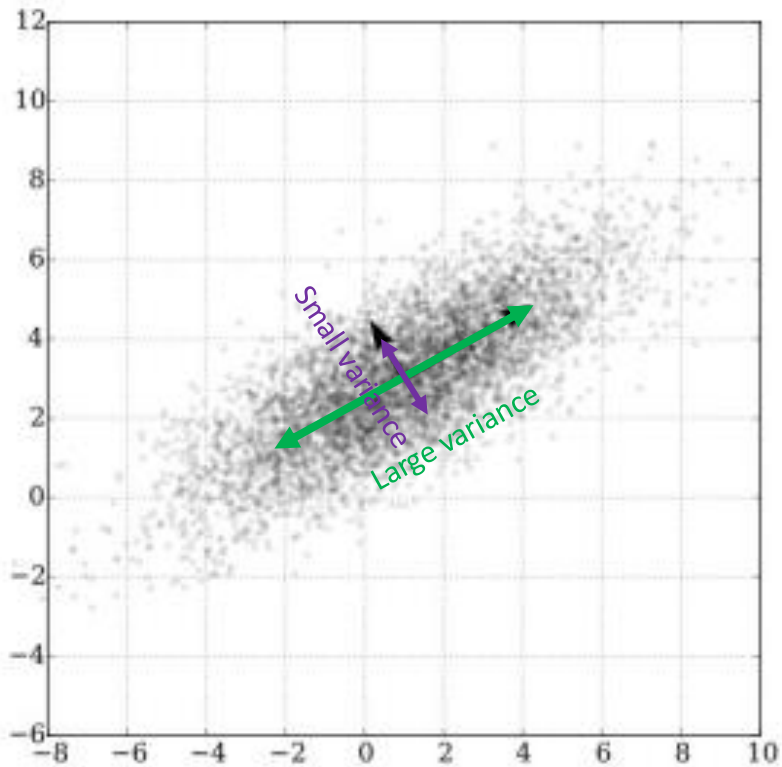
1. Identify a set of orthogonal directions
(Make sure features are independent)

Basic Ideas of PCA



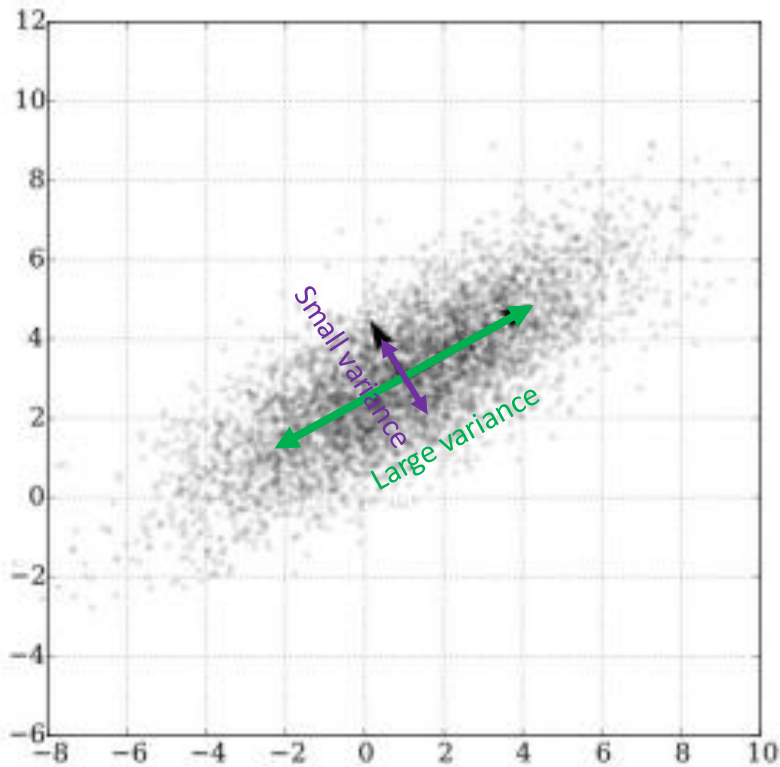
1. Identify a set of orthogonal directions
(Make sure features are independent)
2. For each direction, check the variance of the
data (Expressiveness of data)

Basic Ideas of PCA



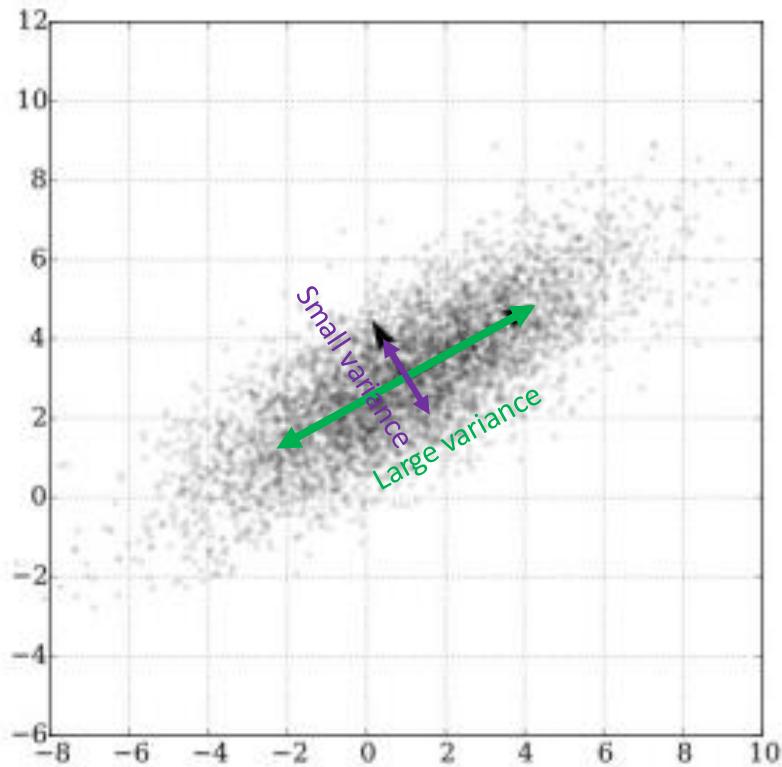
1. Identify a set of orthogonal directions (Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. Keep N directions in which most of the variances have been considered (Finding Major Information)

Basic Ideas of PCA



1. Identify a set of orthogonal directions (Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. Keep N directions in which most of the variances have been considered (Finding Major Information)
4. Discard the other directions

Basic Ideas of PCA



1. Identify a set of orthogonal directions (Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. Keep N directions in which most of the variances have been considered (Finding Major Information)
4. Discard the other directions
5. Use K directions as the new axes and transform data to the new coordinates.

Calculations of PCA

Basic Ideas

1. Identify a set of orthogonal directions (Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. Keep K directions in which most of the variances have been considered (Finding Major Information)
4. Discard the other directions
5. Use K directions as the new axes and transform data to the new coordinates.

Calculations

Exactly what eigenvalue analysis is doing!

- Center the data ($\mathbf{x}_n = \mathbf{x}_n - \boldsymbol{\mu}$, $\boldsymbol{\mu} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}_n$ from each raw data point)
- Compute $D \times D$ **covariance matrix** $\mathbf{S}$, D is the feature dimension

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X}$$

$\mathbf{X}$ is arranged as $N \times D$

We first calculate the variance of the data, and will decompose it into orthogonal directions soon

Calculations of PCA

Basic Ideas

1. Identify a set of orthogonal directions (Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. Keep K directions in which most of the variances have been considered (Finding Major Information)
4. Discard the other directions
5. Use K directions as the new axes and transform data to the new coordinates.

Calculations

Exactly what eigenvalue analysis is doing!

- Eigen decomposition of the covariance matrix $\mathbf{S}$

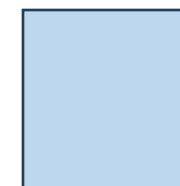
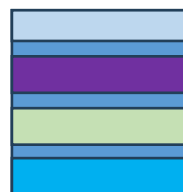
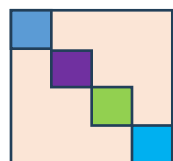
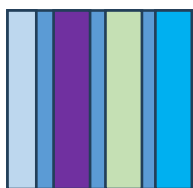
$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T = \sum_{k=1}^D \lambda_k \mathbf{u}_k \mathbf{u}_k^T$$

The eigenvalue decomposition finds D orthogonal directions (eigenvectors) $\mathbf{u}_k$, and also gives the span of these directions (eigenvalues) λ_k

Calculations of PCA

Further explanation of eigenvalue decomposition

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T = \sum_{k=1}^D \lambda_k \mathbf{u}_k \mathbf{u}_k^T$$



$$\|\mathbf{u}_k\| = 1$$

What the covariance matrix will look like if all data is in this direction?

Calculations of PCA

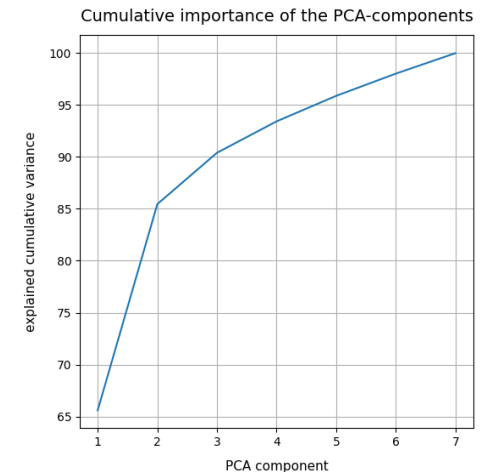
Basic Ideas

1. Identify a set of orthogonal directions
(Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. **Keep K directions in which most of the variances have been considered (Finding Major Information)**
4. **Discard the other directions**
5. Use K directions as the new axes and transform data to the new coordinates.

Calculations

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T = \sum_{k=1}^D \lambda_k \mathbf{u}_k \mathbf{u}_k^T$$

Take top $K < D$ leading eigenvectors with eigenvalues decreasing. K can be determined by accumulating the sum of eigenvalues.



Calculations of PCA

Basic Ideas

1. Identify a set of orthogonal directions
(Make sure features are independent)
2. For each direction, check the variance of the data (Expressiveness of data)
3. Keep K directions in which most of the variances have been considered (Finding Major Information)
4. Discard the other directions
5. Use K directions as the new axes and transform data to the new coordinates.

Calculations

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X} = \mathbf{U} \mathbf{\Lambda} \mathbf{U}^T = \sum_{k=1}^D \lambda_k \mathbf{u}_k \mathbf{u}_k^T$$

Take top $K < D$ leading eigenvectors with eigenvalues decreasing. K can be determined by accumulating the sum of eigenvalues.

$$\mathbf{z}_n \approx \mathbf{W}_K^T \mathbf{x}_n$$

$\mathbf{W}_K = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_K]$ is the “projection matrix” of size $D \times K$. $\mathbf{x}_n$ is centred.

The inner product calculates the coordinate after projection when $\|\mathbf{u}_k\| = 1$

Calculations of PCA

Summary of Calculations

$$\mathbf{x}_n = \mathbf{x}_n - \boldsymbol{\mu}$$

$$\mathbf{S} = \frac{1}{N} \mathbf{X}^T \mathbf{X}$$

$$\mathbf{S} = \mathbf{U} \boldsymbol{\Lambda} \mathbf{U}^T = \sum_{k=1}^D \lambda_k \mathbf{u}_k \mathbf{u}_k^T$$

Take top $K < D$ leading eigenvectors with eigenvalues decreasing. K can be determined by accumulating the sum of eigenvalues.

$$\mathbf{z}_n \approx \mathbf{W}_K^T \mathbf{x}_n$$

1. Center the data

2. Compute the covariance matrix

3. Eigenvalue decomposition

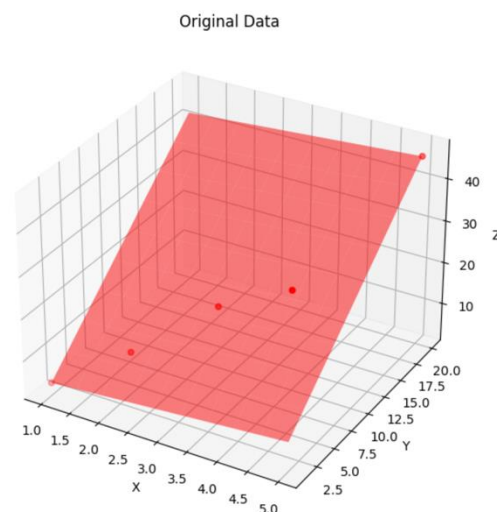
4. Choose dominant eigenvectors and form transformation matrix

5. Perform transformation

Examples of PCA

x	y	z
0.49	1.46	4.41
2.15	-0.62	1.9
1.03	3.92	9.86
0.45	4.64	10.72
-0.76	-1.17	-2.09

$$z = 1 * x + 2 * y + 1$$



1. Center the data

x	y	z
-0.18	-0.19	-0.55
1.48	-2.27	-3.06
0.36	2.27	4.9
-0.22	2.99	5.76
-1.43	-2.81	-7.05

2. Compute the covariance matrix

$$\mathbf{S} = \frac{1}{5} \mathbf{X}^T \mathbf{X}$$

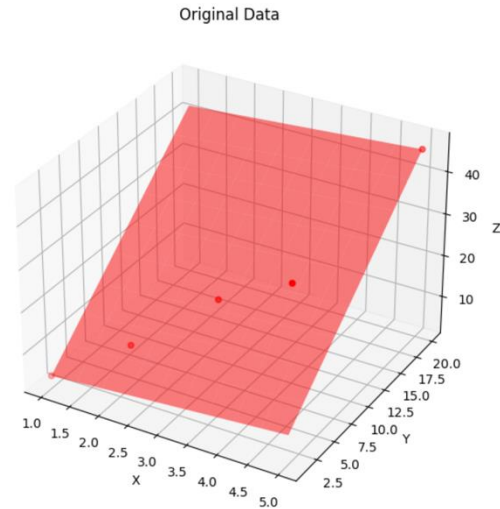
$$\begin{bmatrix} 0.89 & 0.17 & 1.23 \\ 0.17 & 5.44 & 11.05 \\ 1.23 & 11.05 & 23.33 \end{bmatrix}$$

Examples of PCA

3. Eigenvalue decomposition $\mathbf{S} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$

x	y	z
1	1	4
2	4	11
3	8	20
4	10	25
5	20	46

$$z = 1 * x + 2 * y + 1$$



$$\mathbf{S} = \begin{bmatrix} 0.89 & 0.17 & 1.23 \\ 0.17 & 5.44 & 11.05 \\ 1.23 & 11.05 & 23.33 \end{bmatrix}$$

$$\mathbf{\Lambda} = \begin{bmatrix} 2.87 \times 10^1 & 0 & 0 \\ 0 & 1.01 & 0 \\ 0 & 0 & 1.01 \times 10^{-15} \end{bmatrix}$$

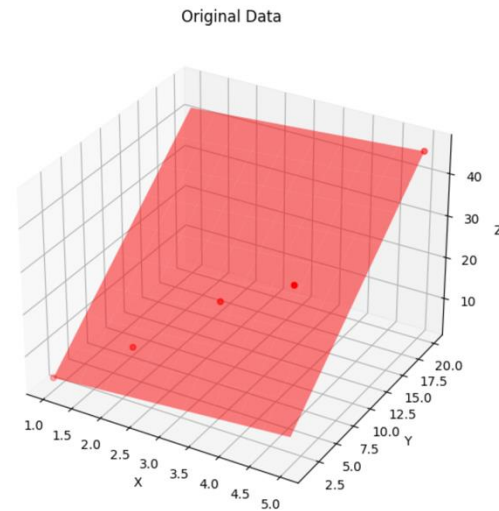
$$\mathbf{U} = \begin{bmatrix} -0.04 & -0.91 & -0.41 \\ -0.43 & 0.39 & -0.82 \\ -0.90 & -0.14 & 0.41 \end{bmatrix}$$

Examples of PCA

4. Choose dominant eigenvectors and form transformation matrix

x	y	z
1	1	4
2	4	11
3	8	20
4	10	25
5	20	46

$$z = 1 * x + 2 * y + 1$$



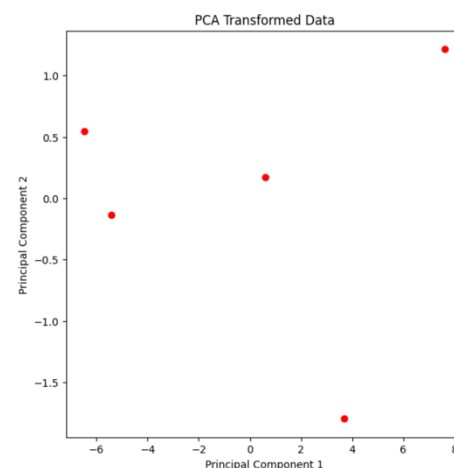
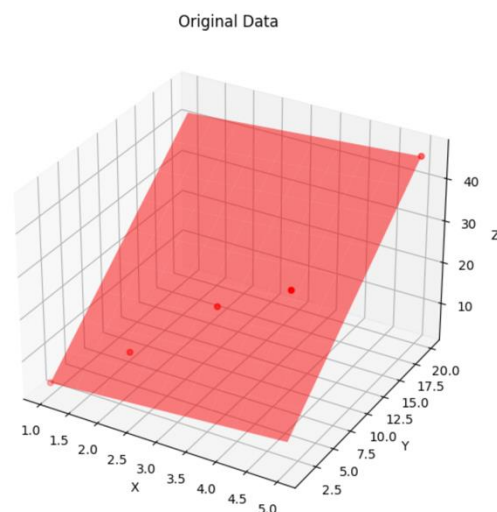
$$U = \begin{bmatrix} -0.04 & -0.91 & -0.41 \\ -0.43 & 0.39 & -0.82 \\ -0.90 & -0.14 & 0.41 \end{bmatrix}$$

$$W = \begin{bmatrix} -0.04 & -0.91 \\ -0.43 & 0.39 \\ -0.90 & -0.14 \end{bmatrix}$$

Examples of PCA

x	y	z
1	1	4
2	4	11
3	8	20
4	10	25
5	20	46

$$z = 1 * x + 2 * y + 1$$



5. Perform transformation

$$\begin{bmatrix} -0.04 & -0.91 \\ -0.43 & 0.39 \\ -0.90 & -0.14 \end{bmatrix}^T *$$

x	y	z
-0.18	-0.19	-0.55
1.48	-2.27	-3.06
0.36	2.27	4.9
-0.22	2.99	5.76
-1.43	-2.81	-7.05



x	y
0.59	0.17
3.67	-1.8
-5.41	-0.14
-6.47	0.55
7.63	1.22

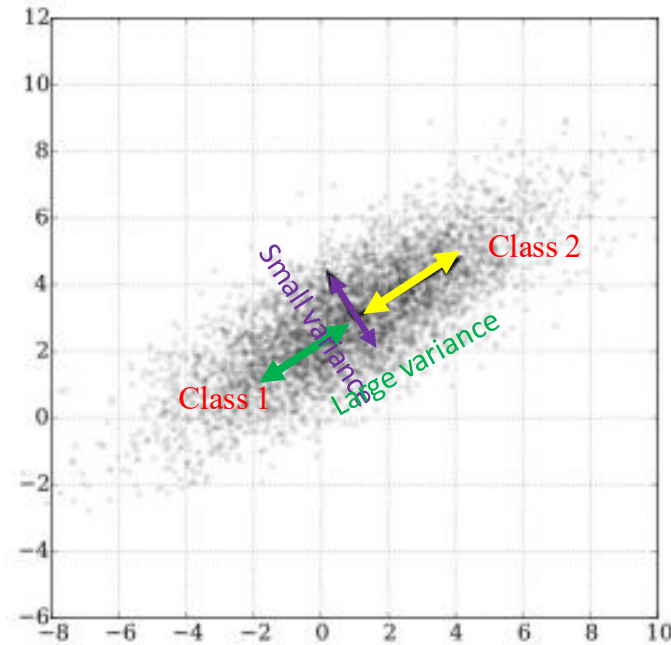
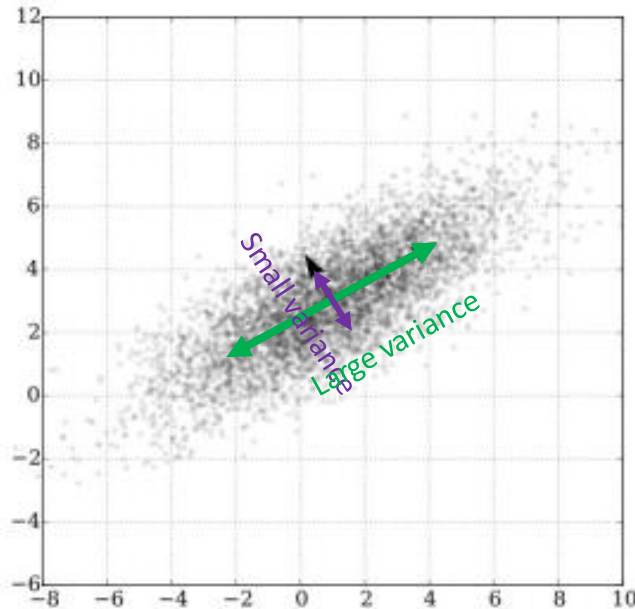
Unsupervised Learning Outline

- Clustering
 - Evaluation
 - K-Means Clustering
 - Hierarchical Clustering
 - Data Normalization
- Association Rule Learning
 - The Apriori Algorithm
 - Evaluation
- Dimensionality Reduction
 - PCA
 - LDA (Supervised Learning)

Why Linear Discriminant Analysis Needed?

Linear Discriminant Analysis (LDA) not only **reduces dimensionality** but also aims to **ensure good class-separation**.

- Requires class labels, making it supervised learning.



When class labels are given, LDA further imposes class separation in the resulting data.

Calculations of LDA

To ensure class separation, similar to K-Means, we first define two matrices

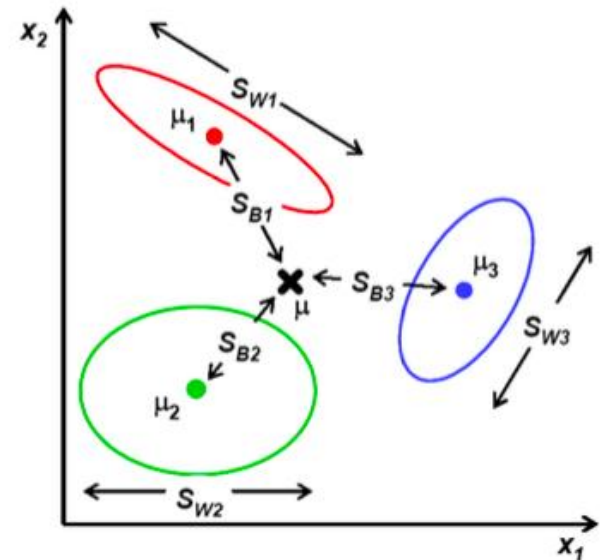
- Within-Class Scatter Matrix: Compactness of the classes
 - Supposing we have c classes, for each class sample set D_i , we compute the covariance matrix

$$S_{W_i} = \sum_{x \in D_i} (x - \mu_i)(x - \mu_i)^T$$

Where μ_i is the mean vector of the in-class samples

- Considering all classes, we define the **within-class scatter matrix**:

$$S_W = \sum_{i=1}^c S_{W_i}$$



Calculations of LDA

To ensure class separation, similar to K-Means, we first define two matrices

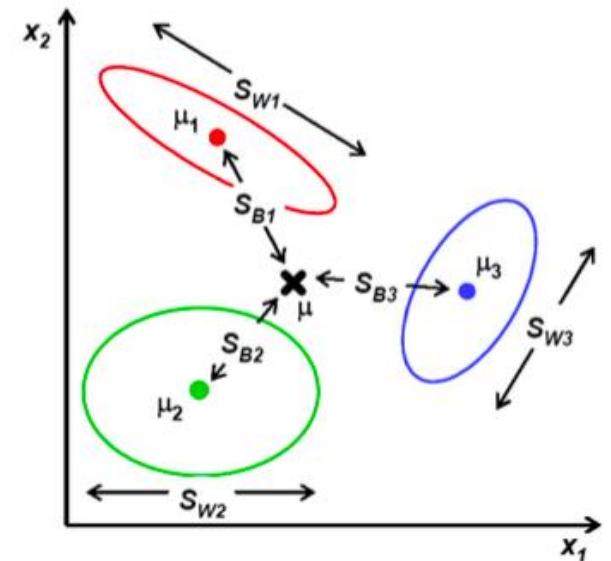
- Between-Class Scatter Matrix: Separation of the classes
 - How far each class centroid is from the overall mean of the data?
 - Supposing we have c classes, for each class, we compute a scatter matrix with respect to the data centroid

$$S_{B_i} = (\mu_i - \mu)(\mu_i - \mu)^T$$

Where μ is the mean vector of all samples

- Considering all classes, we define the **between-class scatter matrix**:

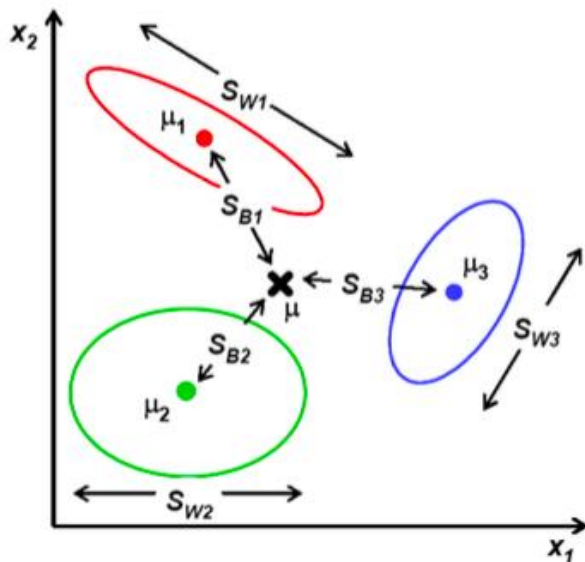
$$S_B = \sum_{i=1}^c N_i (\mu_i - \mu)(\mu_i - \mu)^T$$



Calculations of LDA

The optimization goal of LDA

Find a matrix W that linearly transforms the data to a new space where the ratio of the determinant of the between-class scatter matrix to the determinant of the within-class scatter matrix is maximized.



Take top $K < D$ leading eigenvectors with eigenvalues decreasing. K can be determined by accumulating the sum of eigenvalues.

$$\mathbf{z}_n \approx \mathbf{W}_K^T \mathbf{x}_n$$

$\mathbf{W}_K = [\mathbf{u}_1, \mathbf{u}_2, \dots, \mathbf{u}_K]$ is the “projection matrix” of size $D \times K$

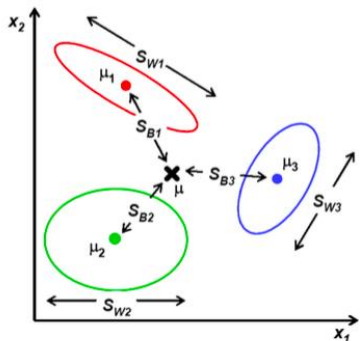
Matrix Transformation in PCA

$$\mathbf{W}^* = \operatorname{argmax}_W \left(\frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}} \right)$$

Optimization Goal of LDA

Calculations of LDA

The optimization goal of LDA



$$W^* = \operatorname{argmax}_W \left(\frac{\mathbf{w}^T \mathbf{S}_b \mathbf{w}}{\mathbf{w}^T \mathbf{S}_w \mathbf{w}} \right)$$

Optimization Goal of LDA

Mathematically, this problem can be solved using generalized eigenvalue decomposition

$$\mathbf{S}_B \mathbf{v} = \lambda \mathbf{S}_W \mathbf{v}$$

i.e.,

$$\mathbf{S}_W^{-1} \mathbf{S}_B \mathbf{v} = \lambda \mathbf{v}$$

Sort the eigenvalues, the eigenvectors corresponding to the d dominant eigenvalues form the columns of W .

At most, we can choose $c - 1$ dominant eigenvalues because only $c - 1$ axes are needed to define separation hyperplanes.

Calculations of LDA

Projection of LDA

With W , the data samples are projected by $\mathbf{z}_n \approx W\mathbf{x}_n$

Summary of Calculations

$$S_W = \sum_{i=1}^c S_{W_i}$$

$$S_B = \sum_{i=1}^c N_i(\mu_i - \mu)(\mu_i - \mu)^T$$

Projection of LDA
With W , the data samples are projected by $\mathbf{z}_n \approx W\mathbf{x}_n$
Summary of Calculations

The eigenvectors corresponding to the $c - 1$ dominant eigenvalues form the columns of W

$$\mathbf{z}_n \approx W\mathbf{x}_n$$

1. Compute Within-Class Scatter Matrix

2. Compute Between-Class Scatter Matrix

3. Generalized Eigenvalue decomposition

4. Choose dominant eigenvectors and form transformation matrix

5. Perform transformation